

BICCAD

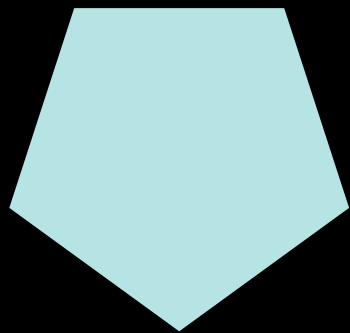


research & early development

Юра Сарапулов

Введение в глубокое обучение

Классическое обучение с учителем

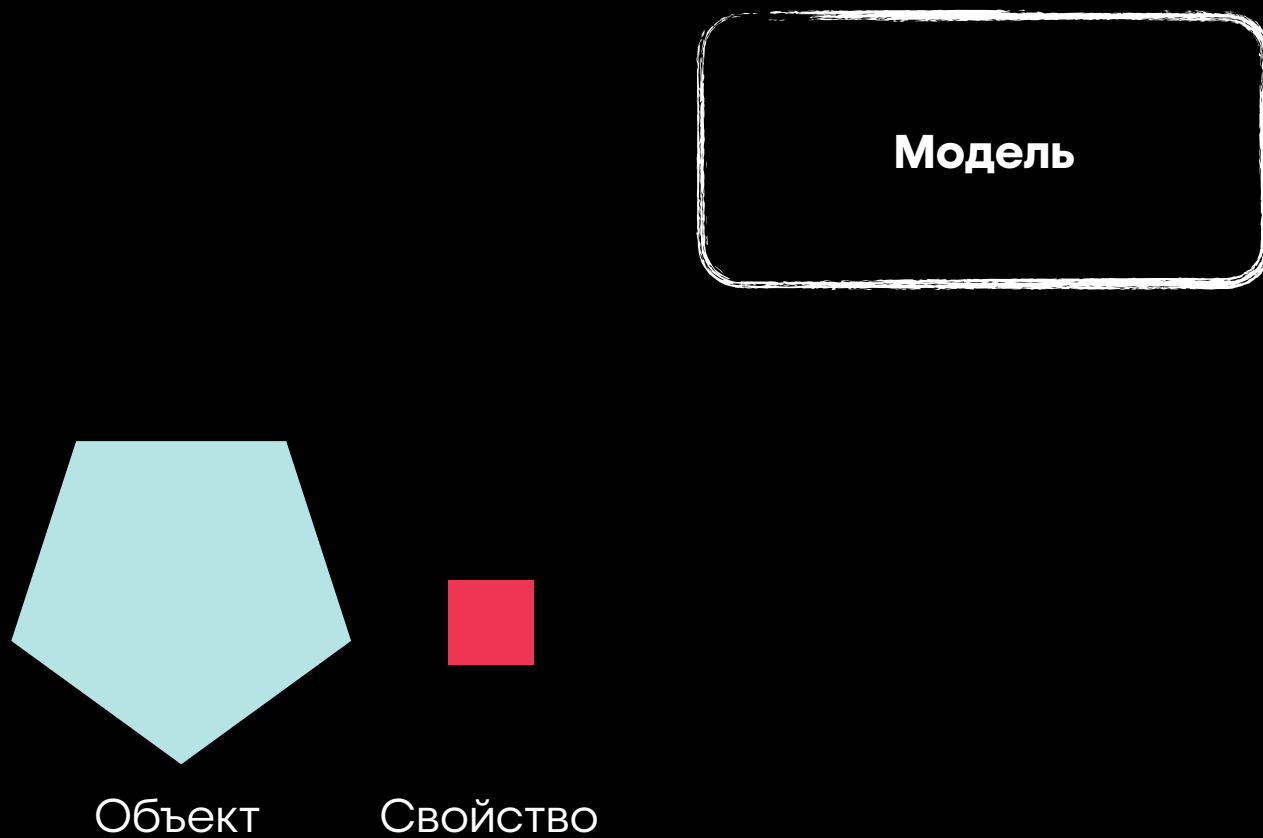


Объект

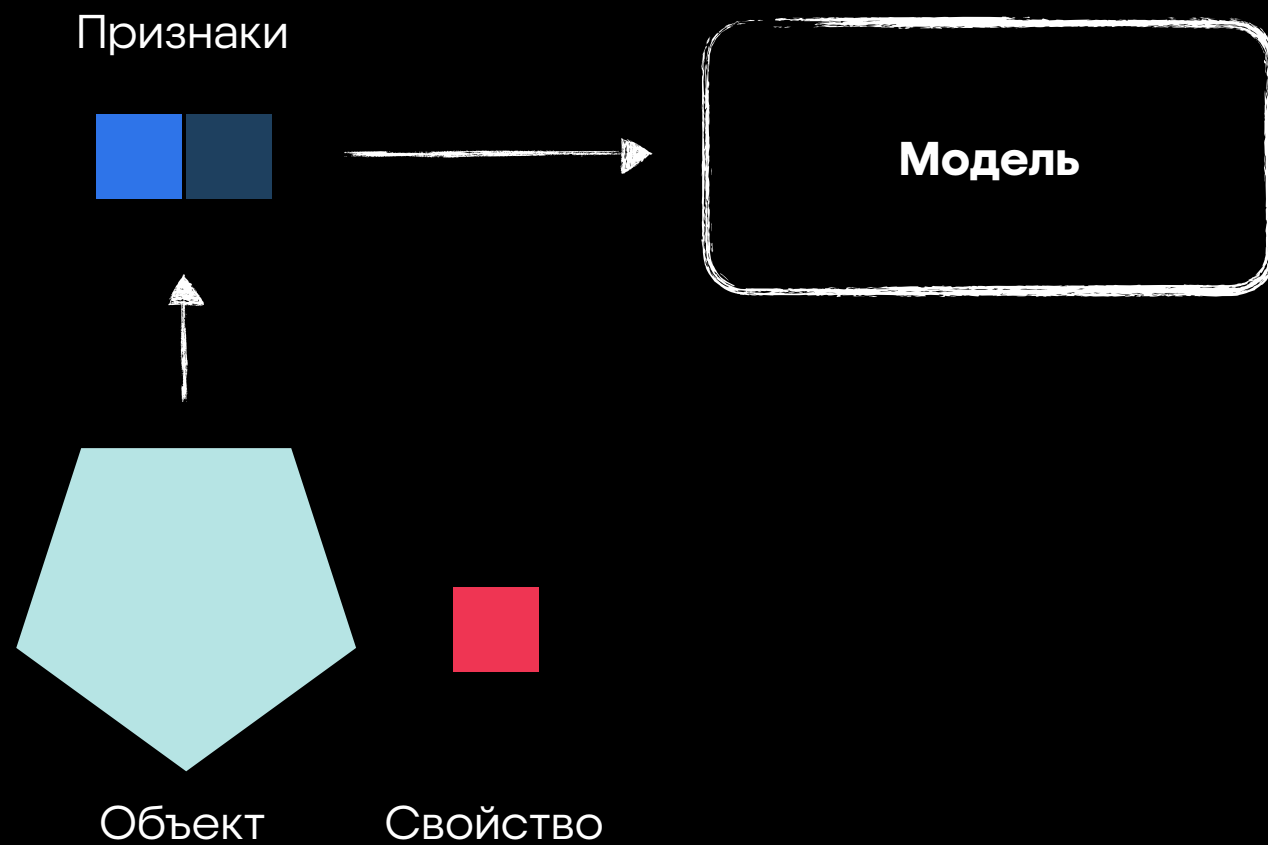


Свойство

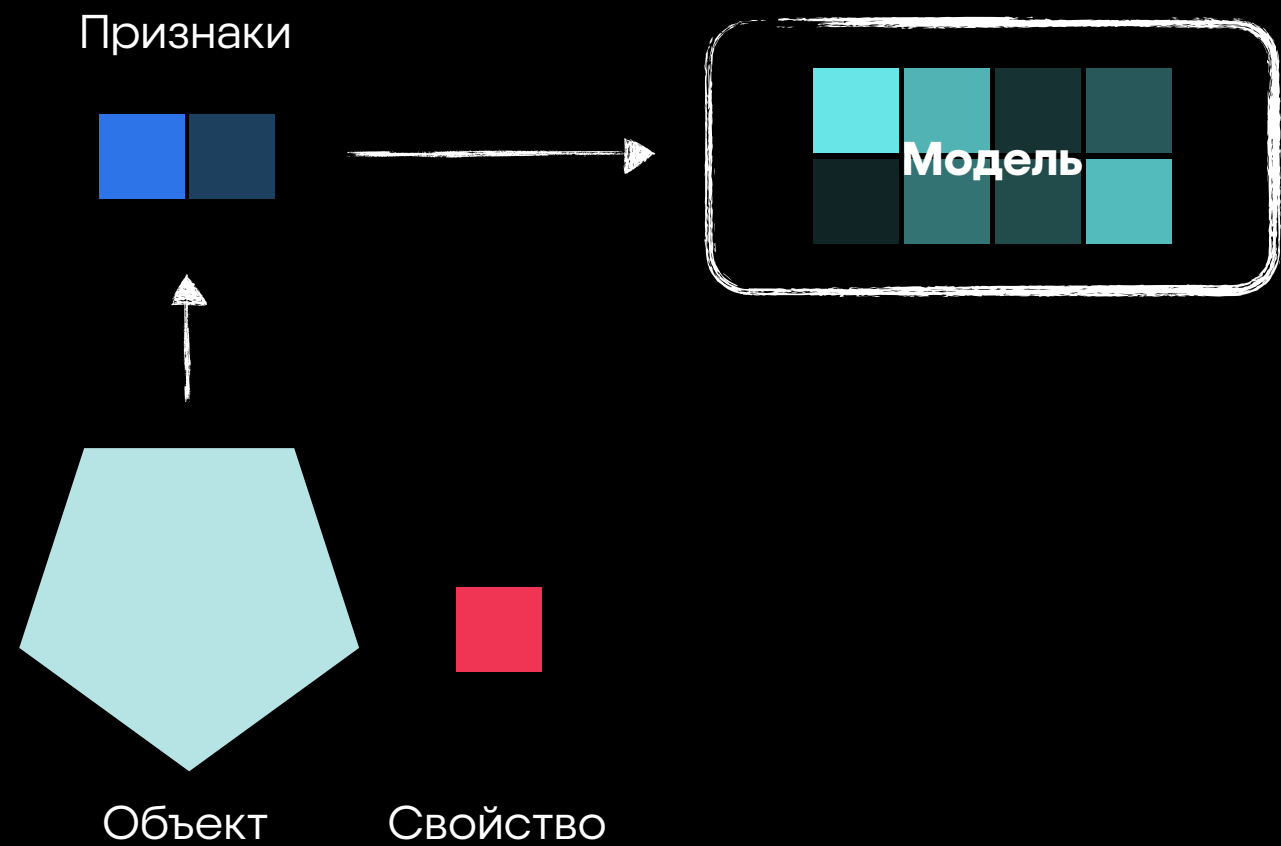
Классическое обучение с учителем



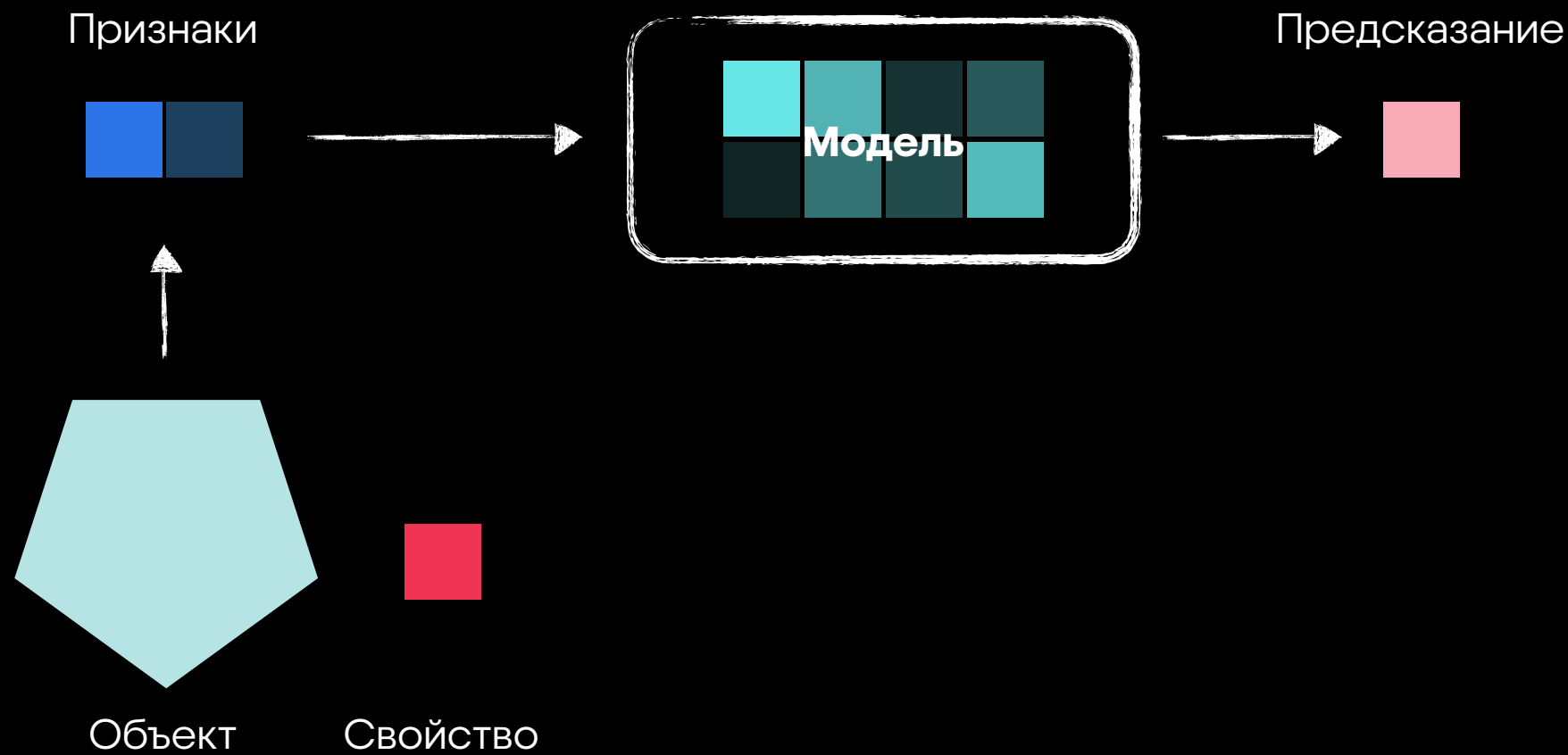
Классическое обучение с учителем



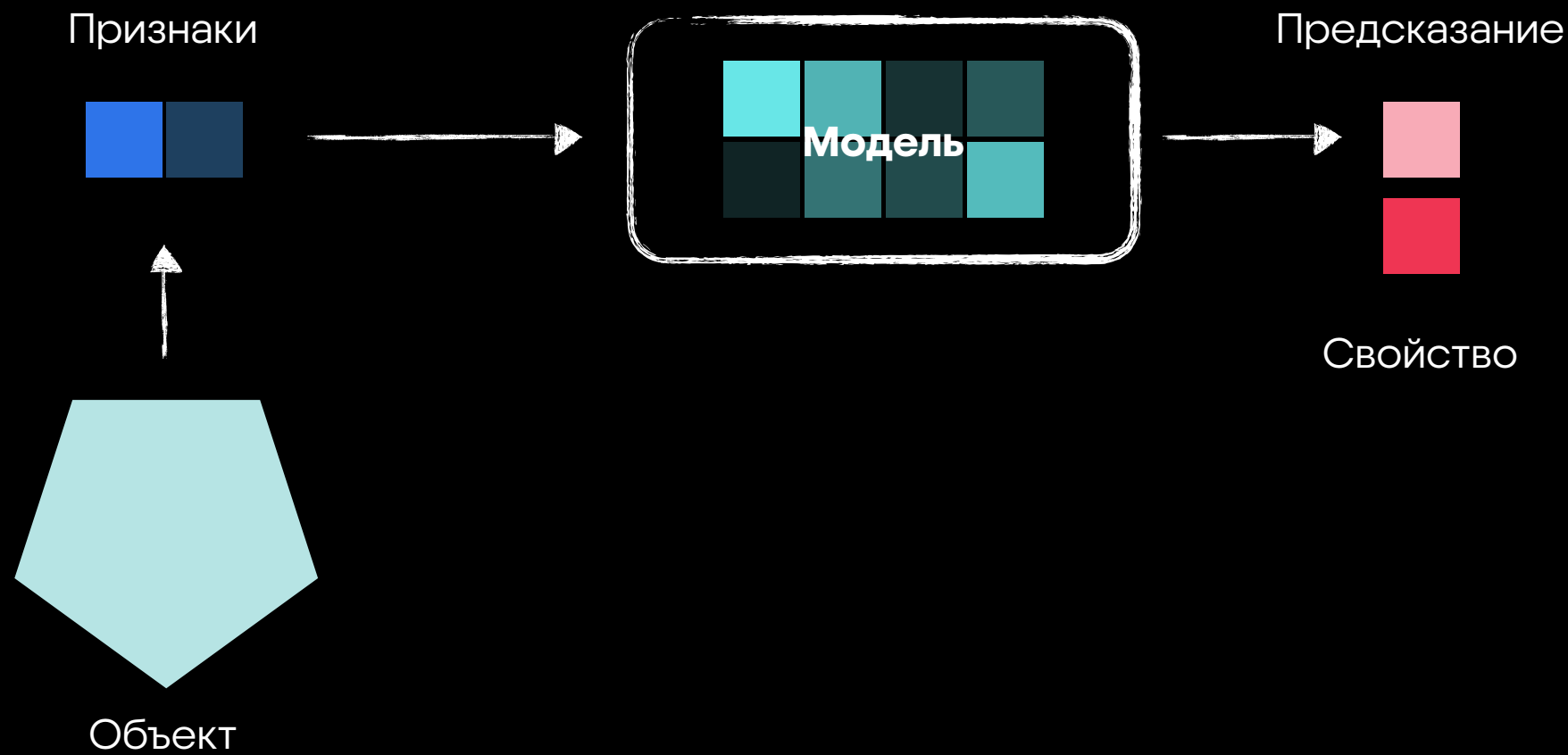
Классическое обучение с учителем



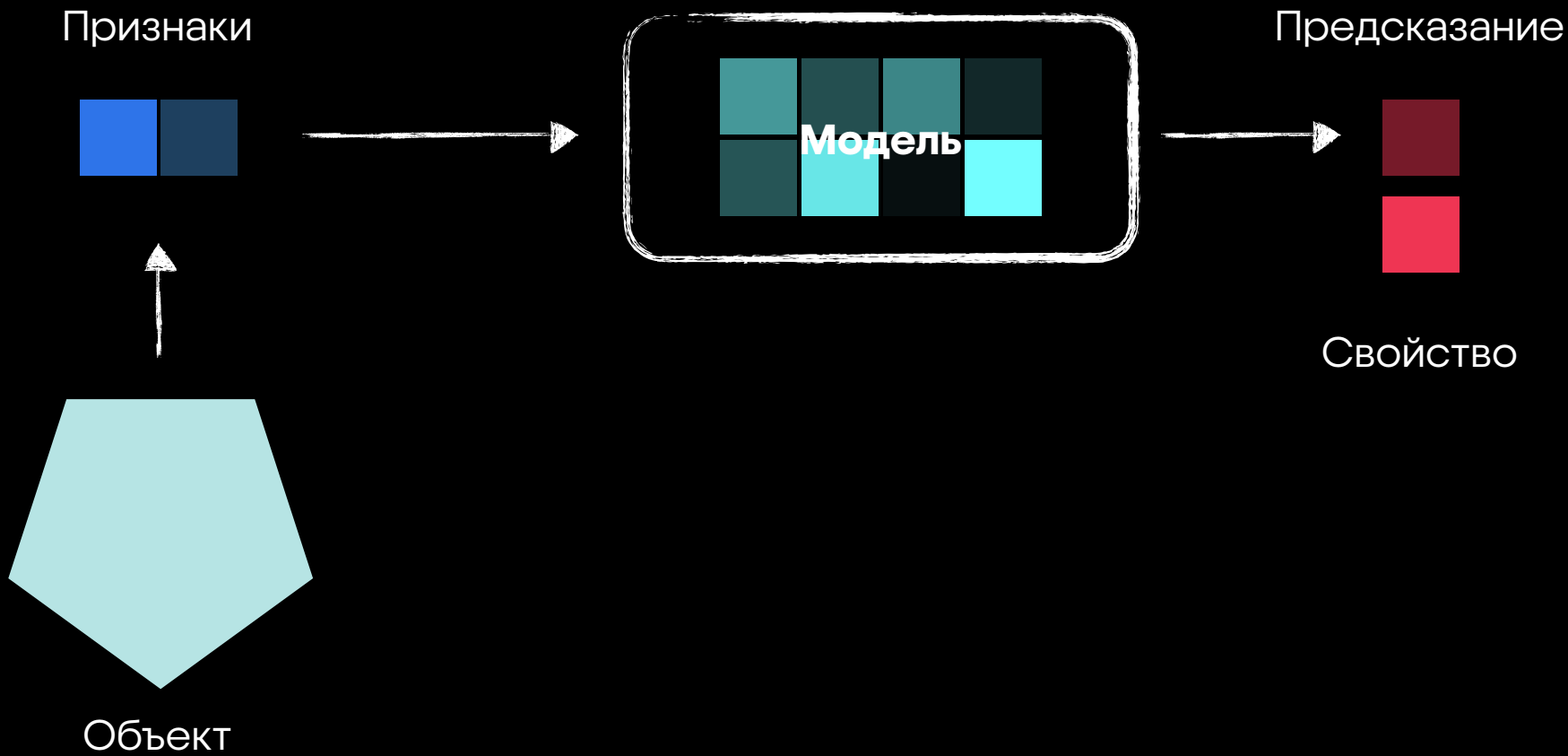
Классическое обучение с учителем



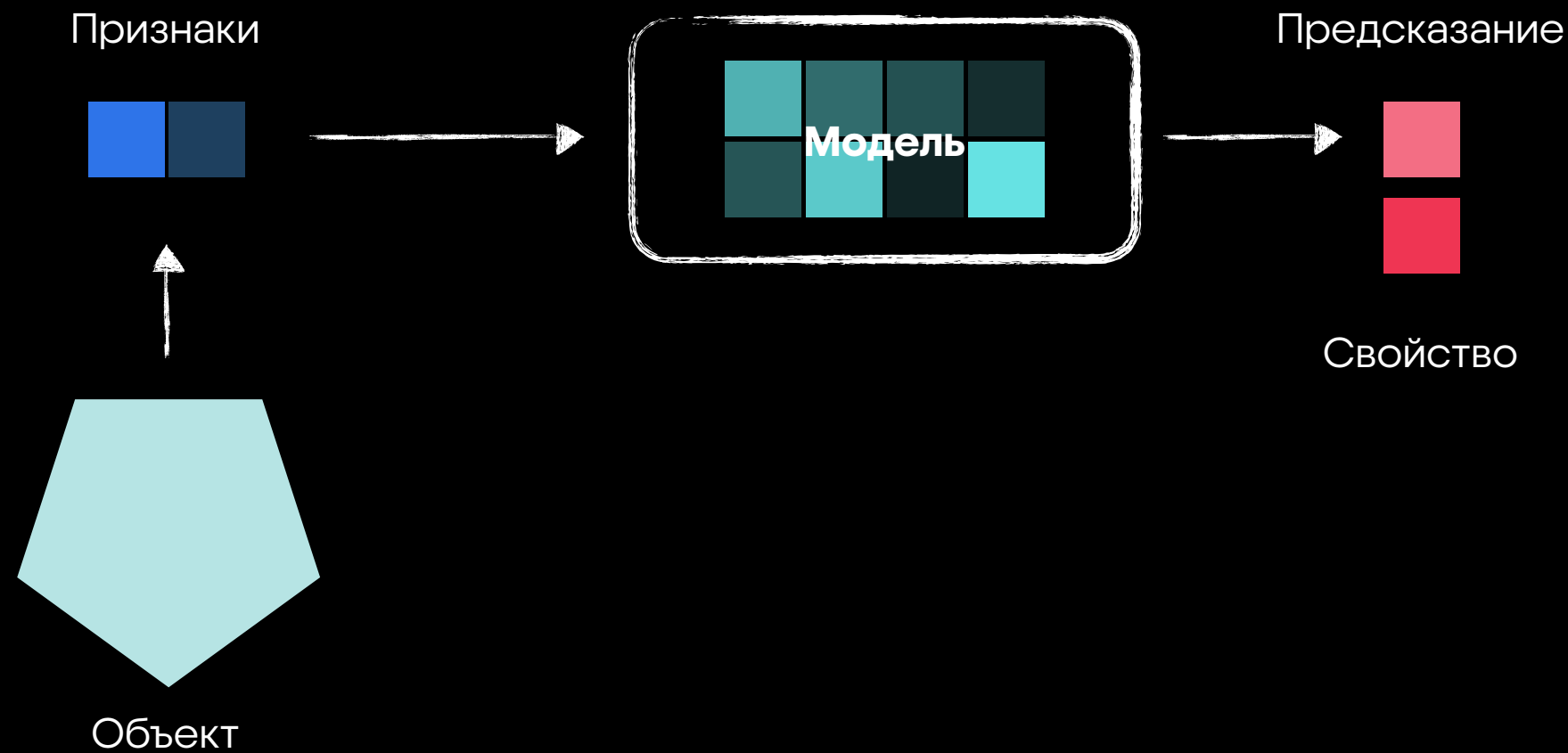
Классическое обучение с учителем



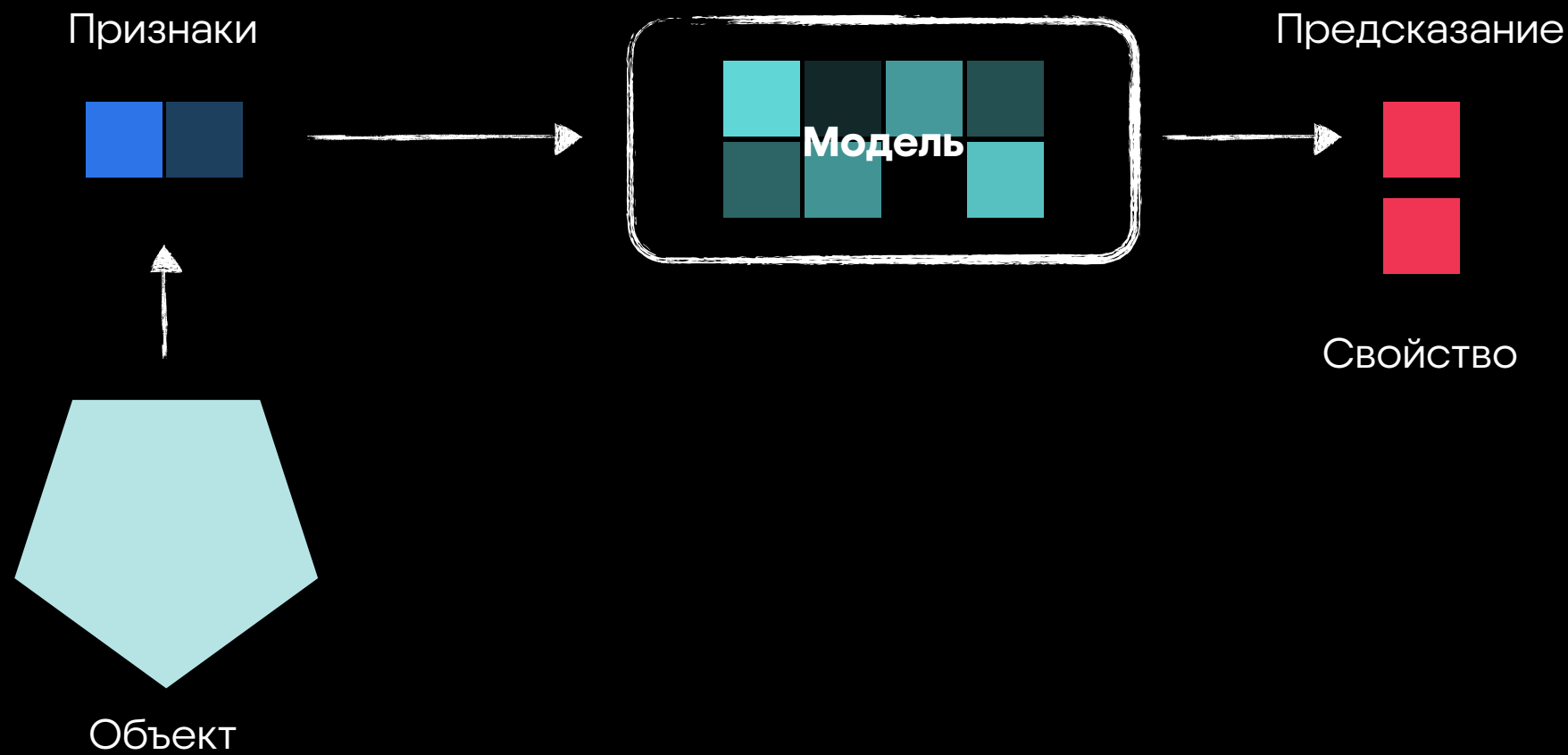
Классическое обучение с учителем



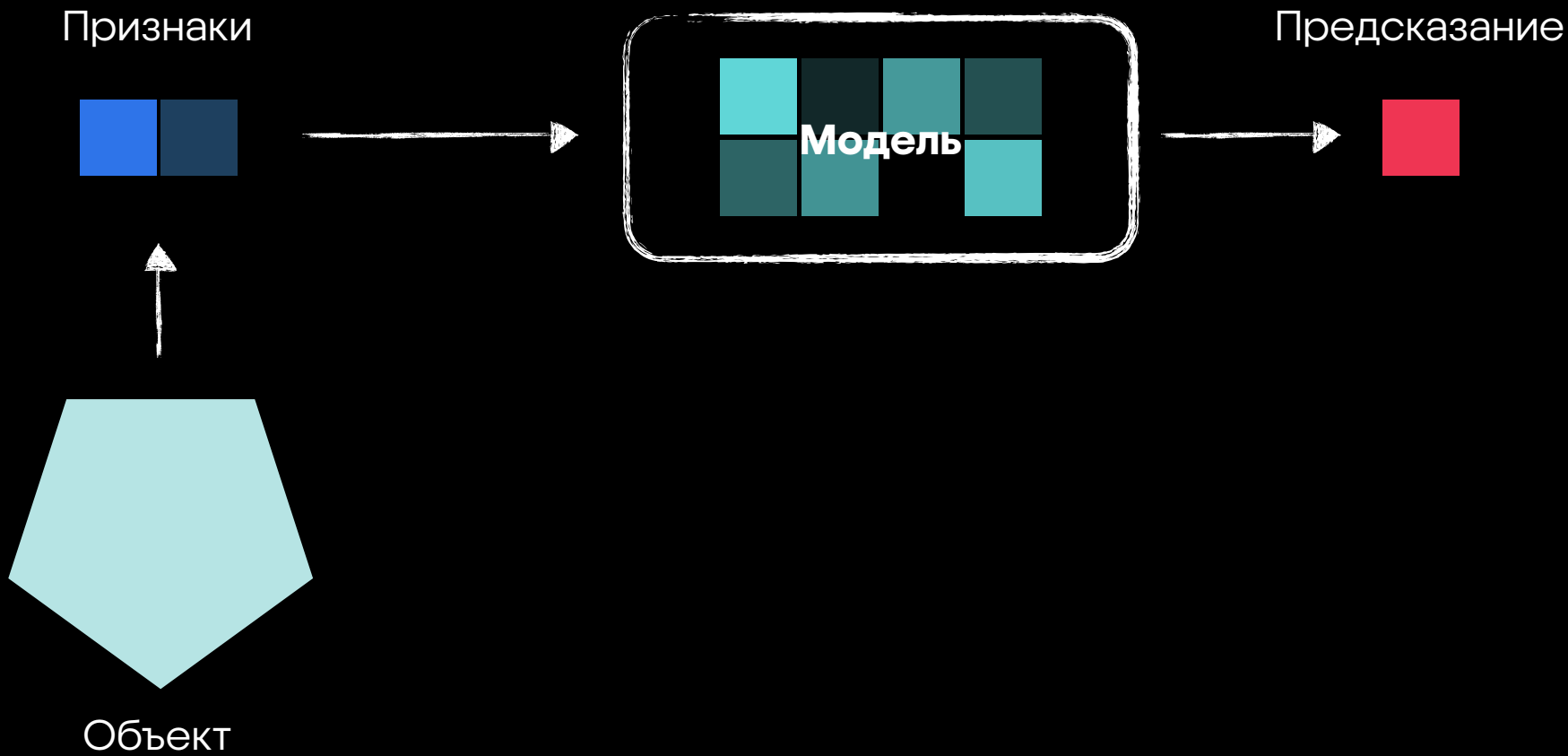
Классическое обучение с учителем



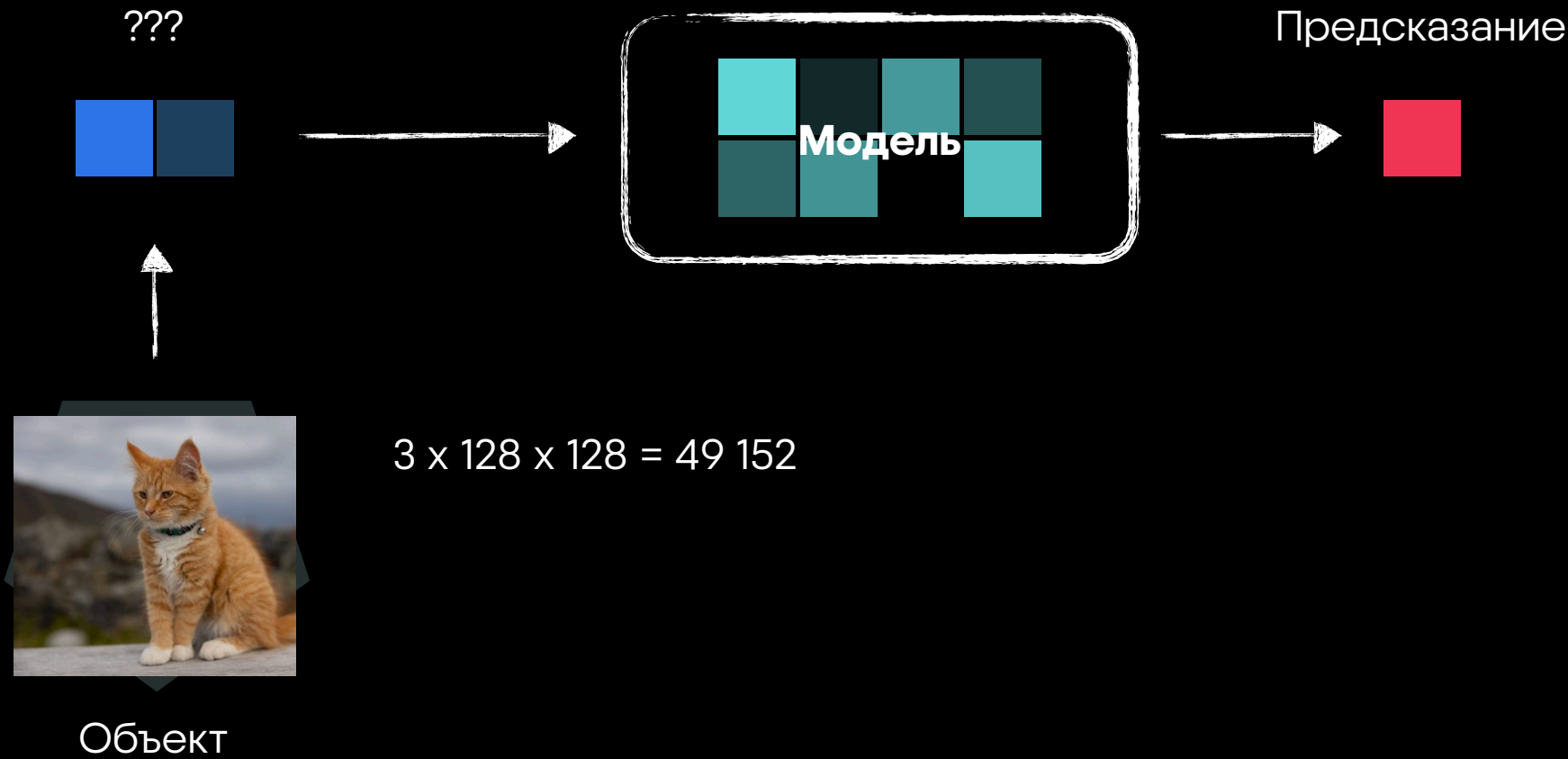
Классическое обучение с учителем



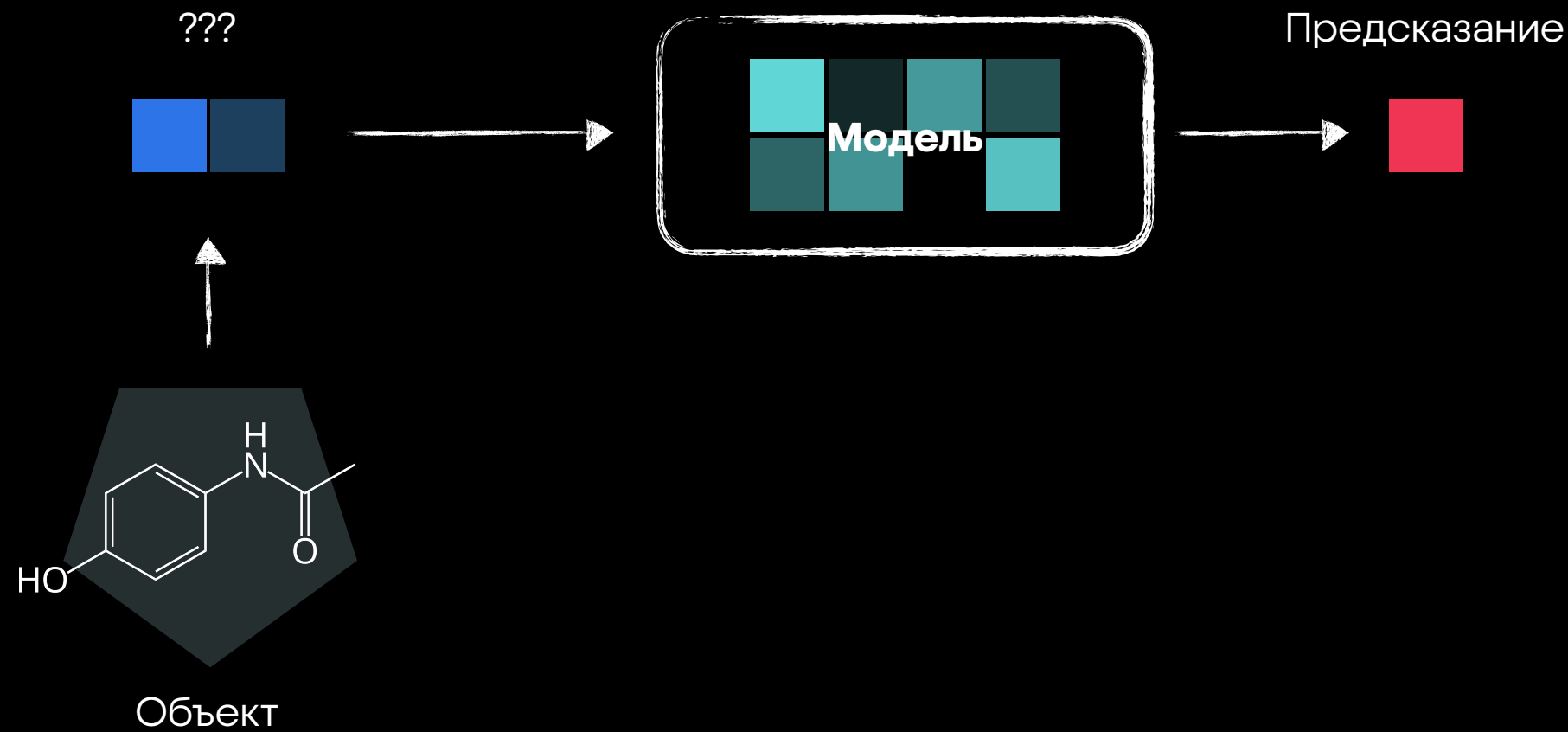
Сложные объекты



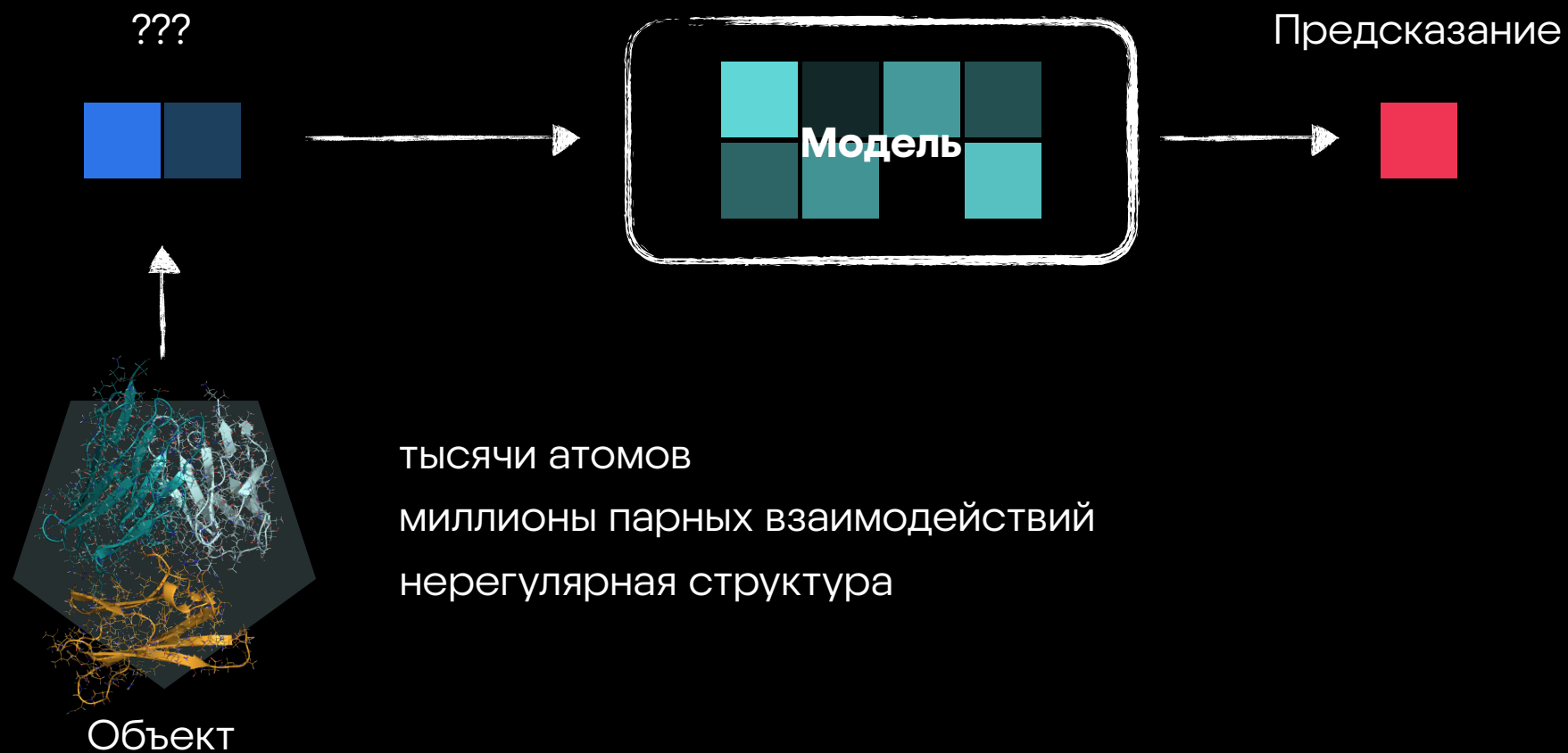
Сложные объекты



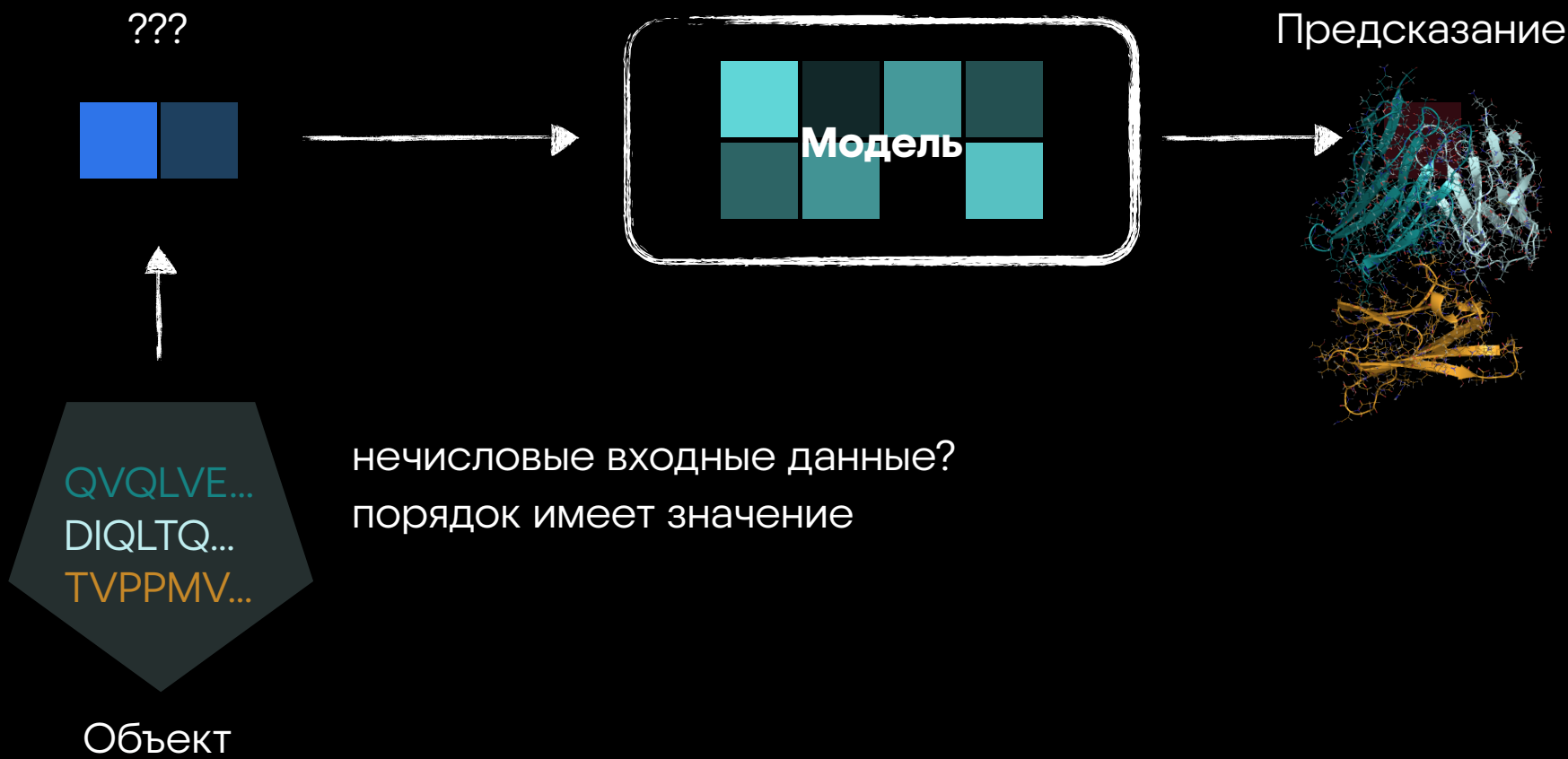
Сложные объекты



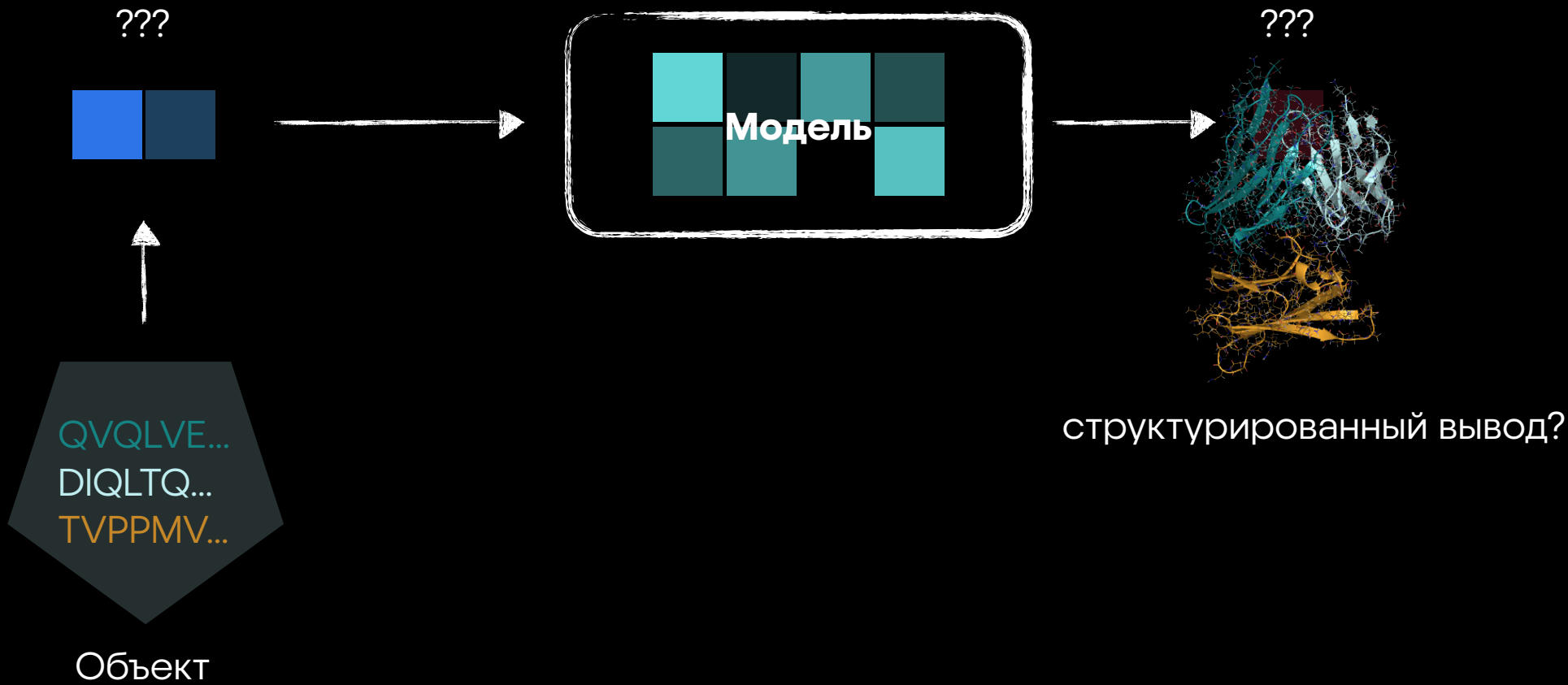
Сложные объекты



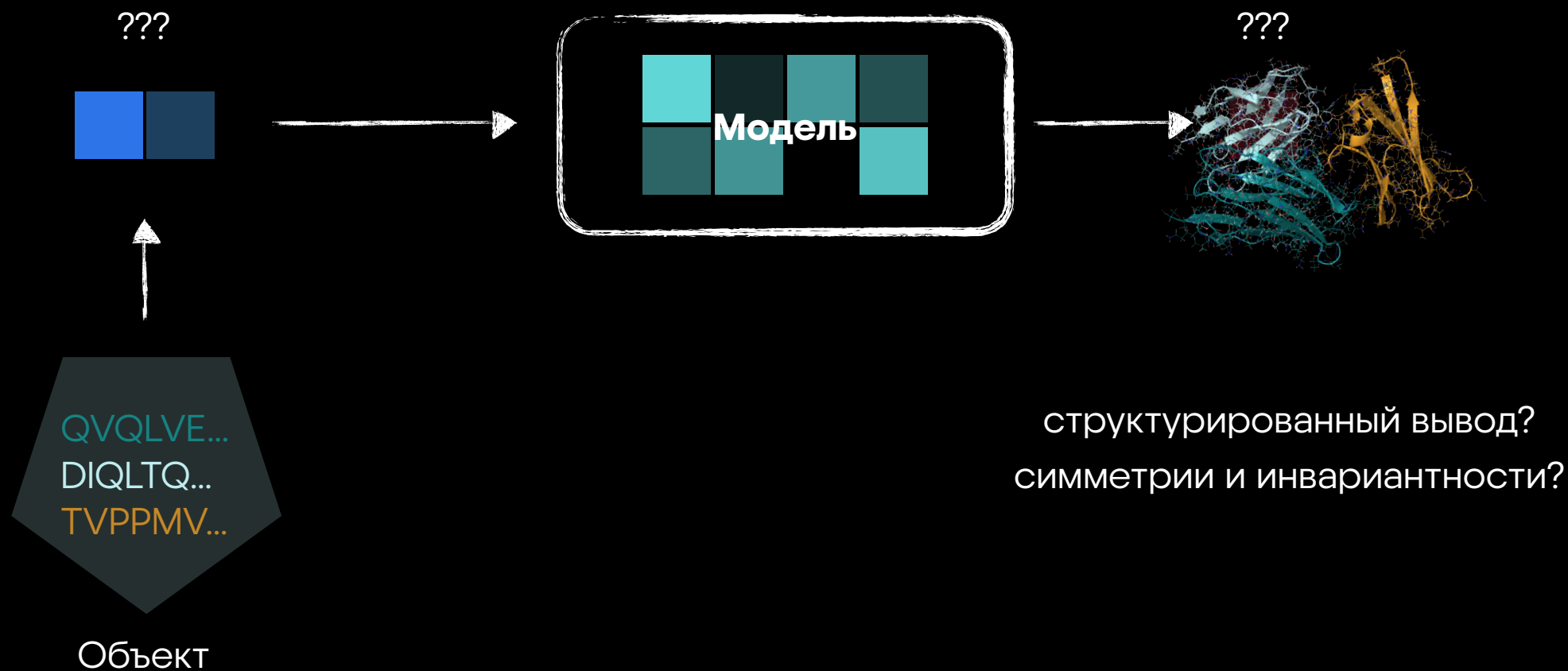
Сложные объекты



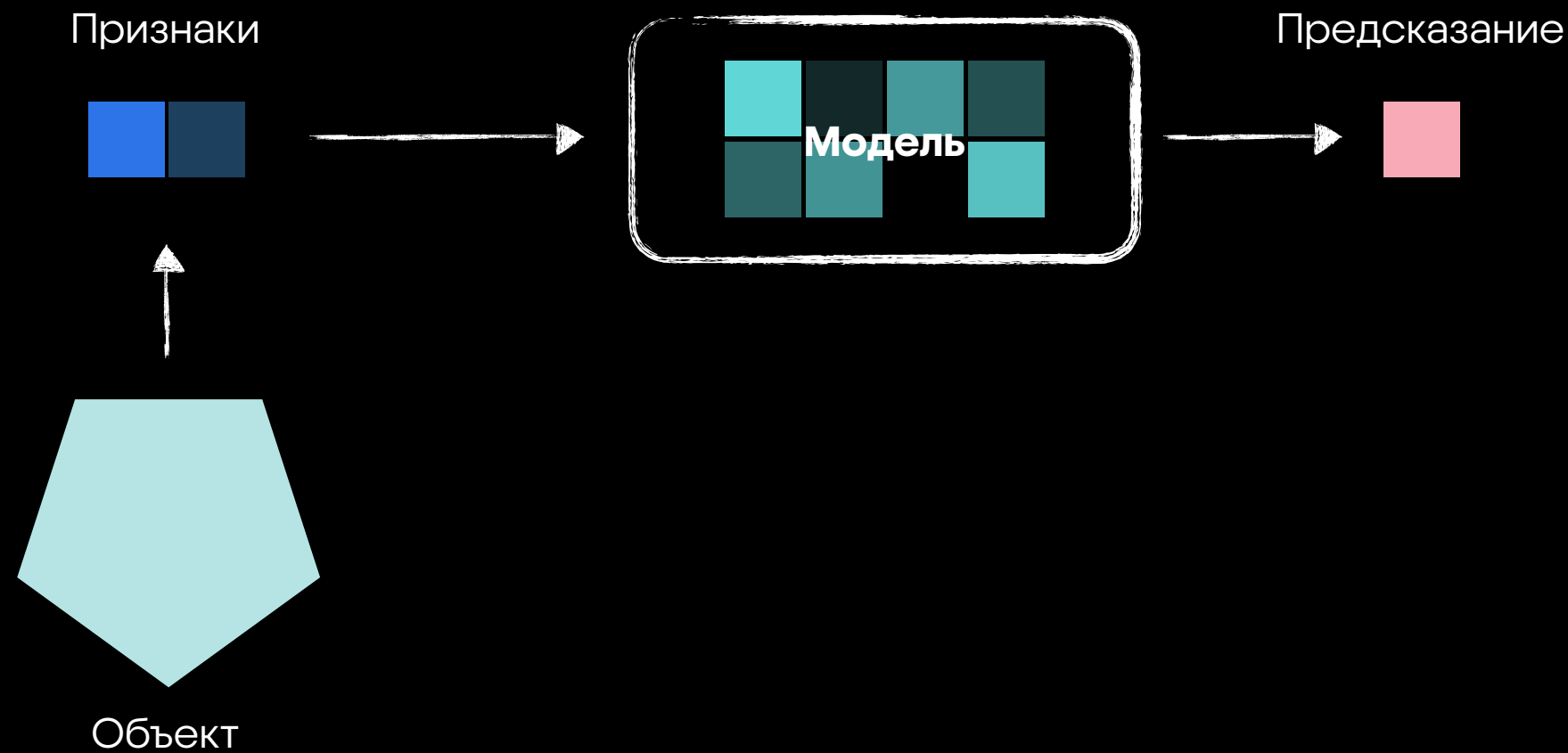
Сложные объекты



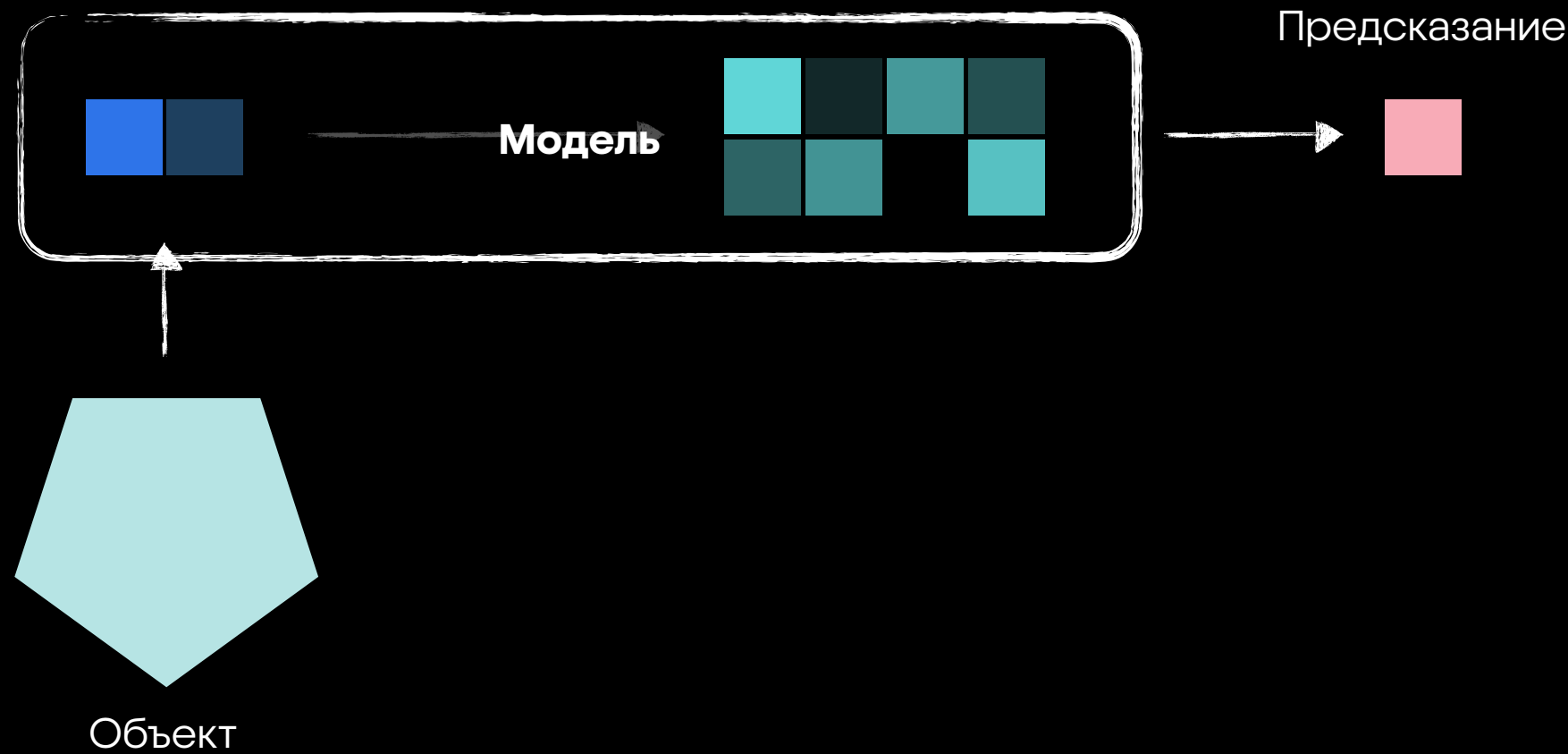
Сложные объекты



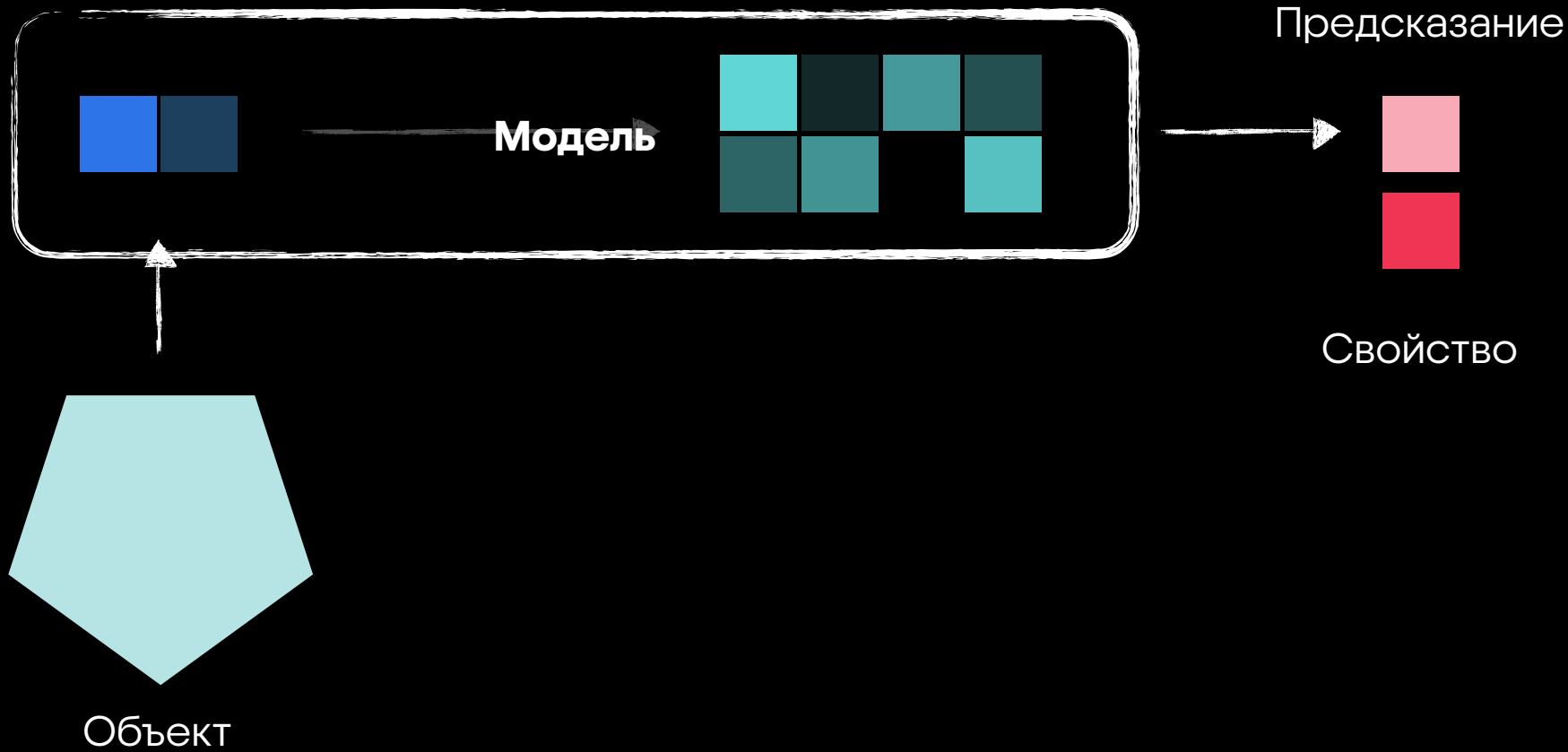
Сложные объекты



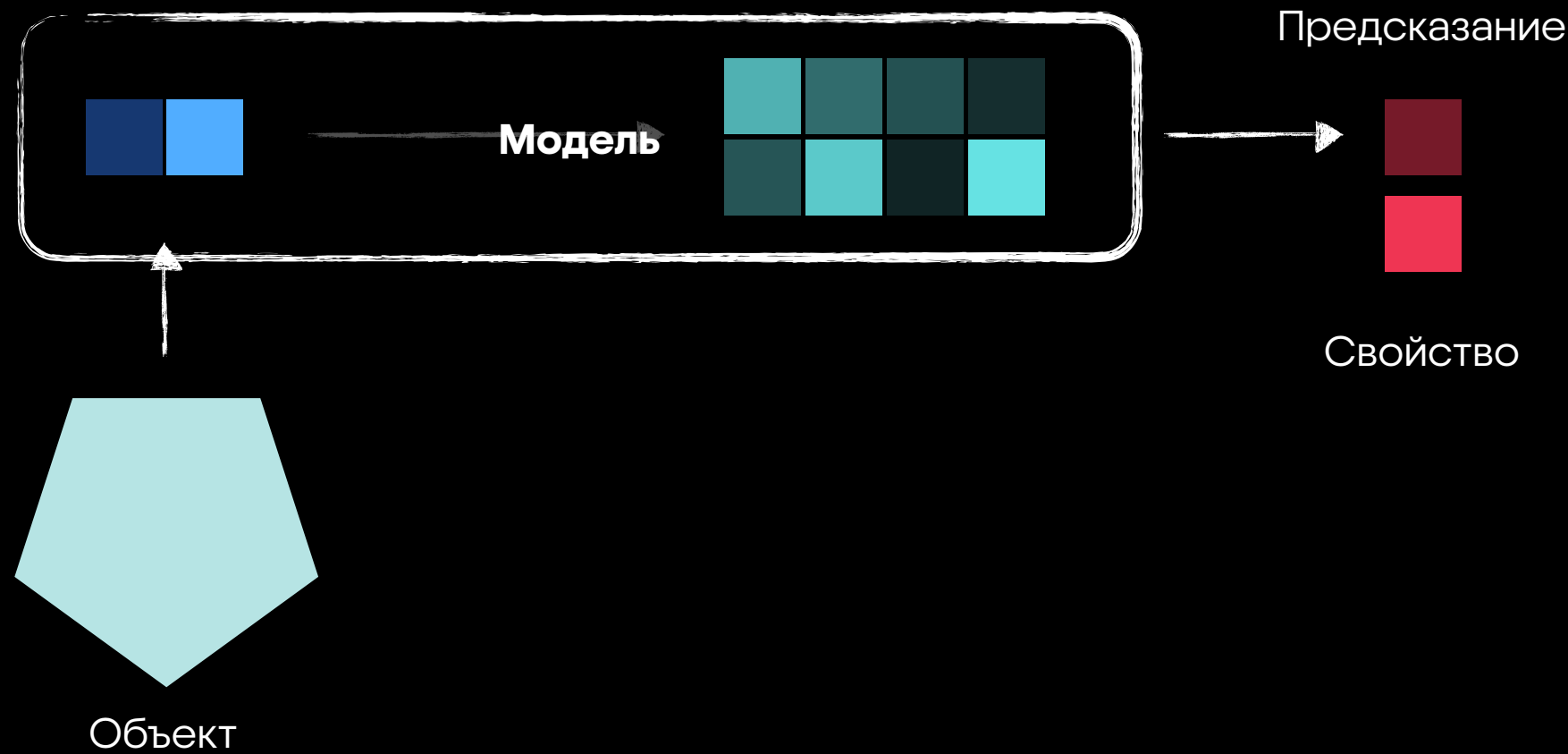
Обучение представлений данных



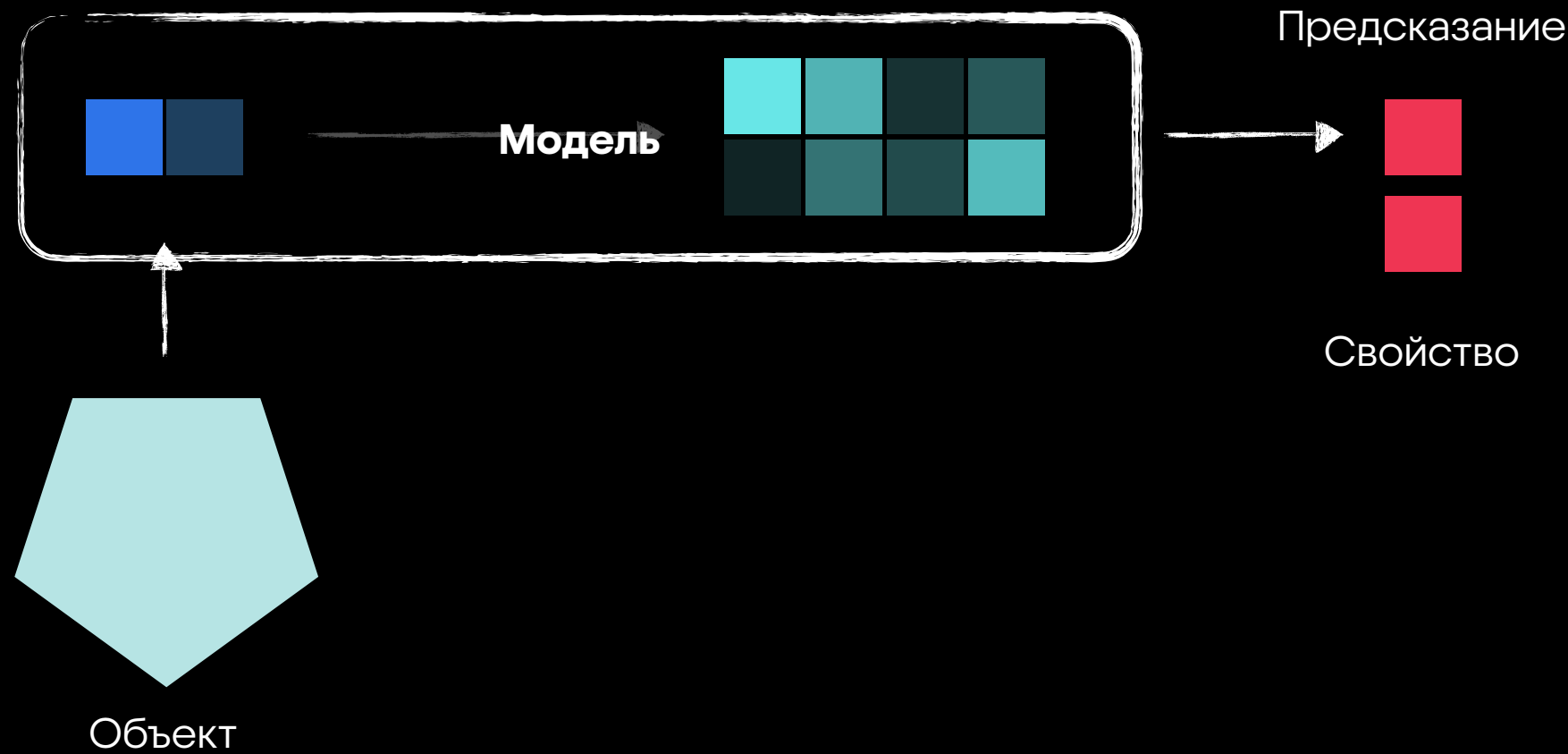
Обучение представлений данных



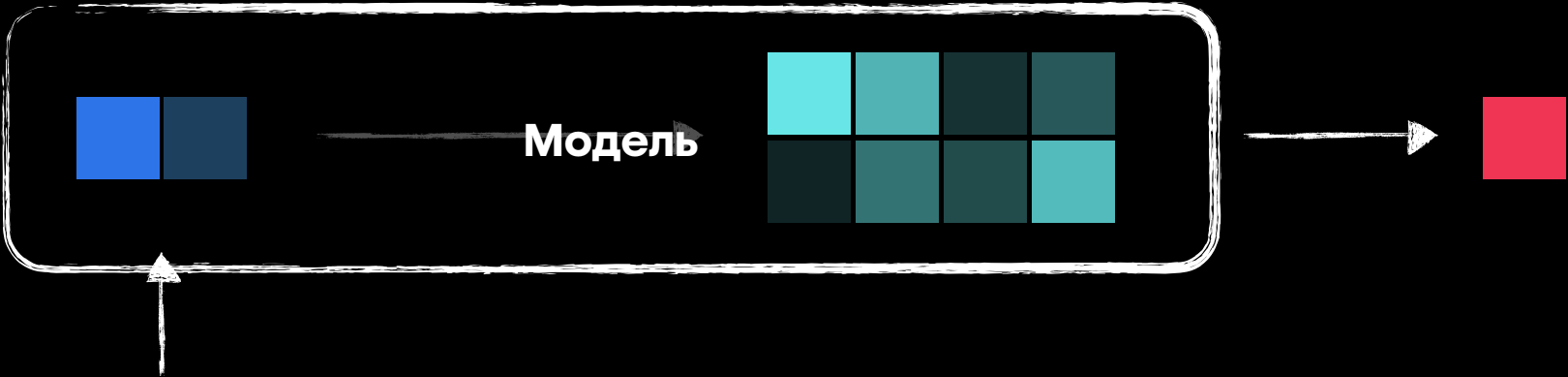
Обучение представлений данных



Обучение представлений данных

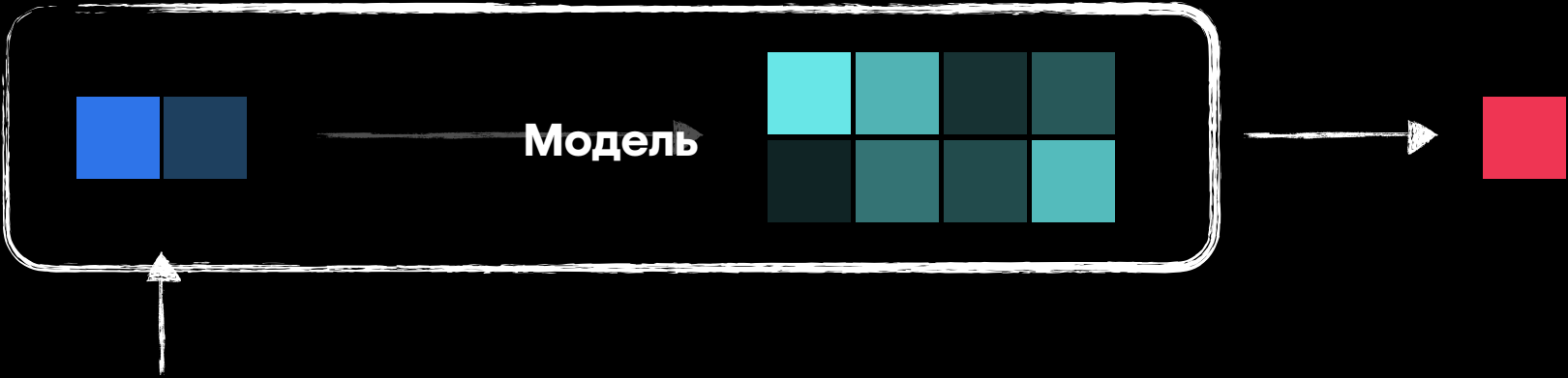


Обучение представлений данных



Изображения

Обучение представлений данных



QVQLVE...
DIQLTQ...
TVPPMV...

Последовательности



Изображения

Обучение представлений данных



Как обычно проходят глубокое обучение

Основы нейронных сетей



Свёрточные сети
в компьютерном зрении



Рекуррентные сети и трансформеры
для последовательностей



Графовые сети



Генеративные модели
(GAN, VAE, DDPM)



Как будем проходить мы

Основы нейронных сетей



Свёрточные сети
в компьютерном зрении



Рекуррентные сети и трансформеры
для последовательностей



Графовые сети



Генеративные модели
(GAN, VAE, DDPM)



Сколько времени у нас

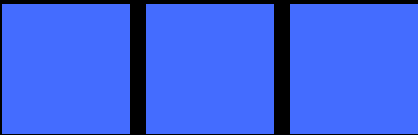


Как будем проходить мы

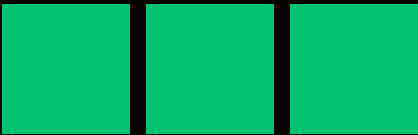
Основы нейронных сетей



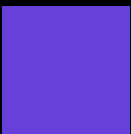
Свёрточные сети
в компьютерном зрении



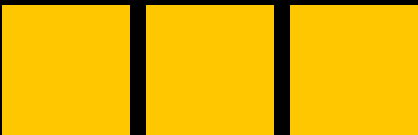
Рекуррентные сети и трансформеры
для последовательностей



Графовые сети



Генеративные модели
(GAN, VAE, DDPM)



Сколько времени у нас



Как будем проходить мы

Основы нейронных сетей



Свёрточные сети
в компьютерном зрении



Рекуррентные сети и трансформеры
для последовательностей



Графовые сети



Генеративные модели
(GAN, VAE, DDPM)



Сколько времени у нас



Как будем проходить мы

Основы нейронных сетей



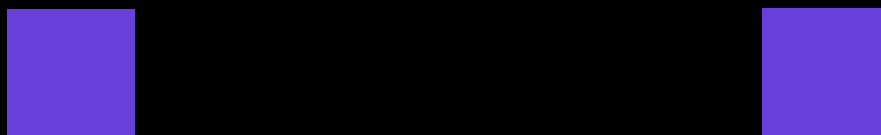
Свёрточные сети
в компьютерном зрении



Рекуррентные сети и трансформеры
для последовательностей



Графовые сети



Генеративные модели
(GAN, VAE, DDPM)



Сколько времени у нас

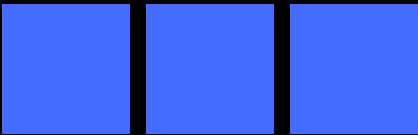


Как будем проходить мы

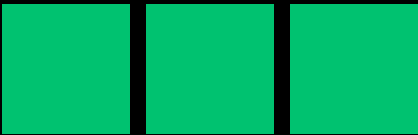
Основы нейронных сетей



Свёрточные сети
в компьютерном зрении



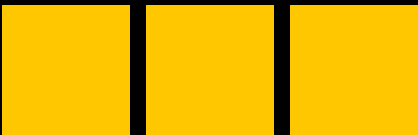
Рекуррентные сети и трансформеры
для последовательностей



Графовые сети



Генеративные модели
(GAN, VAE, DDPM)

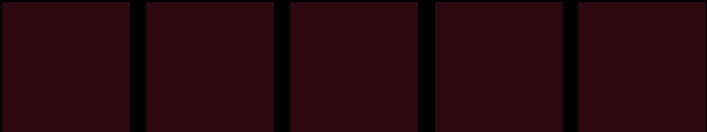


Сколько времени у нас



Как будем проходить мы

Основы нейронных сетей



Свёрточные сети
в компьютерном зрении



Рекуррентные сети и трансформеры
для последовательностей



Графовые сети



Генеративные модели
(GAN, VAE, DDPM)



Сколько времени у нас



Как будем проходить мы

Основы нейронных сетей

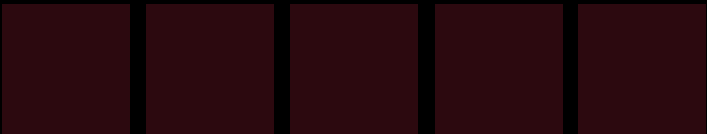
Свёрточные сети
в компьютерном зрении

Рекуррентные сети и трансформеры
для последовательностей

Графовые сети

Генеративные модели
(GAN, VAE, DDPM)

Сколько времени у нас



Адаптация моделей

Управление
экспериментами

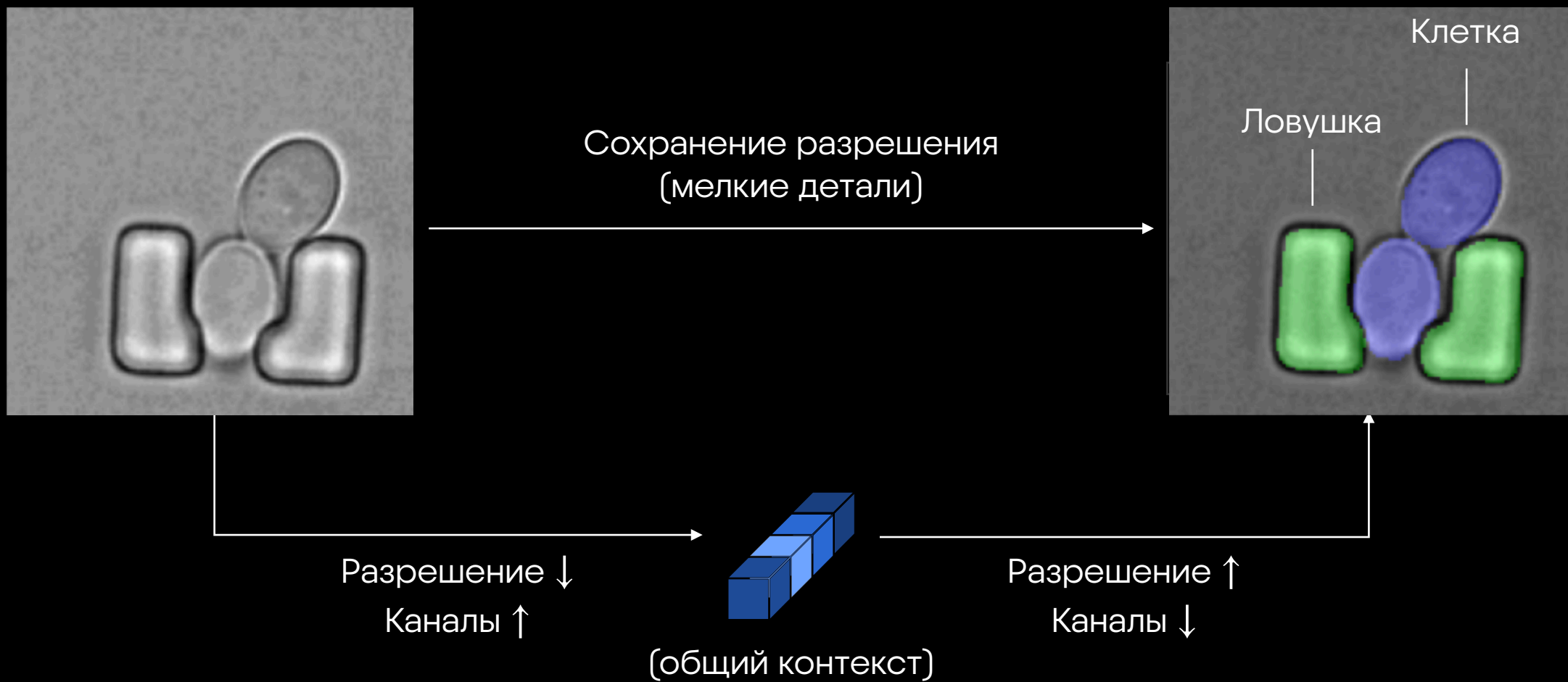
Вспомогательные
инструменты



Обучение представлений данных



Проект №1. Сегментация изображений с клетками дрожжей



Проект №2. Структурная классификация белков

MHHHHHHKLITLLLRSSKSEDLRLSIPVDFTVKDLIKRYCTE
VKISFHERIRLEFEGEWLDPNDQVQSTELEDEDQVSVVLD

Orthogonal Bundle



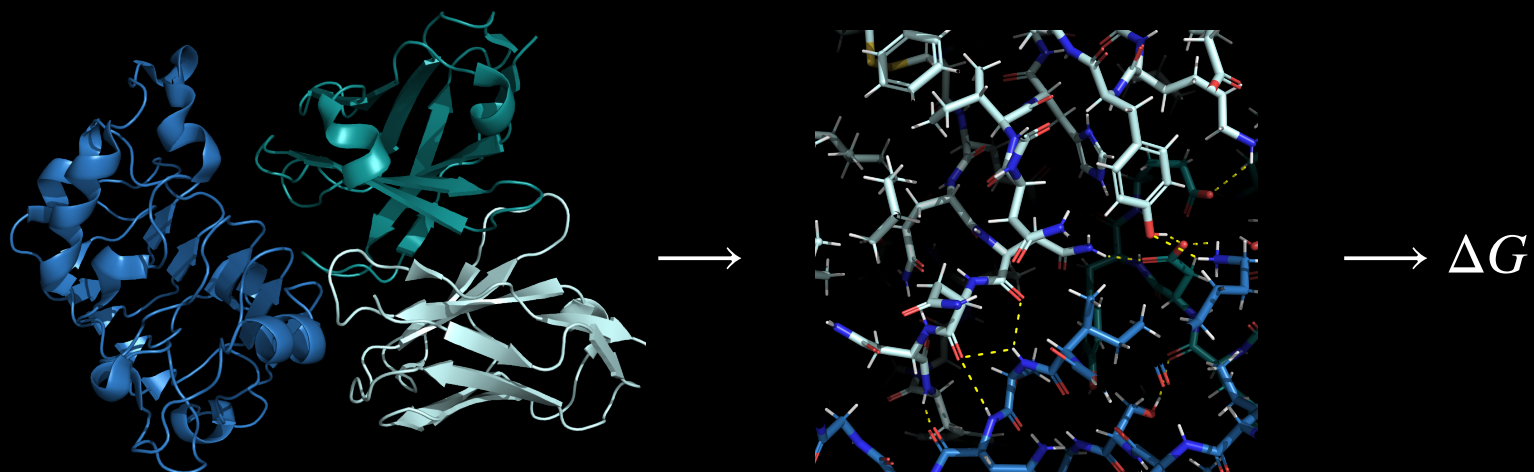
Beta sandwich



Alpha-beta roll



Проект №3. Предсказание свободной энергии связывания



Организационная информация

Материалы курса: <https://github.com/norsage/dl-bioinf>

Семестровый курс (МКН СПбГУ): <https://github.com/norsage/dl-mcs>

Домашние задания (Github Classroom):

- 1-2 разминочных задания
- индивидуальный мини-проект на 2 недели

Варианты работы:

- Colab, Kaggle
- локальная установка (если есть видеокарта NVIDIA или MacBook серии M)
- инструкции по установке окружения — в репозитории курса

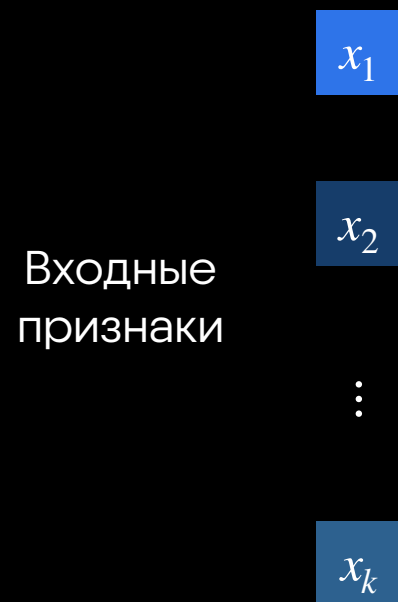
План на сегодня



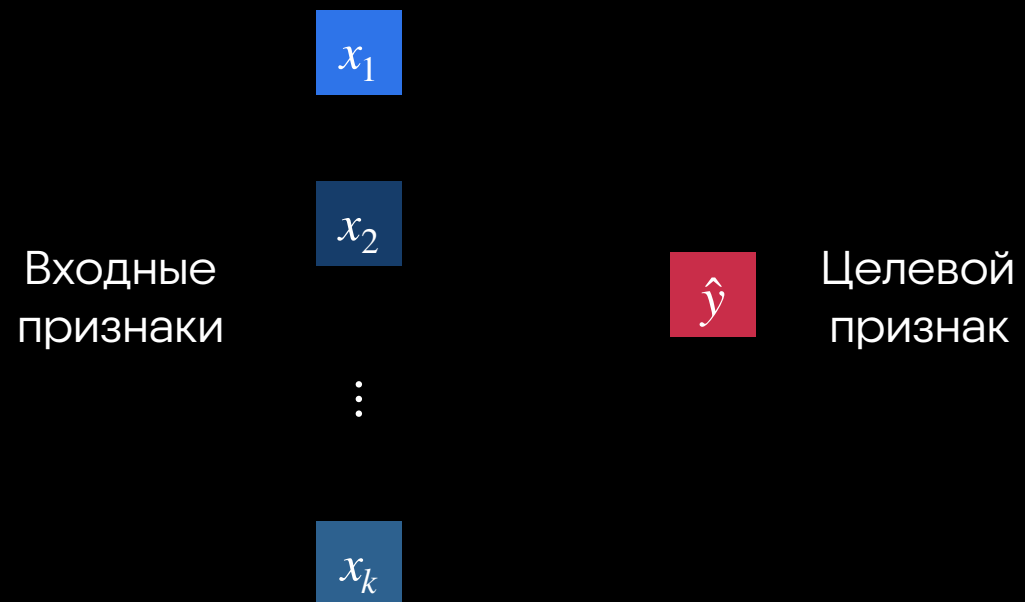
1. Нейронные сети: теоретический минимум
2. Основы Pytorch: тензоры, градиенты и первая нейронная сеть

1. Основы нейронных сетей

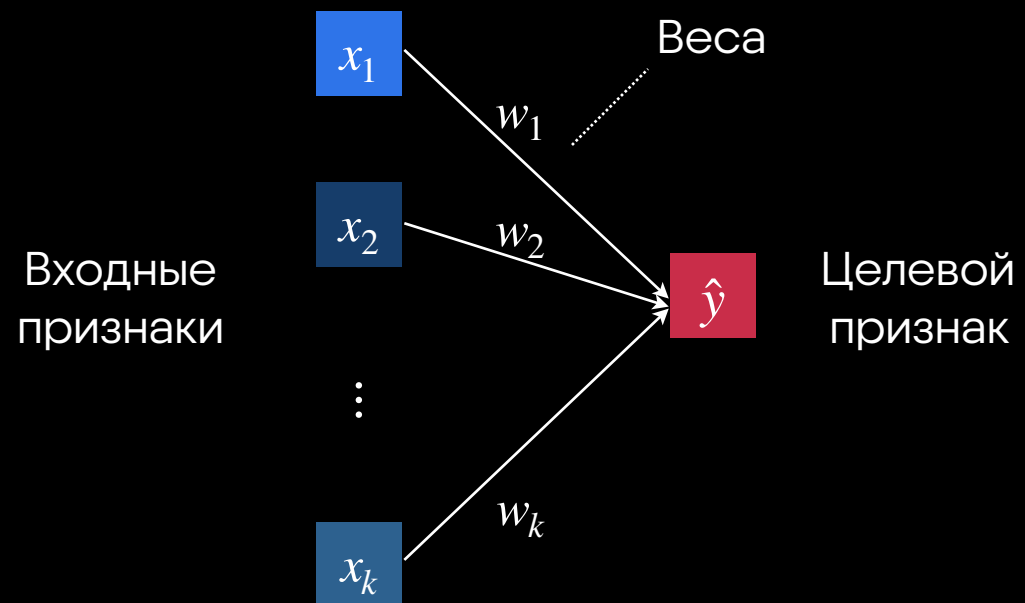
От линейной регрессии к модели нейрона



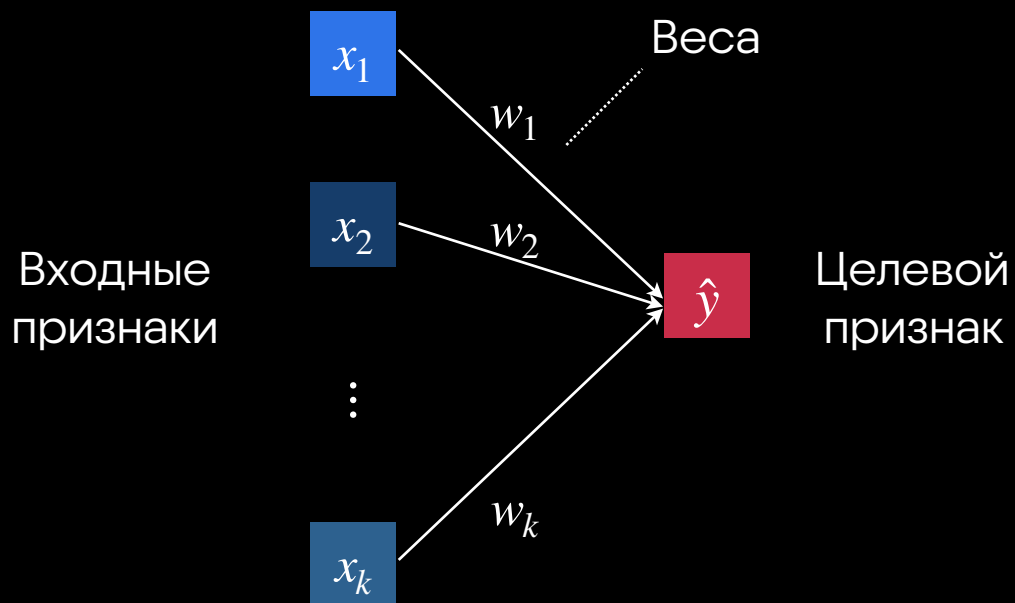
От линейной регрессии к модели нейрона



От линейной регрессии к модели нейрона



От линейной регрессии к модели нейрона

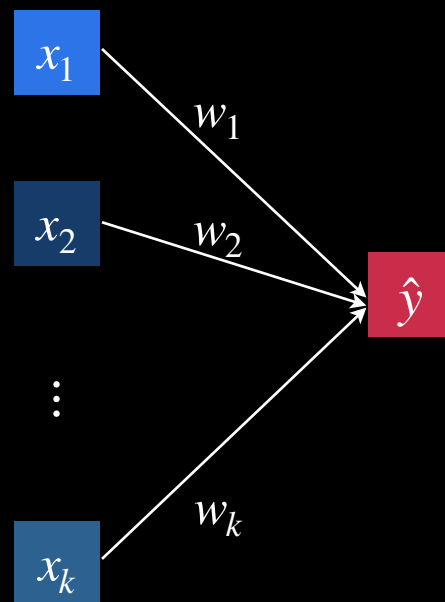


$$\hat{y} = \sum_{i=1}^k w_i x_i$$

В матричной форме:

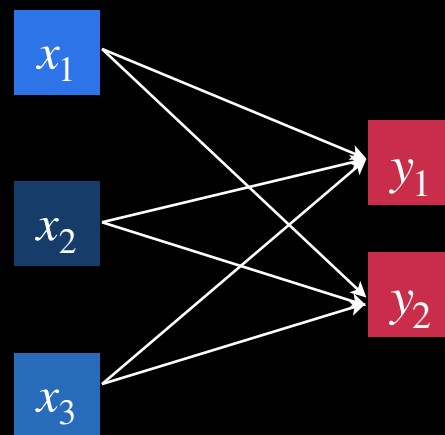
$$\hat{y} = \mathbf{w}^T \mathbf{x}$$

Матричное умножение

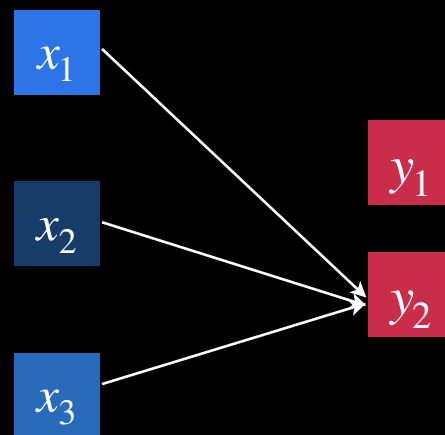


$$\hat{y} = \sum_{i=1}^k w_i x_i$$

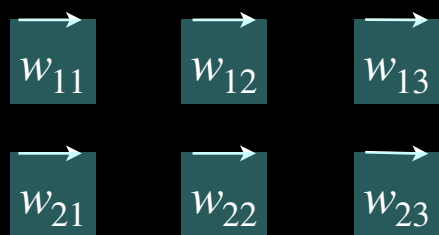
Матричное умножение



Матричное умножение



Матричное умножение



x_1

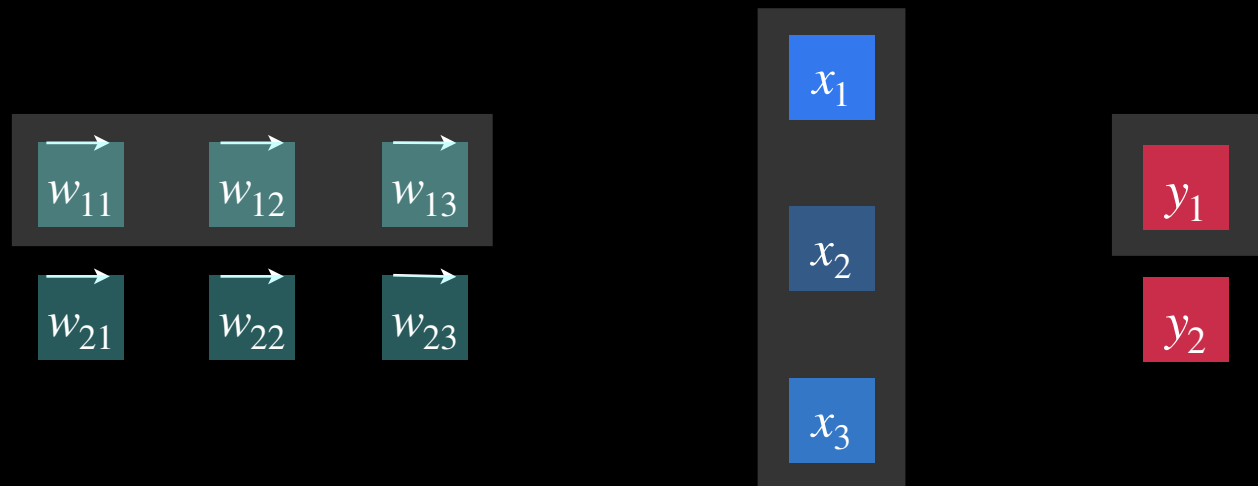
x_2

x_3

y_1

y_2

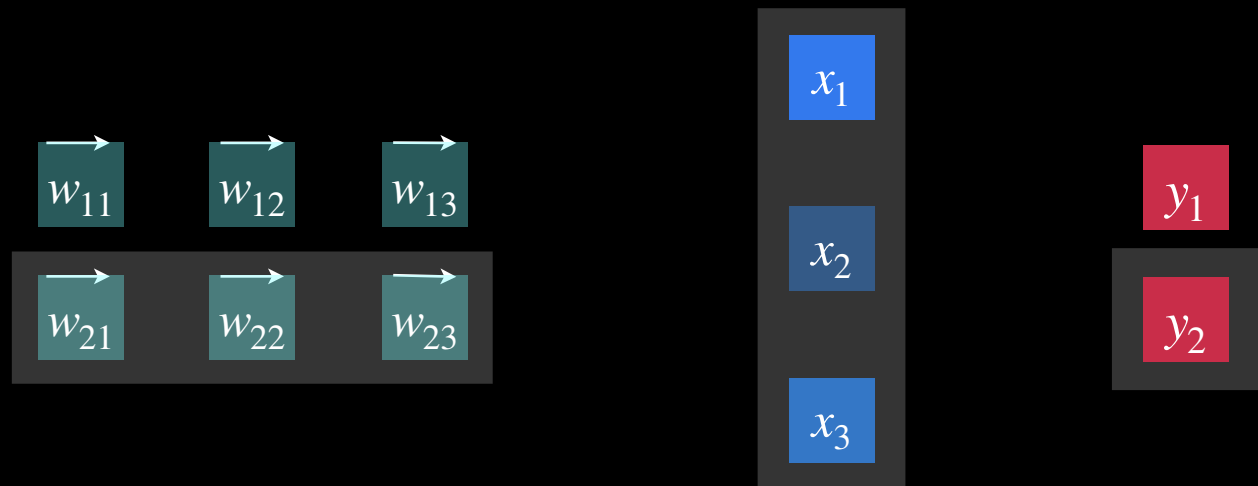
Матричное умножение



$$\mathbf{W} \cdot \mathbf{x} = \mathbf{y}$$

$$y_i = \sum_{j=1}^k w_{ij} x_j = \mathbf{W}_i \cdot \mathbf{x}$$

Матричное умножение



$\mathbf{W}$

$\cdot$

$\mathbf{X}$

$=$

$\mathbf{Y}$

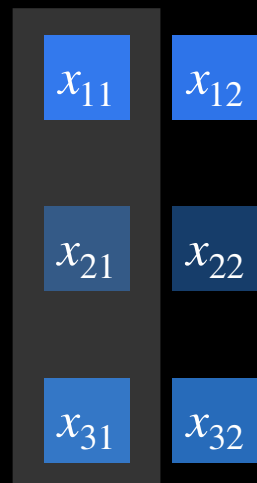
$$y_i = \sum_{j=1}^k w_{ij} x_j = \mathbf{W}_i \cdot \mathbf{x}$$

Матричное умножение



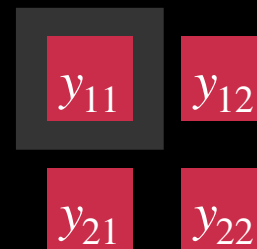
W

•



X

=



Y

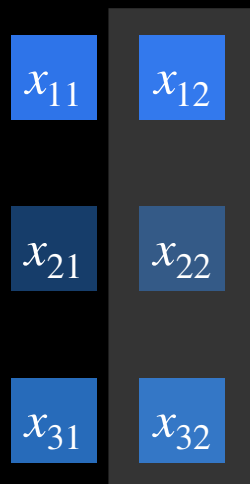
$$y_{ij} = \sum_k w_{ik} x_{kj}$$

Матричное умножение



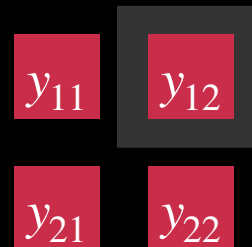
W

•



X

=



Y

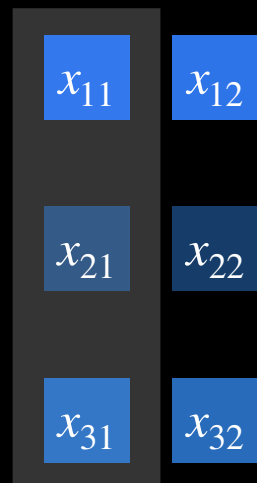
$$y_{ij} = \sum_k w_{ik} x_{kj}$$

Матричное умножение



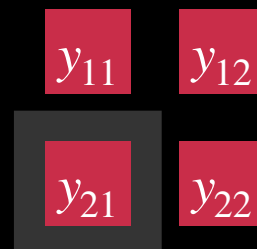
W

$\cdot$



X

$=$



Y

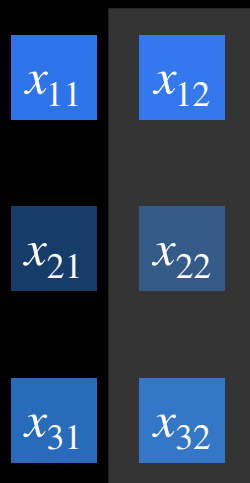
$$y_{ij} = \sum_k w_{ik} x_{kj}$$

Матричное умножение



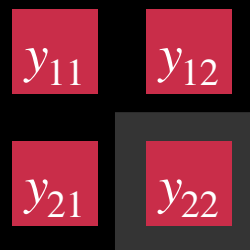
W

$\cdot$



X

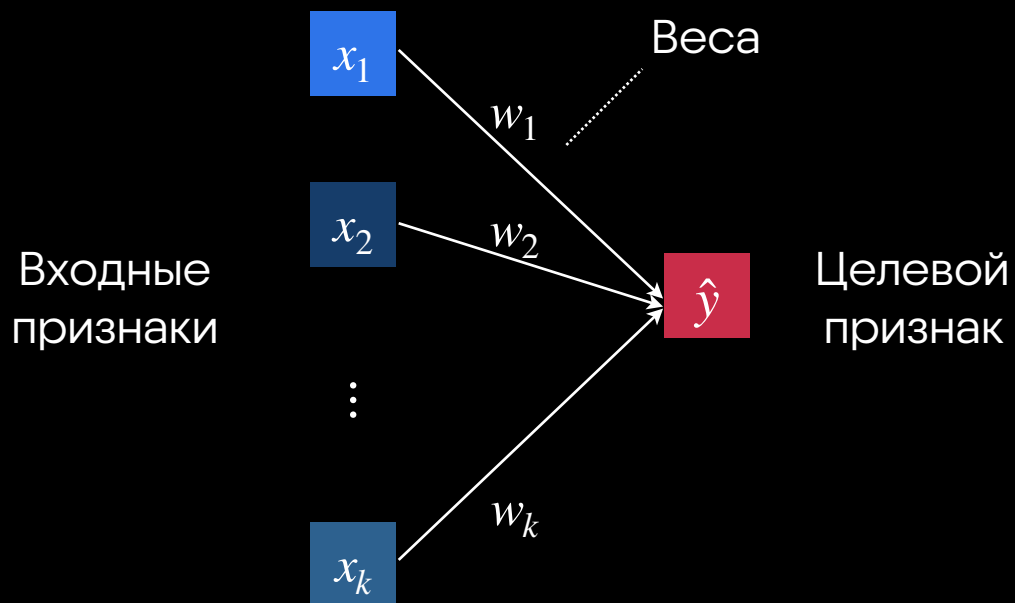
$=$



Y

$$y_{ij} = \sum_k w_{ik} x_{kj}$$

От линейной регрессии к модели нейрона

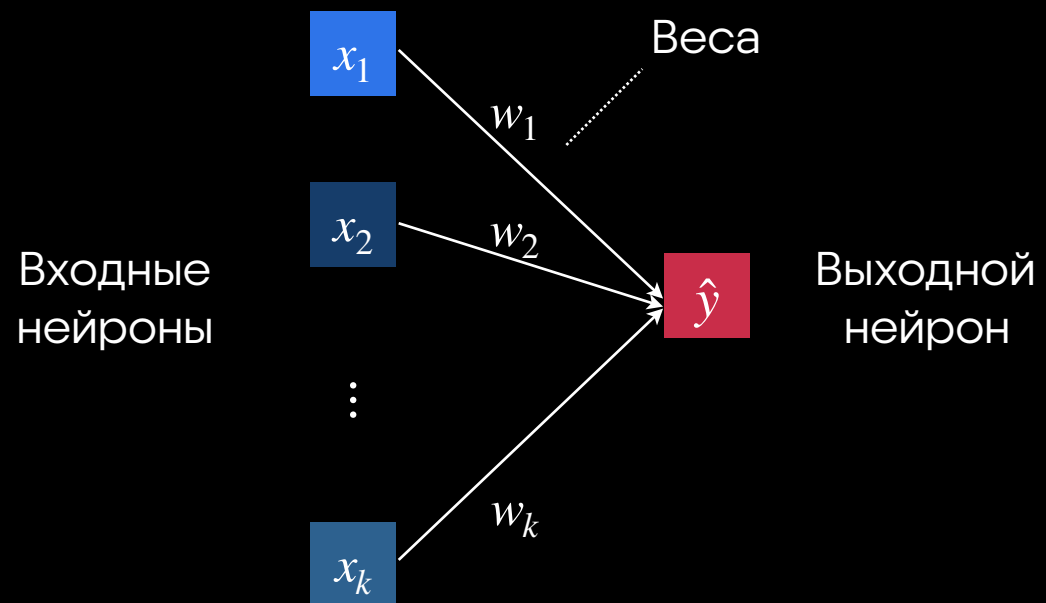


$$\hat{y} = \sum_{i=1}^k w_i x_i$$

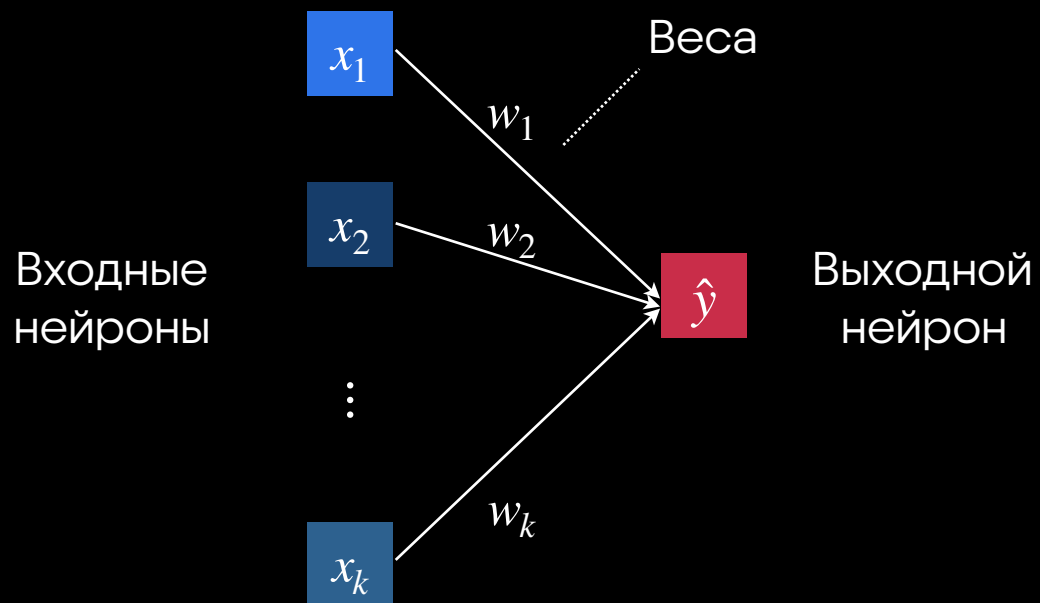
В матричной форме:

$$\hat{y} = \mathbf{w}^T \mathbf{x}$$

От линейной регрессии к модели нейрона

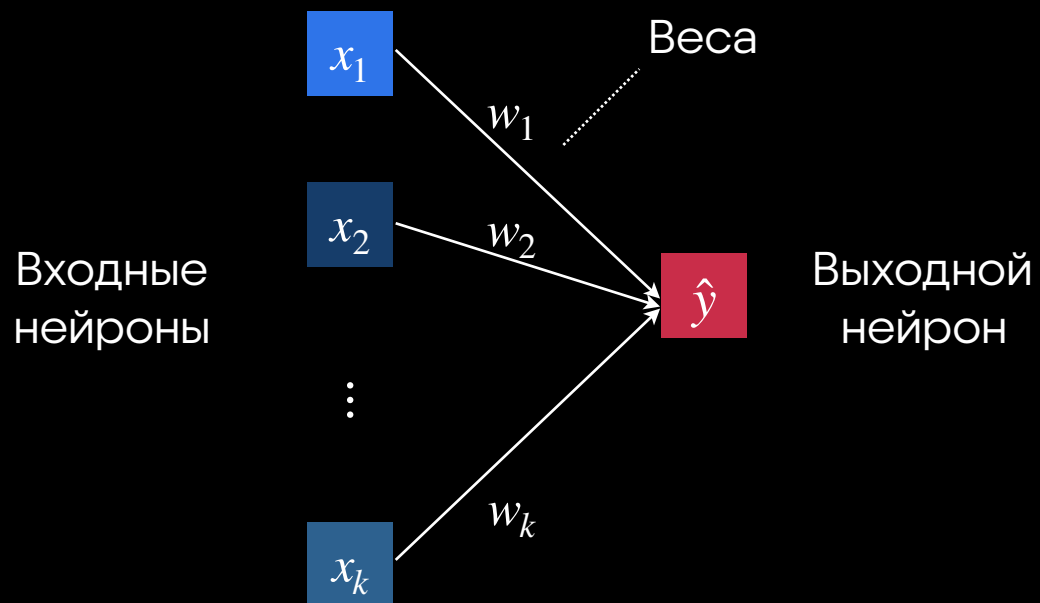


От линейной регрессии к модели нейрона



$$a = \sum_{i=1}^k w_i x_i \quad \text{.....} \quad \text{Пре-активация}$$

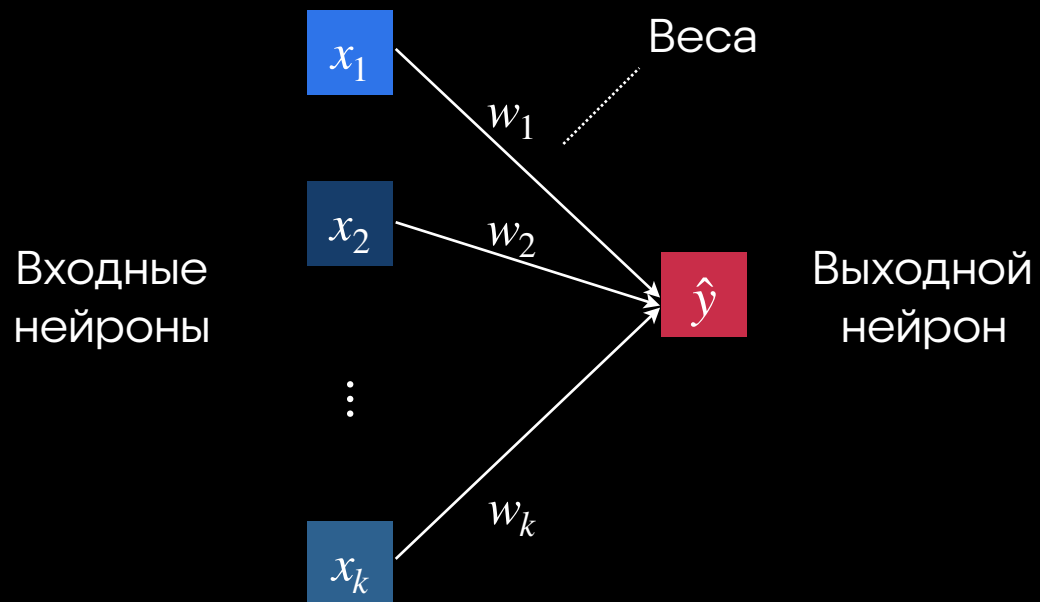
От линейной регрессии к модели нейрона



$$a = \sum_{i=1}^k w_i x_i \quad \text{.....} \quad \text{Пре-активация}$$
$$\hat{y} = f(a) \quad \text{.....} \quad \text{Активация}$$

└──────────┘
Функция активации

От линейной регрессии к модели нейрона

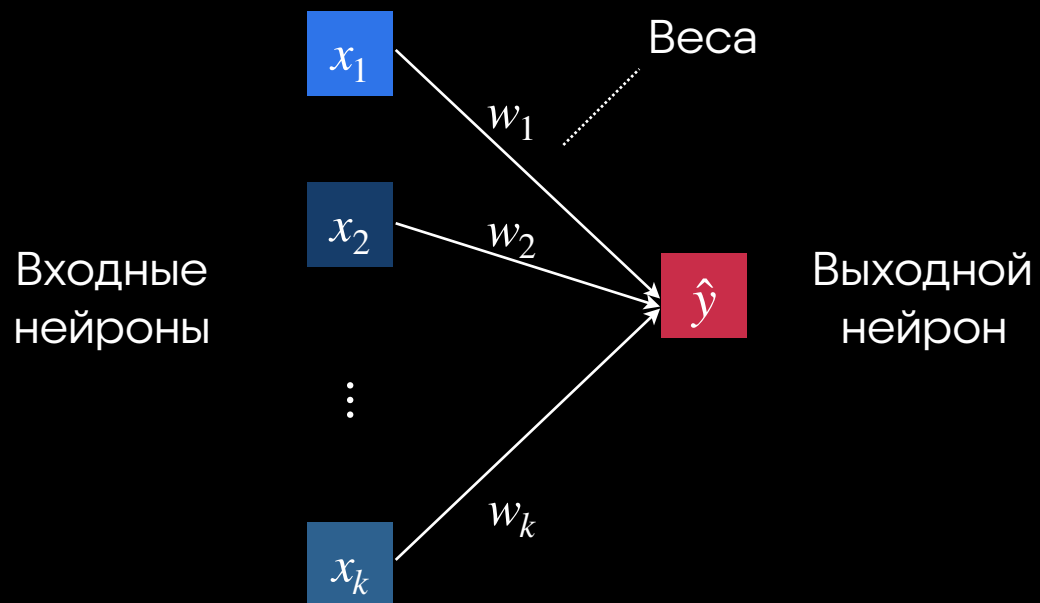


$$a = \sum_{i=1}^k w_i x_i \quad \text{.....} \quad \text{Пре-активация}$$
$$\hat{y} = f(a) \quad \text{.....} \quad \text{Активация}$$

└─────────────────── Функция активации

Перцептрон:
$$f(a) = \begin{cases} 1, & a > 0 \\ 0, & a = 0 \\ -1, & a < 0 \end{cases}$$

От линейной регрессии к модели нейрона



$$a = \sum_{i=1}^k w_i x_i \quad \text{.....} \quad \text{Пре-активация}$$
$$\hat{y} = f(a) \quad \text{.....} \quad \text{Активация}$$

└─────────── Функция активации

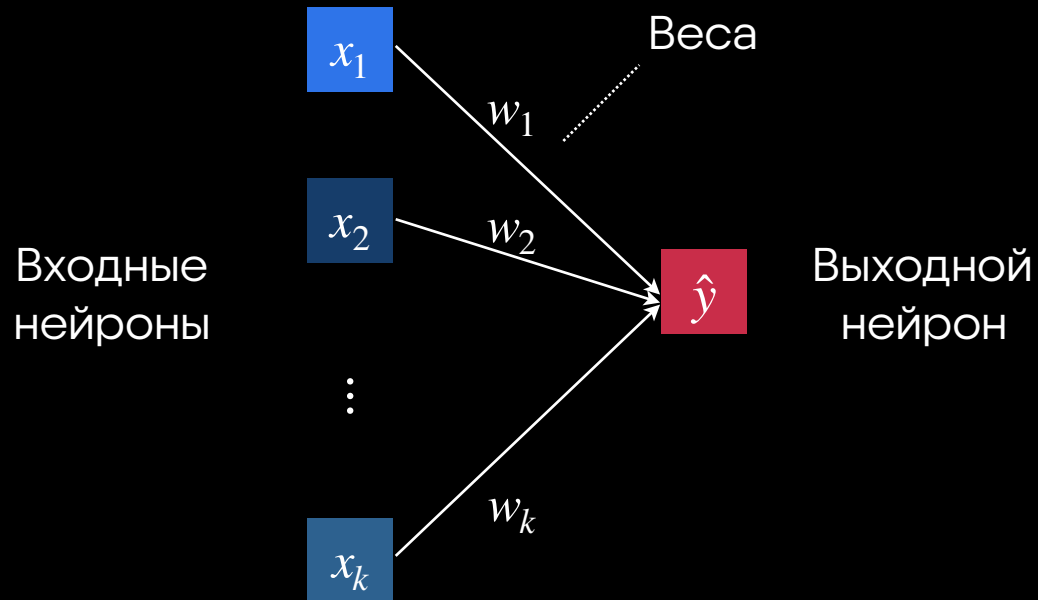
Перцептрон:

$$f(a) = \begin{cases} 1, & a > 0 \\ 0, & a = 0 \\ -1, & a < 0 \end{cases}$$

Линейная регрессия:

$$f(a) = a$$

От линейной регрессии к модели нейрона



$$a = \sum_{i=1}^k w_i x_i \quad \text{.....} \quad \text{Пре-активация}$$

$$\hat{y} = f(a) \quad \text{.....} \quad \text{Активация}$$

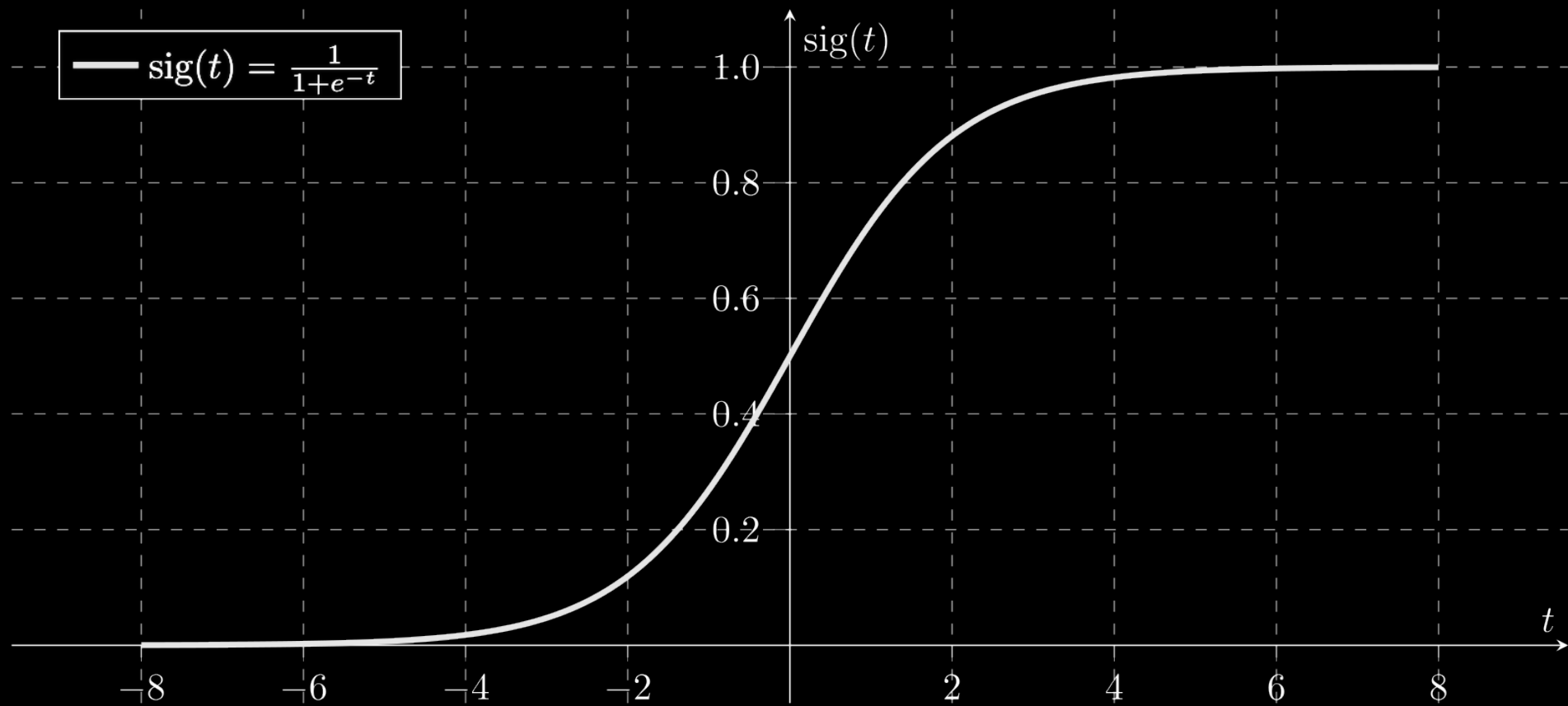
└───────────
Функция активации

Перцептрон:
$$f(a) = \begin{cases} 1, & a > 0 \\ 0, & a = 0 \\ -1, & a < 0 \end{cases}$$

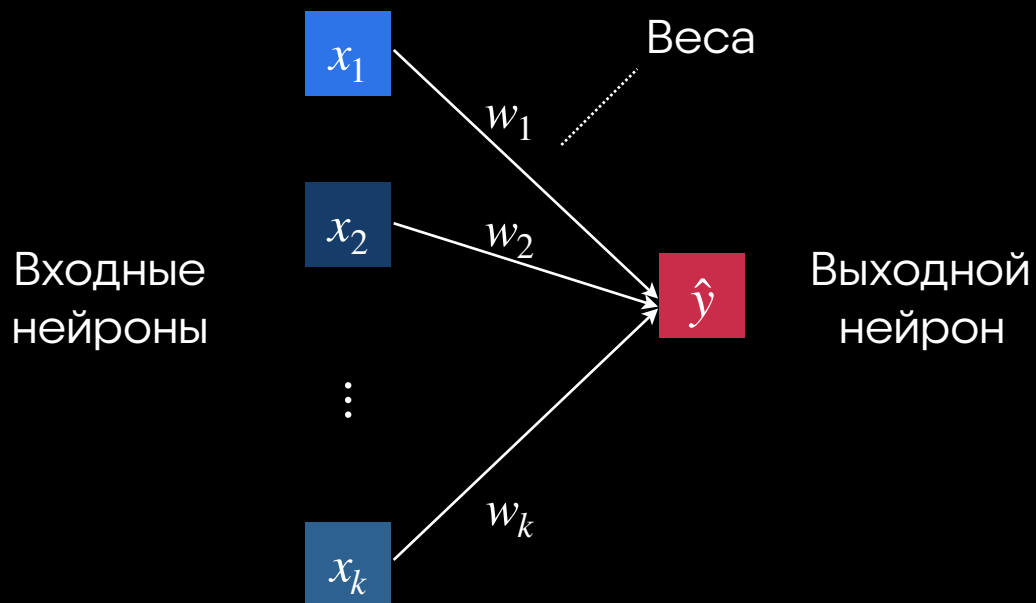
Линейная регрессия:
$$f(a) = a$$

Логистическая регрессия:
$$f(a) = \sigma(a) = \frac{1}{1 + \exp(-a)}$$

От линейной регрессии к модели нейрона



От линейной регрессии к модели нейрона



$$a = \sum_{i=1}^k w_i x_i \quad \text{.....} \quad \text{Пре-активация}$$

Пре-активация

$\hat{y} = f(a)$ Активация

Активация

Функция активации

Перцептрон: $f(a) = \begin{cases} 1, & a > 0 \\ 0, & a = 0 \\ -1, & a < 0 \end{cases}$

Перцептрон:

$$f(a) = \begin{cases} 1, & a > 0 \\ 0, & a = 0 \\ -1, & a < 0 \end{cases}$$

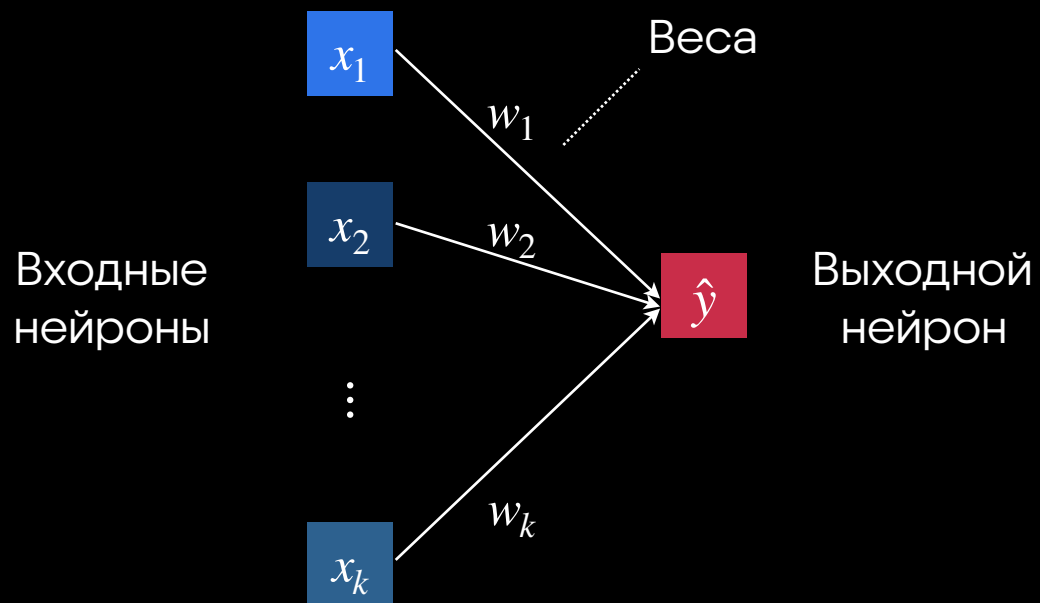
Линейная регрессия:

$$f(a) = a$$

Логистическая регрессия:

$$f(a) = \sigma(a) = \frac{1}{1 + \exp(-a)}$$

От линейной регрессии к модели нейрона



$$a = \sum_{i=1}^k w_i x_i$$

..... Пре-активация

$$\hat{y} = f(a)$$

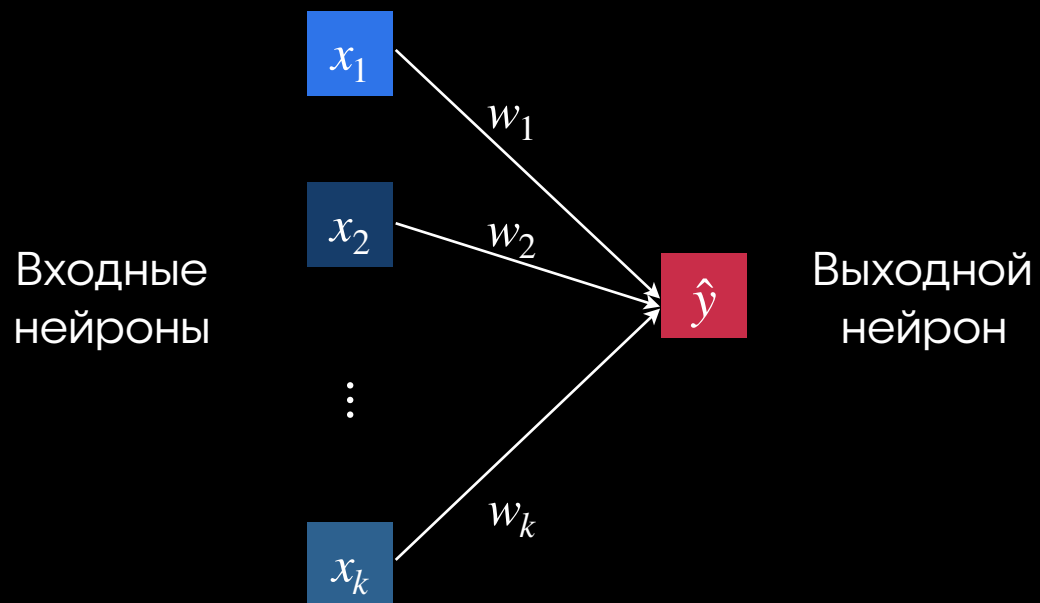
..... Активация

..... Функция активации

В матричной форме:

$$\hat{y} = f(\mathbf{w}^T \mathbf{x})$$

Если классов больше двух?



$$a = \sum_{i=1}^k w_i x_i$$

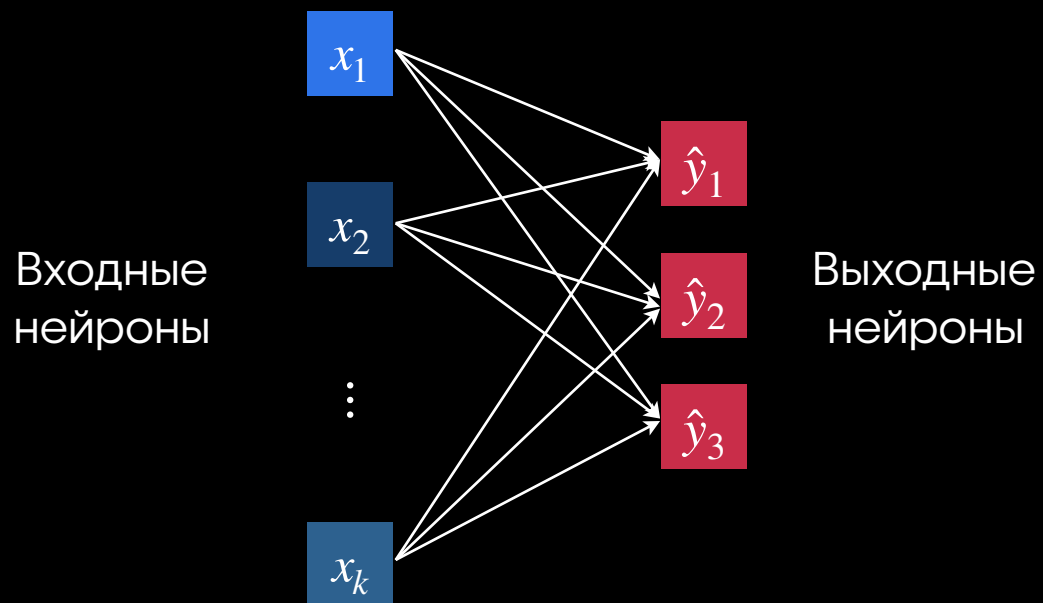
..... Пре-активация

$$\hat{y} = f(a)$$

..... Активация

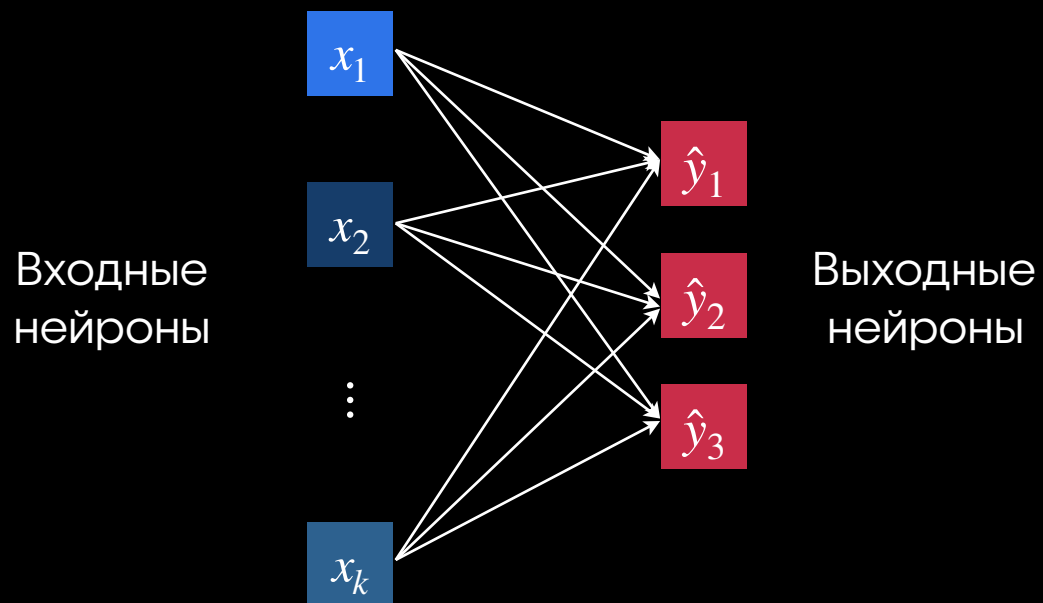
└..... Функция активации

Если классов больше двух?



$$a_i = \sum_{j=1}^k w_{ij} x_j \quad \dots\dots\dots \text{Пре-активация (logits)}$$

Если классов больше двух?

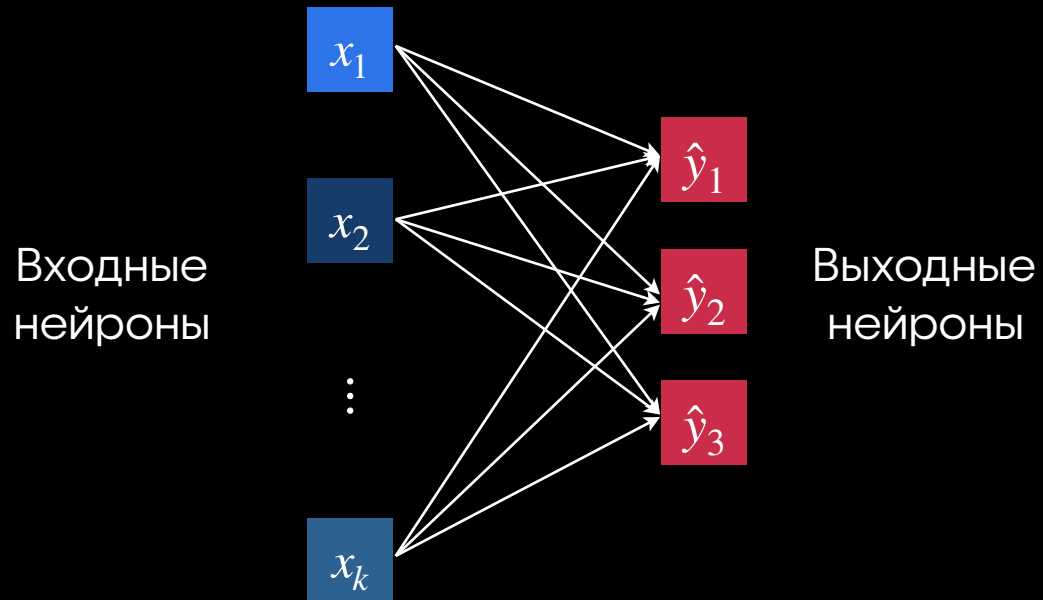


$$a_i = \sum_{j=1}^k w_{ij} x_j \quad \dots \quad \text{Пре-активация (logits)}$$

$$\hat{y}_i = \text{softmax}(\mathbf{a})_i \quad \dots \quad \text{Активация}$$

$$\text{softmax}(\mathbf{a})_i = \frac{\exp(a_i)}{\sum_k \exp(a_k)}$$

Если классов больше двух?



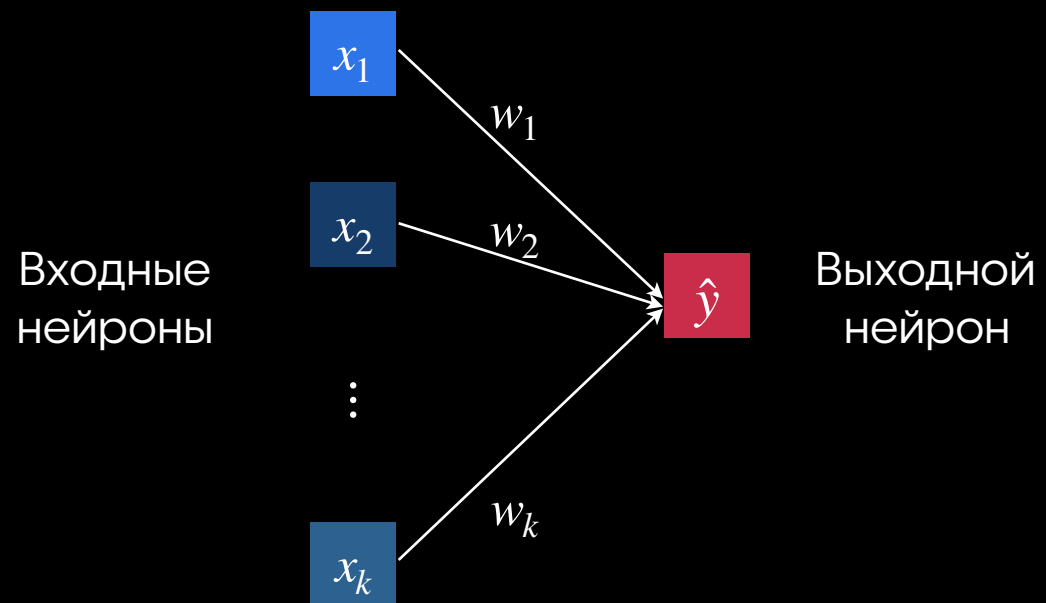
$$a_i = \sum_{j=1}^k w_{ij} x_j \quad \dots \quad \text{Пре-активация (logits)}$$

$$\hat{y}_i = \text{softmax}(\mathbf{a})_i \quad \dots \quad \text{Активация}$$

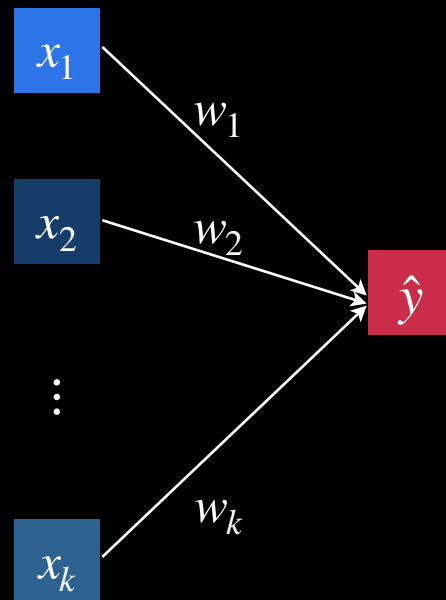
$$\text{softmax}(\mathbf{a})_i = \frac{\exp(a_i)}{\sum_k \exp(a_k)}$$

- все активации неотрицательные
- сумма активаций равна 1
- можно интерпретировать как вероятности классов

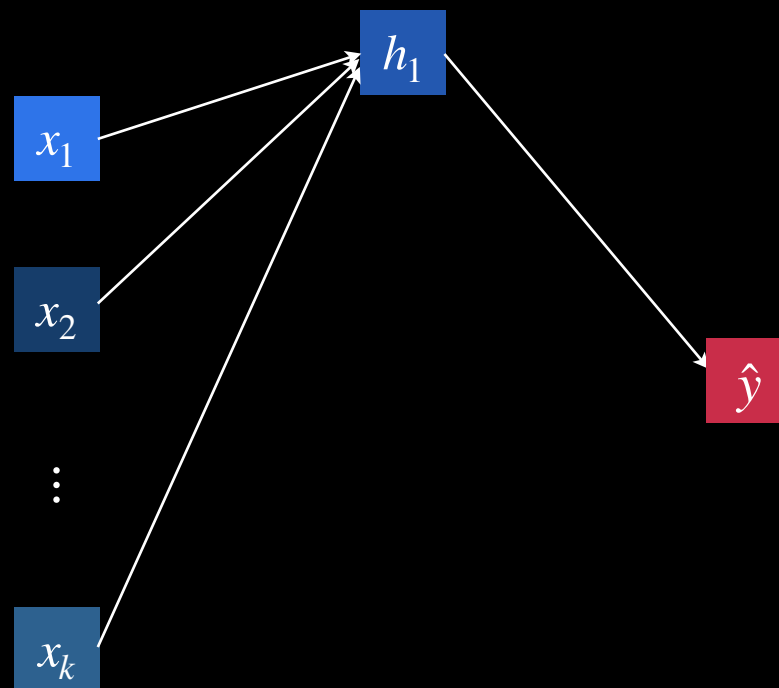
Однослойная нейронная сеть



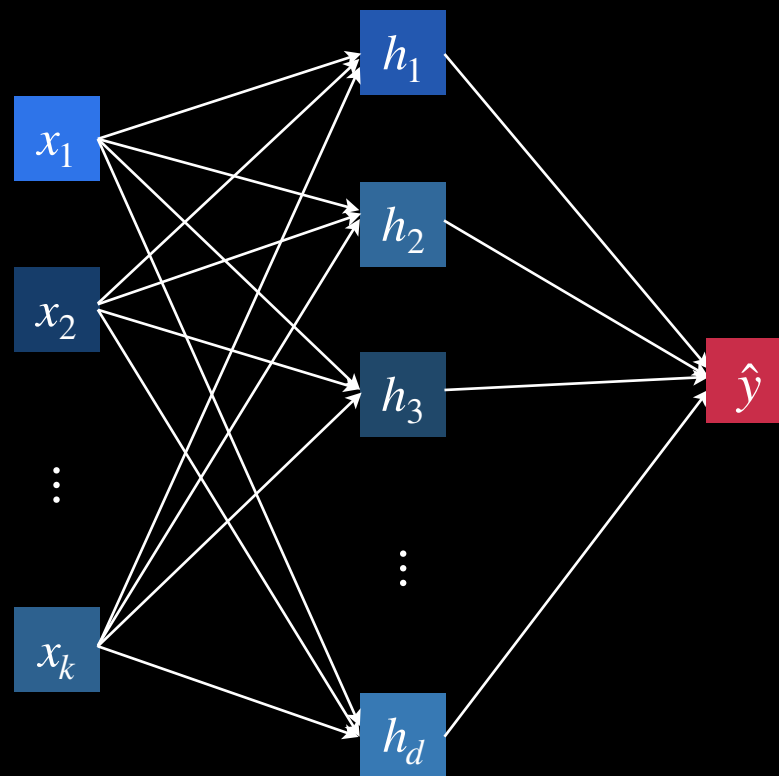
Однослойная нейронная сеть



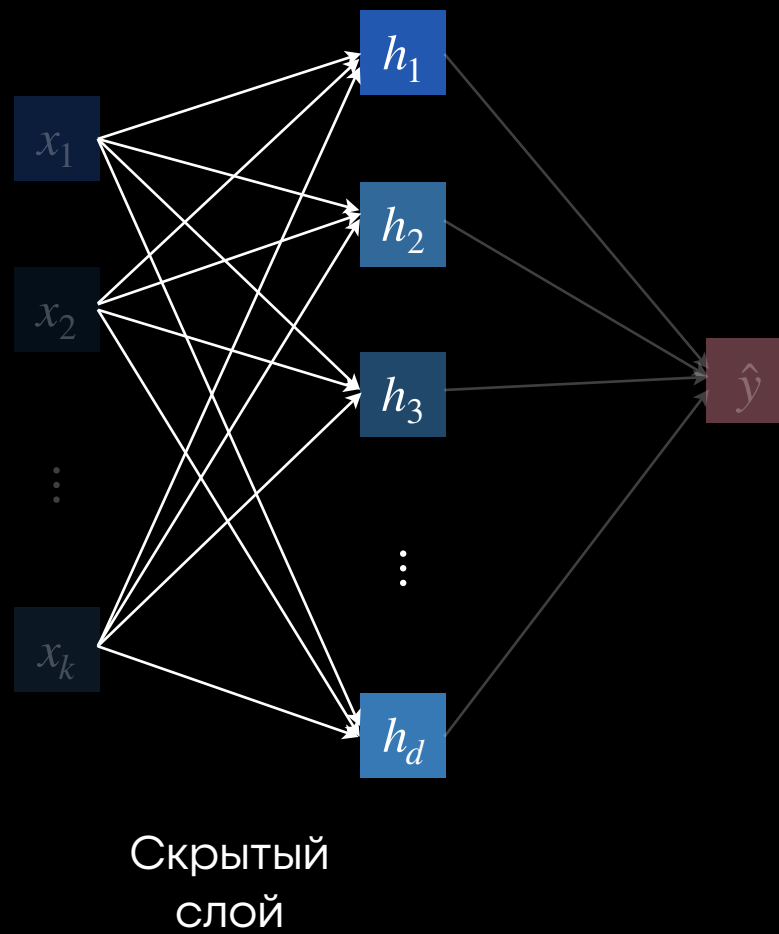
Добавление скрытого слоя



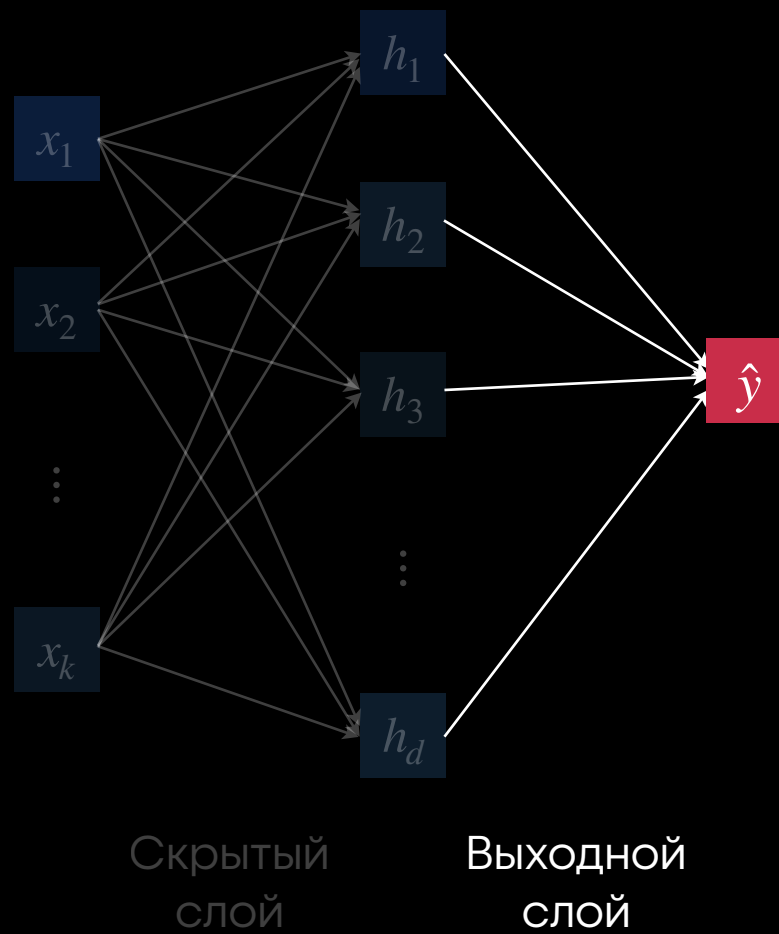
Добавление скрытого слоя



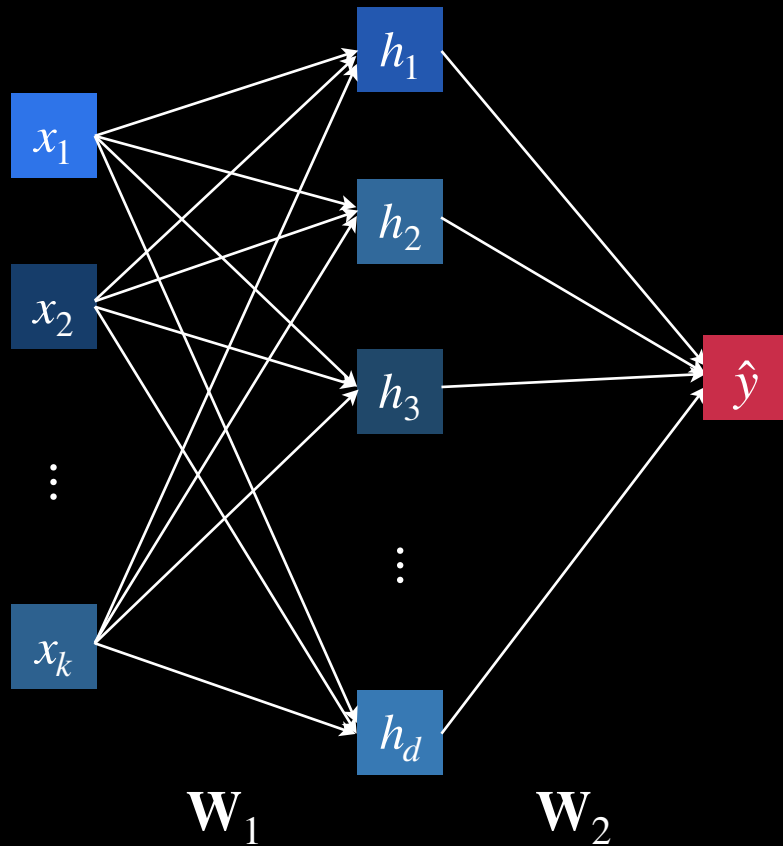
Добавление скрытого слоя



Добавление скрытого слоя



Добавление скрытого слоя



$$h_j = f_h \left(\sum_{i=1}^k w_{ji}^{(1)} x_i \right) \quad \text{Скрытые активации}$$

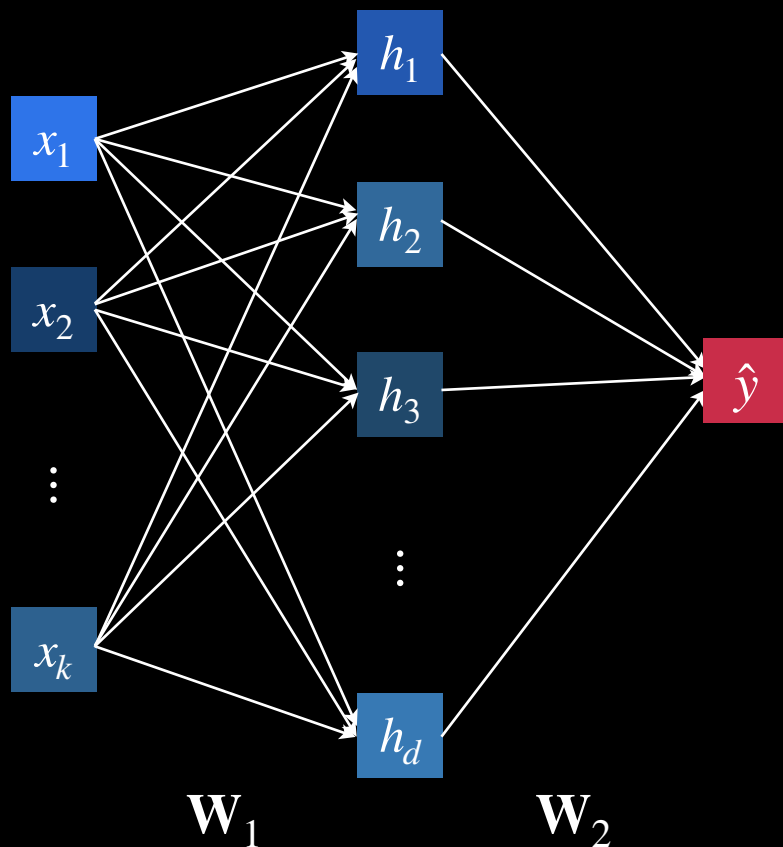
$$\hat{y}_j = f_o \left(\sum_{i=1}^d w_{ji}^{(2)} h_i \right) \quad \text{Выход сети}$$

В матричной форме:

$$\mathbf{h} = f_h (\mathbf{W}_1 \mathbf{x})$$

$$\hat{\mathbf{y}} = f_o (\mathbf{W}_2 \mathbf{h})$$

Добавление скрытого слоя



В матричной форме:

$$\mathbf{h} = f_h(\mathbf{W}_1 \mathbf{x})$$

$$\hat{\mathbf{y}} = f_o(\mathbf{W}_2 \mathbf{h})$$

Для некоторых нелинейных

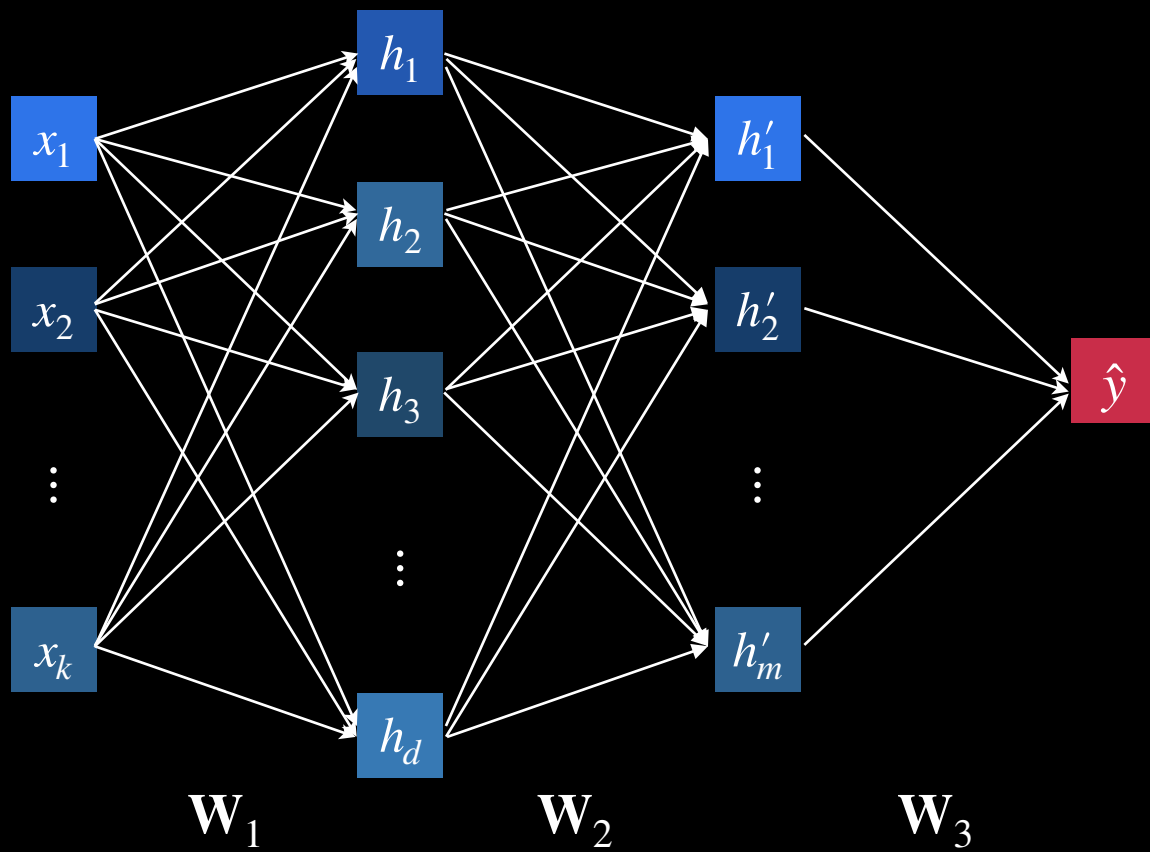
функций активации f_h

(например, для сигмоиды)

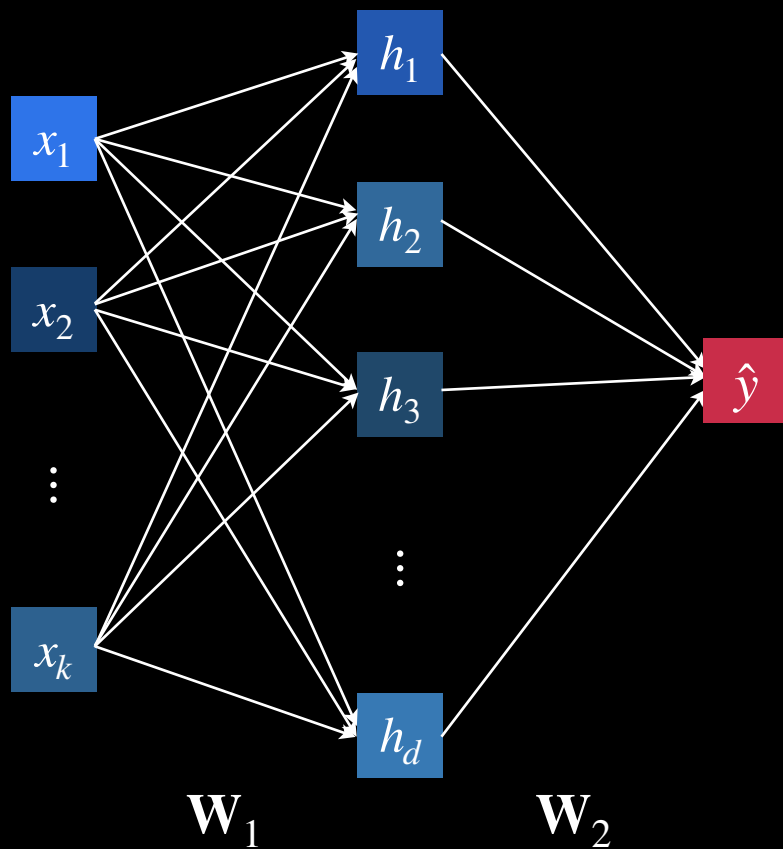
такая сеть —

универсальный аппроксиматор

Добавление скрытого слоя



Добавление скрытого слоя



В матричной форме:

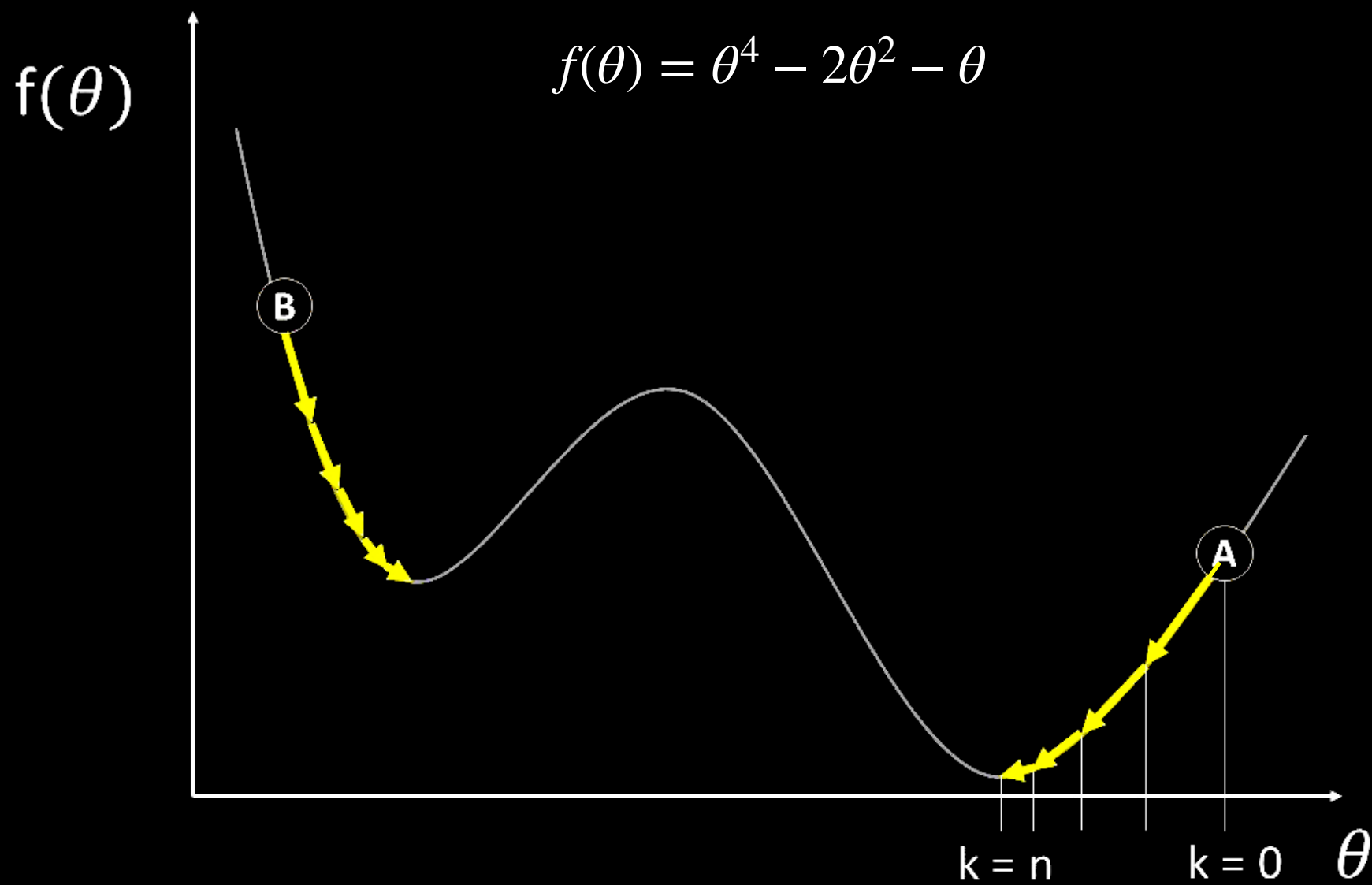
$$\mathbf{h} = f_h(\mathbf{W}_1 \mathbf{x})$$

$$\hat{y} = f_o(\mathbf{W}_2 \mathbf{h})$$

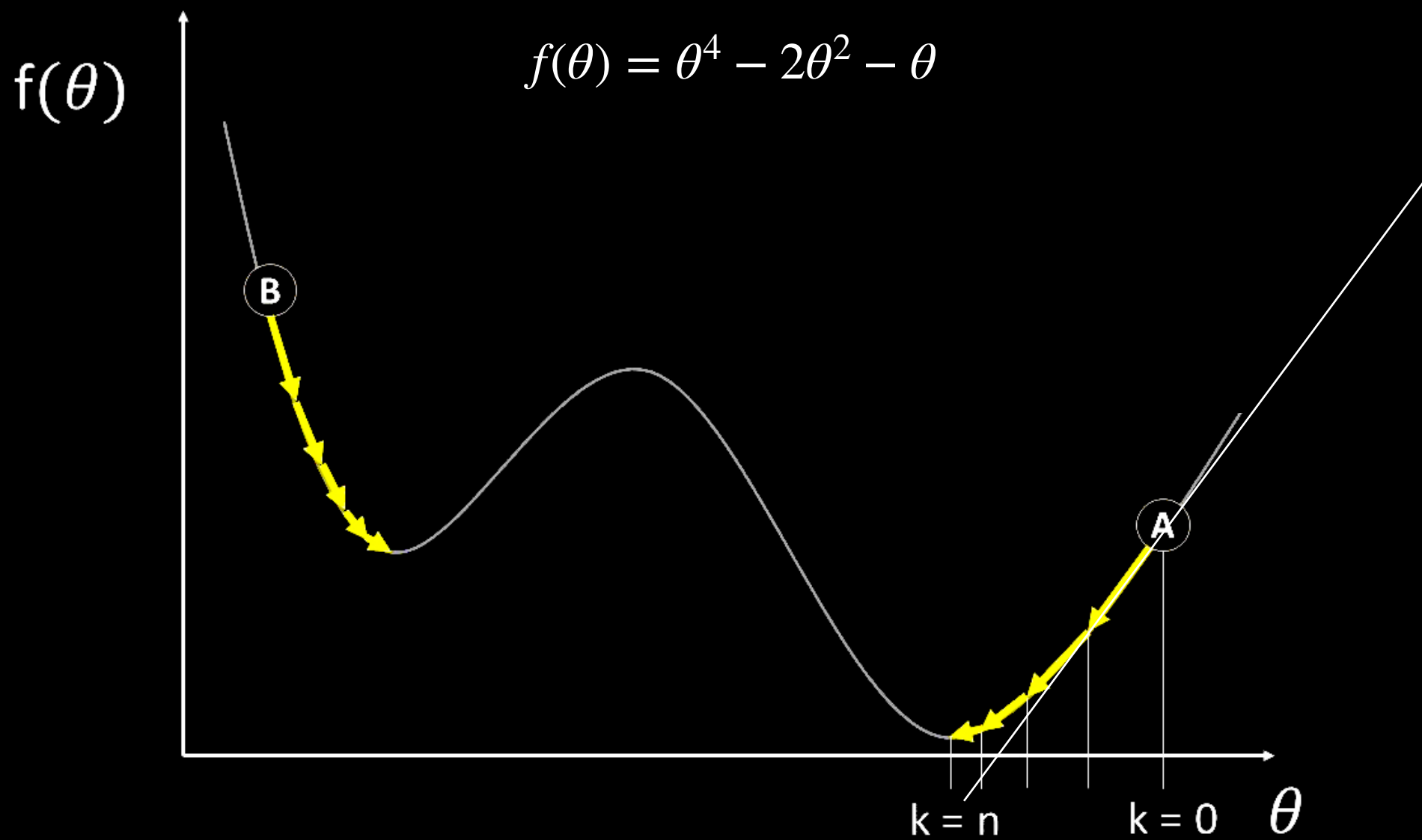
Как такое учить?

Производная и градиент

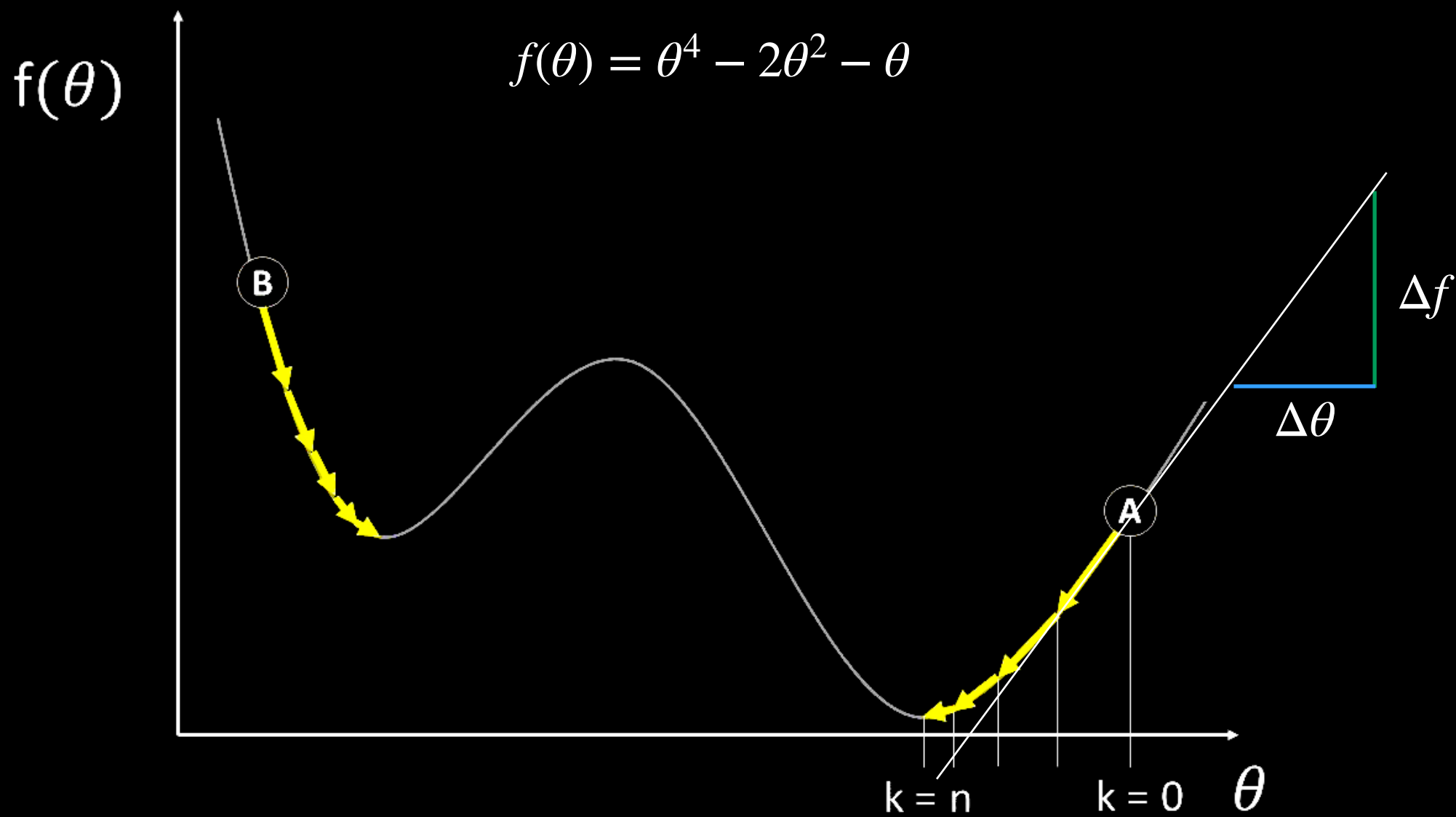
Производная и градиент



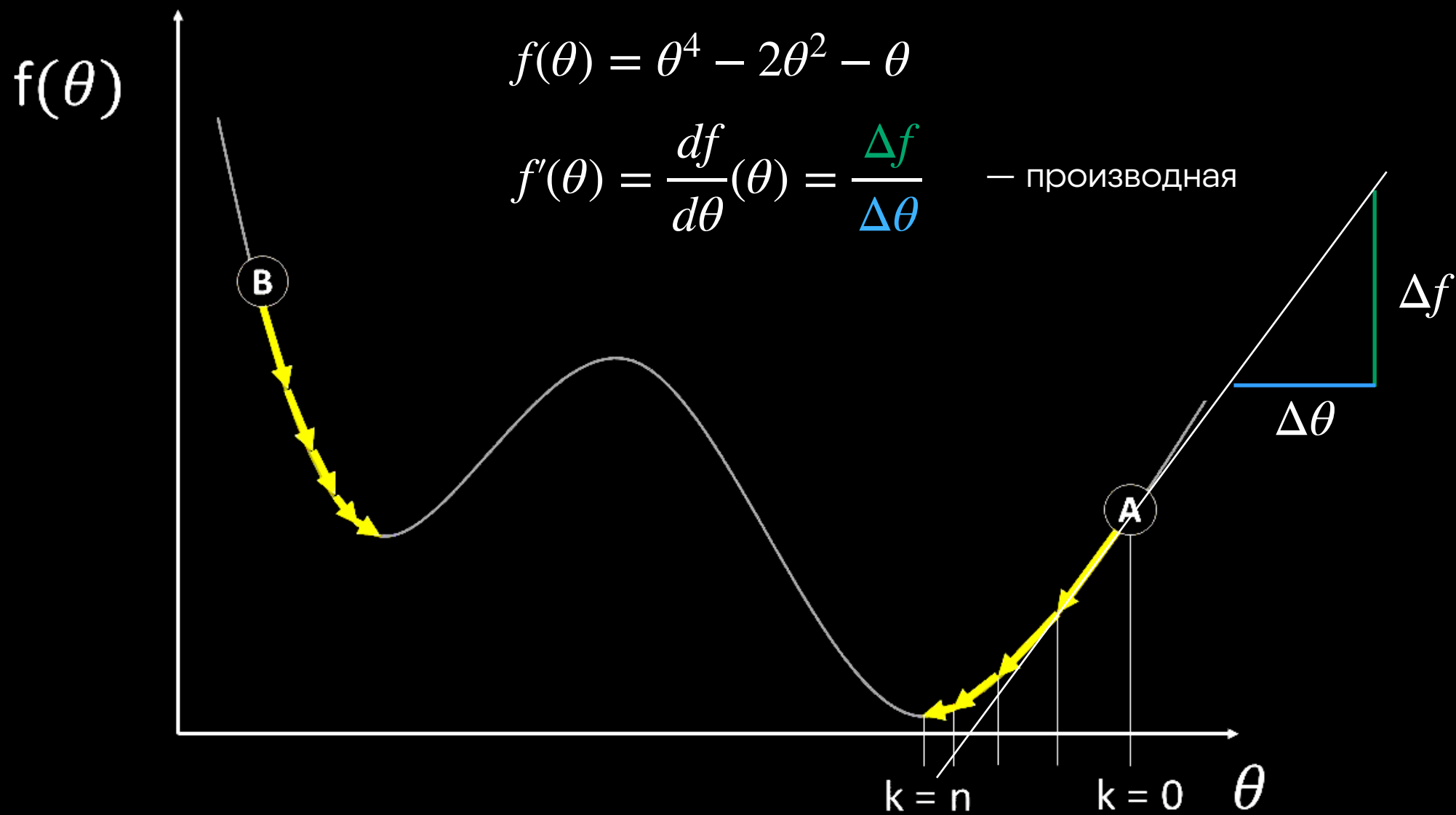
Производная и градиент



Производная и градиент

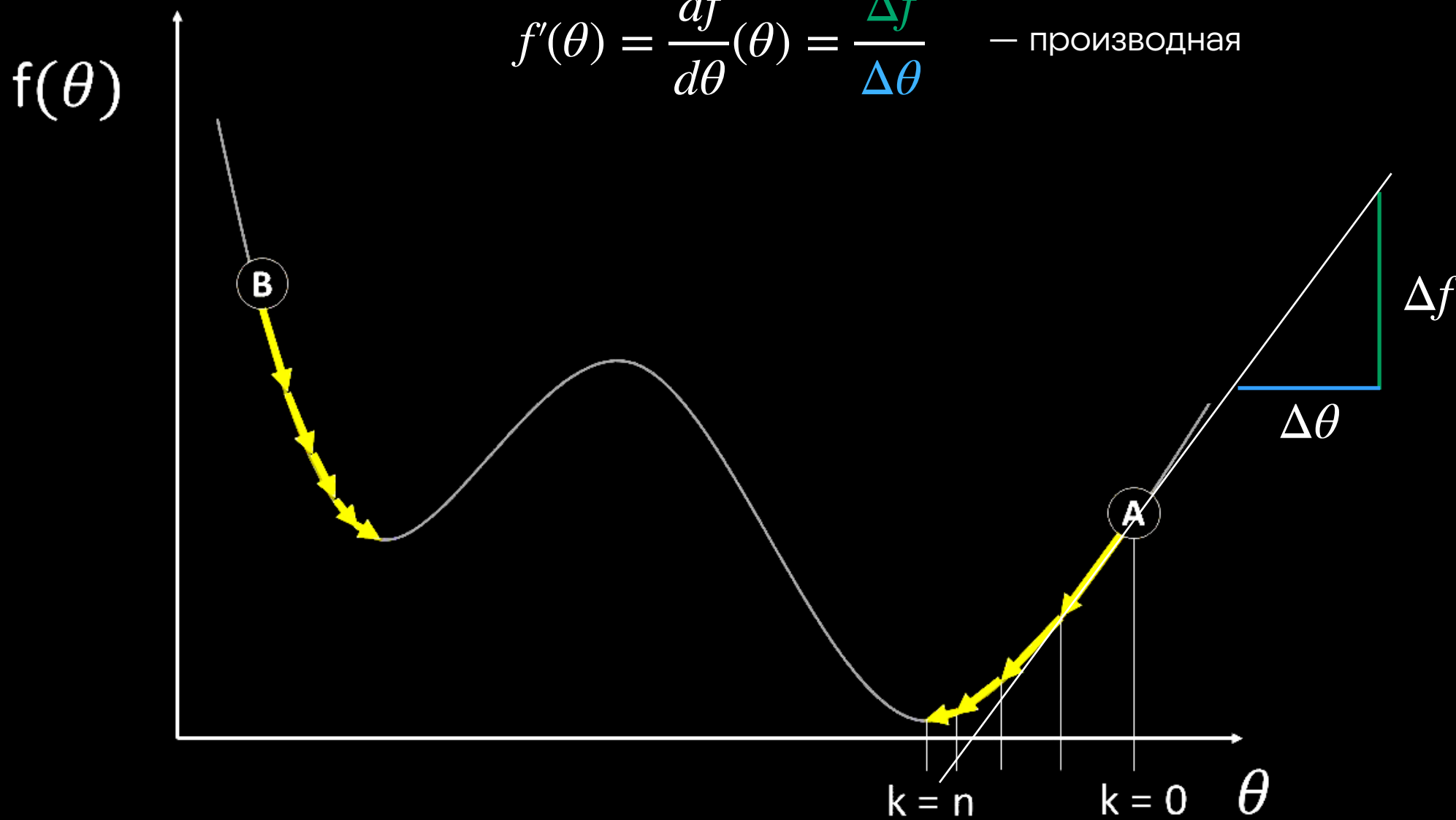


Производная и градиент

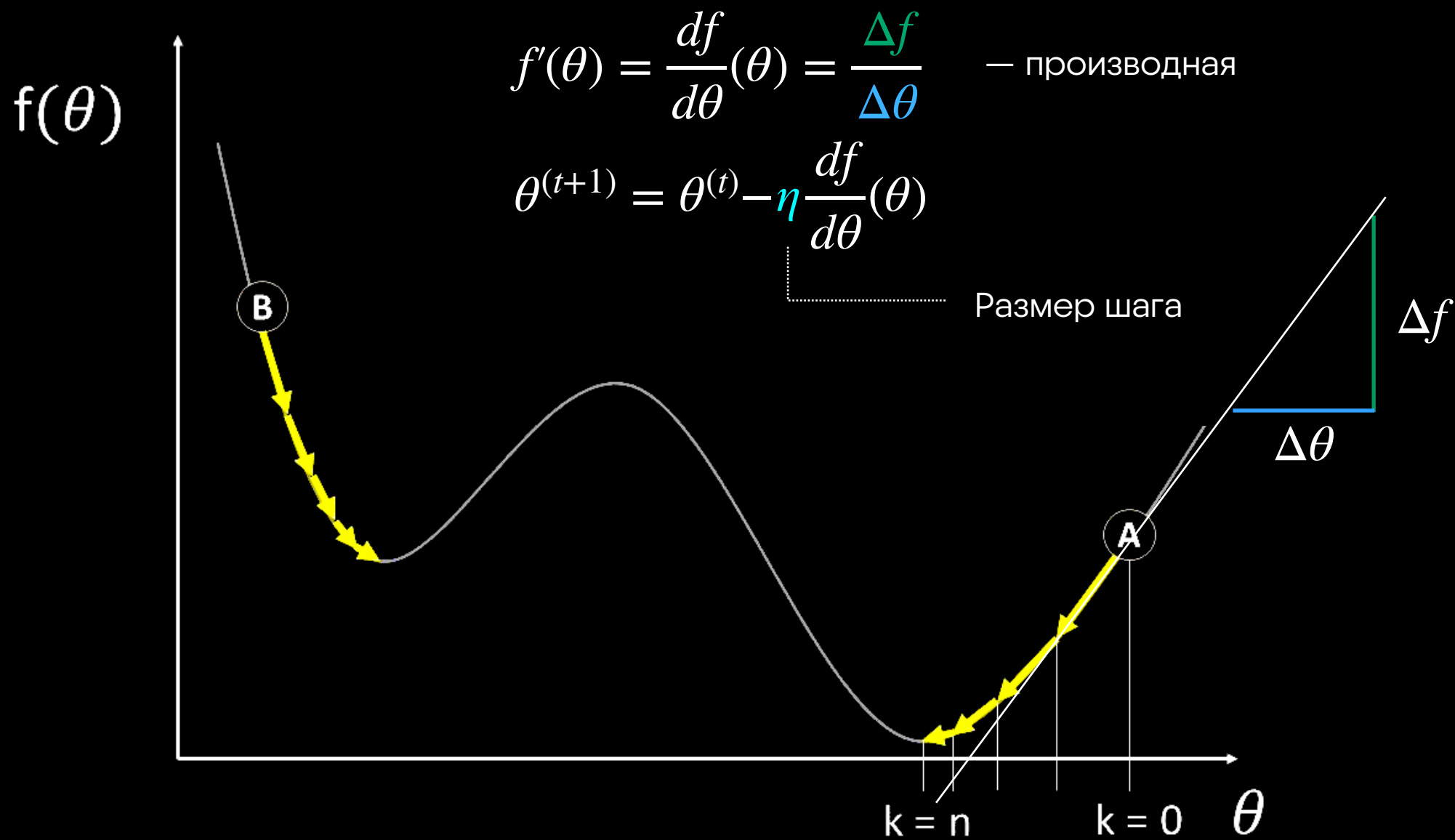


Производная и градиент

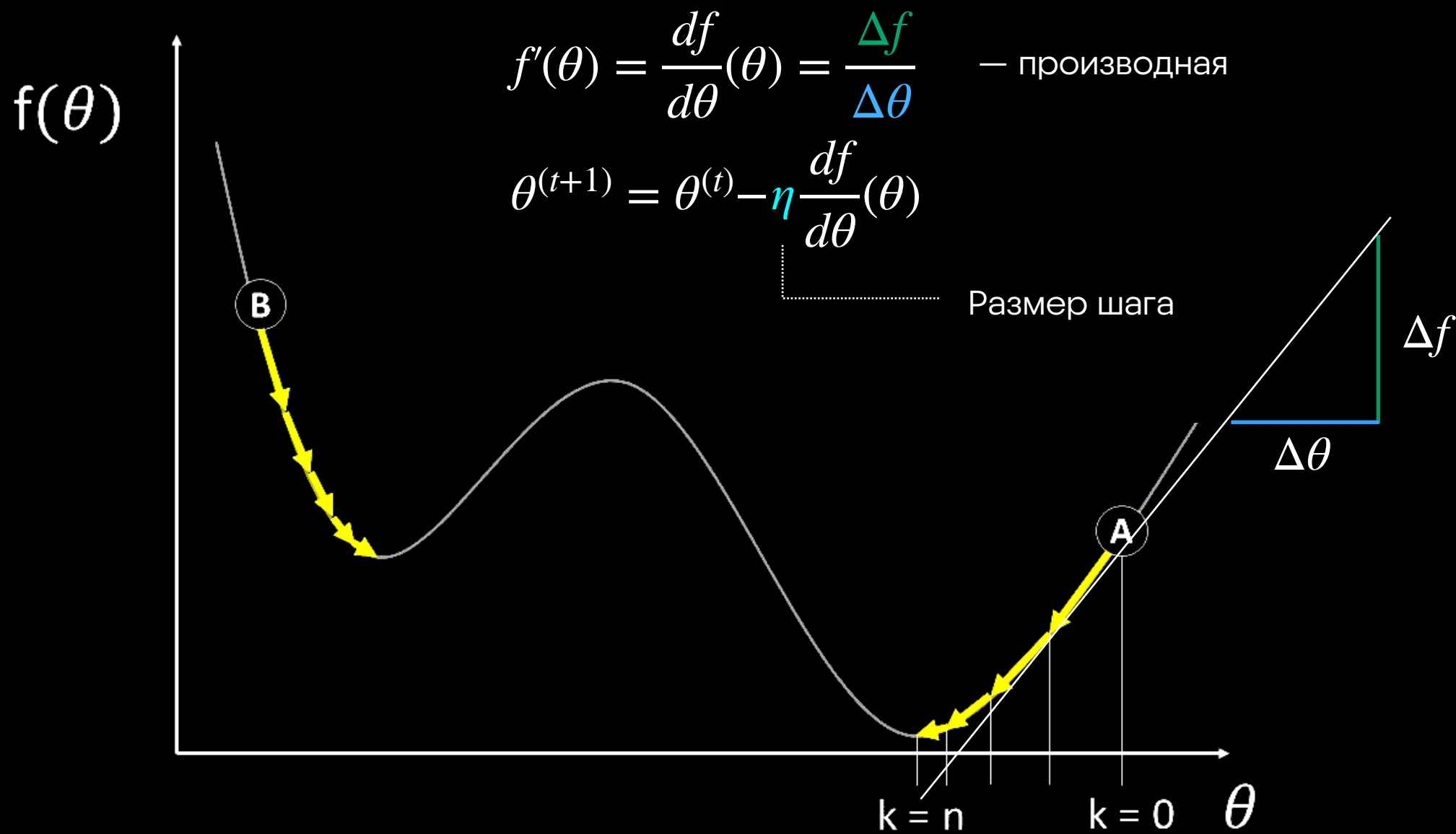
$$f'(\theta) = \frac{df}{d\theta}(\theta) = \frac{\Delta f}{\Delta \theta} \quad \text{— производная}$$



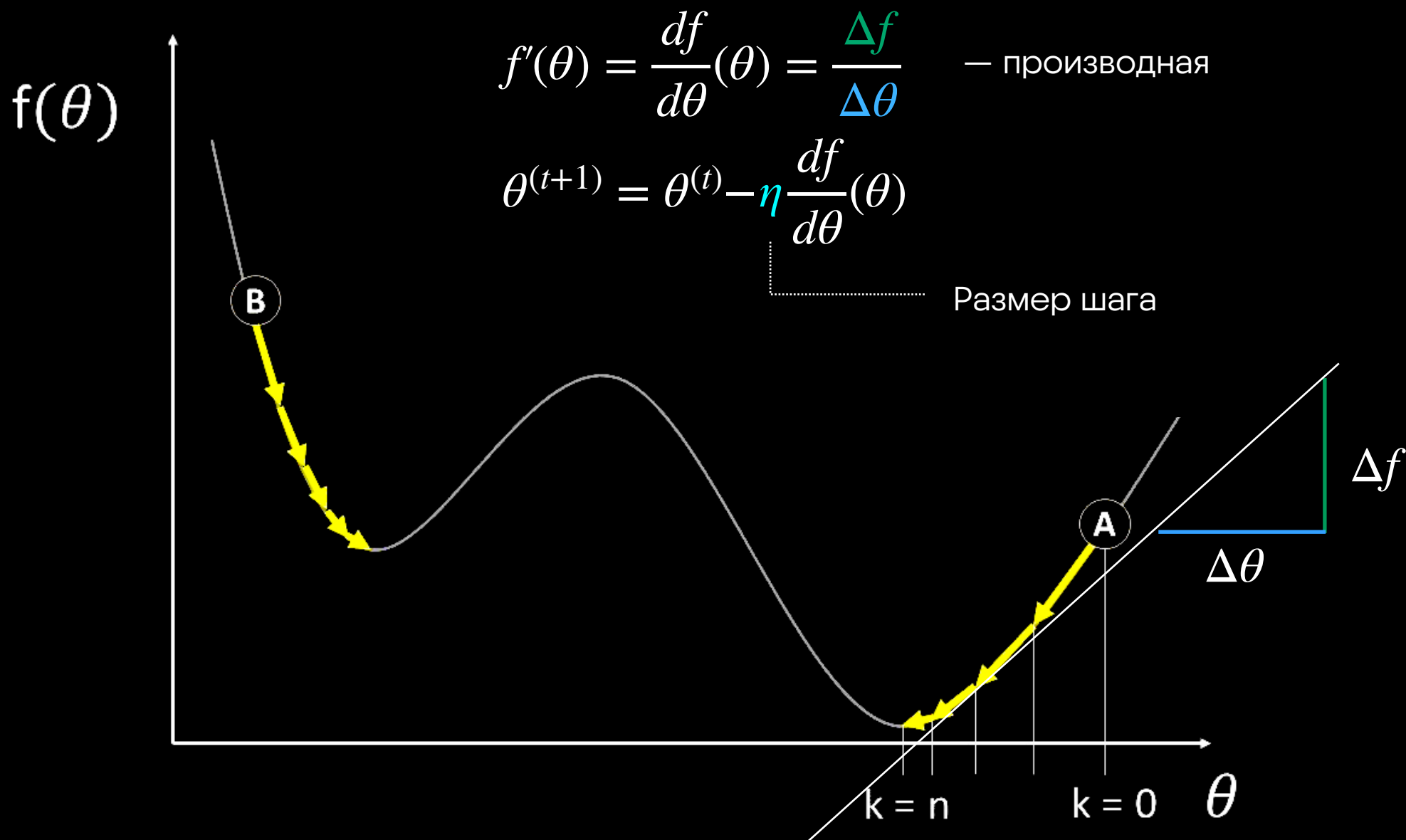
Производная и градиент



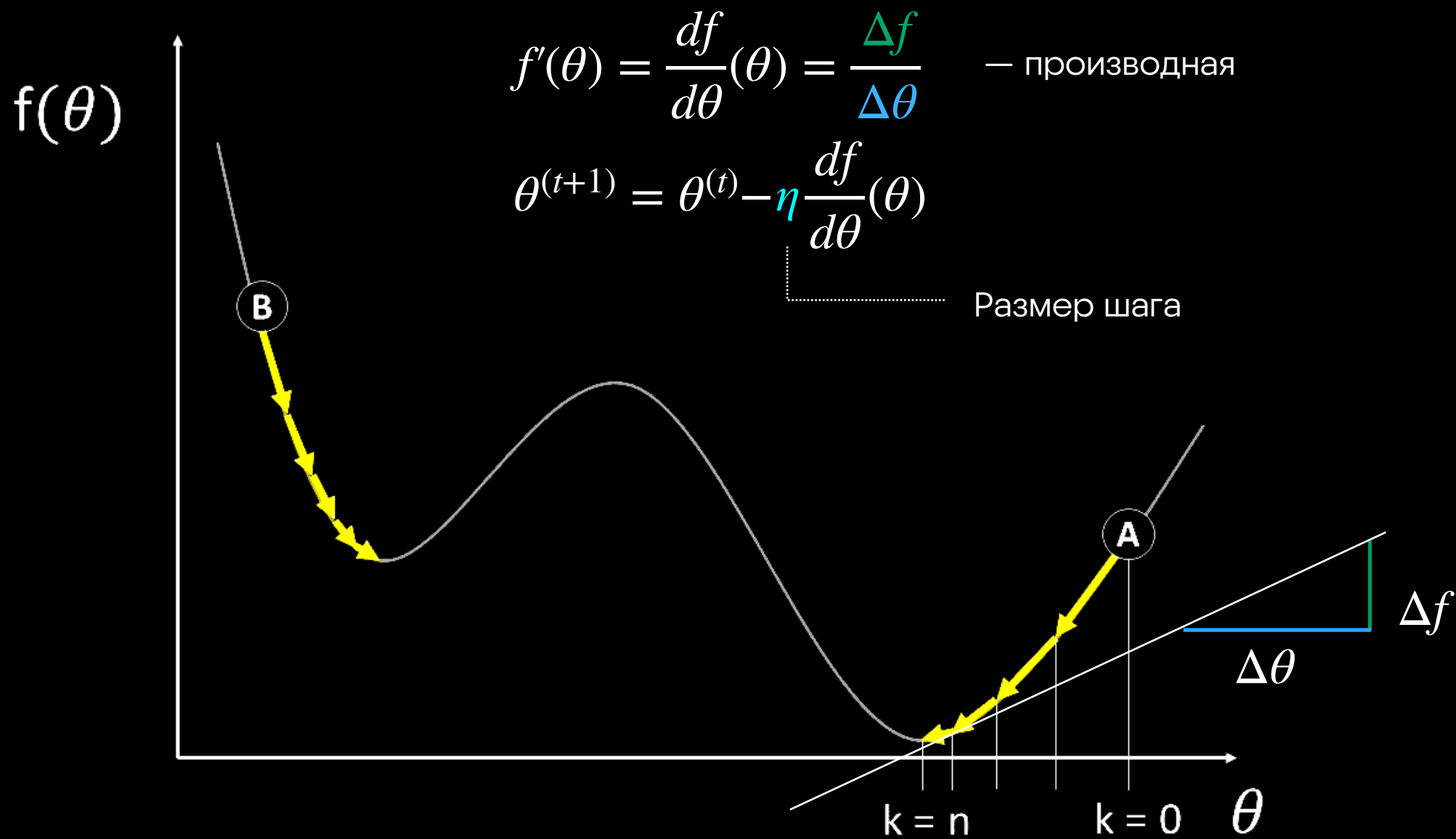
Производная и градиент



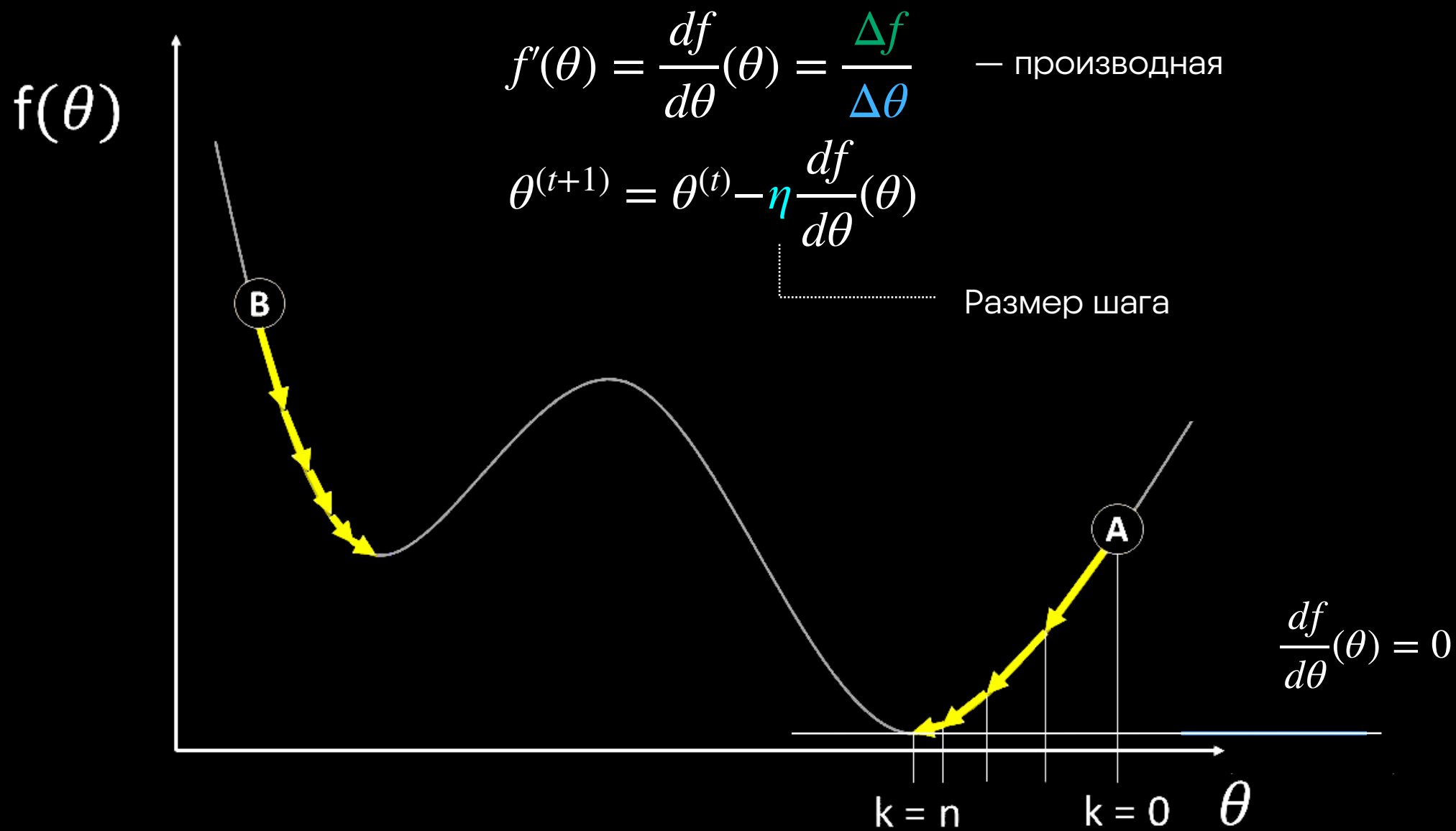
Производная и градиент



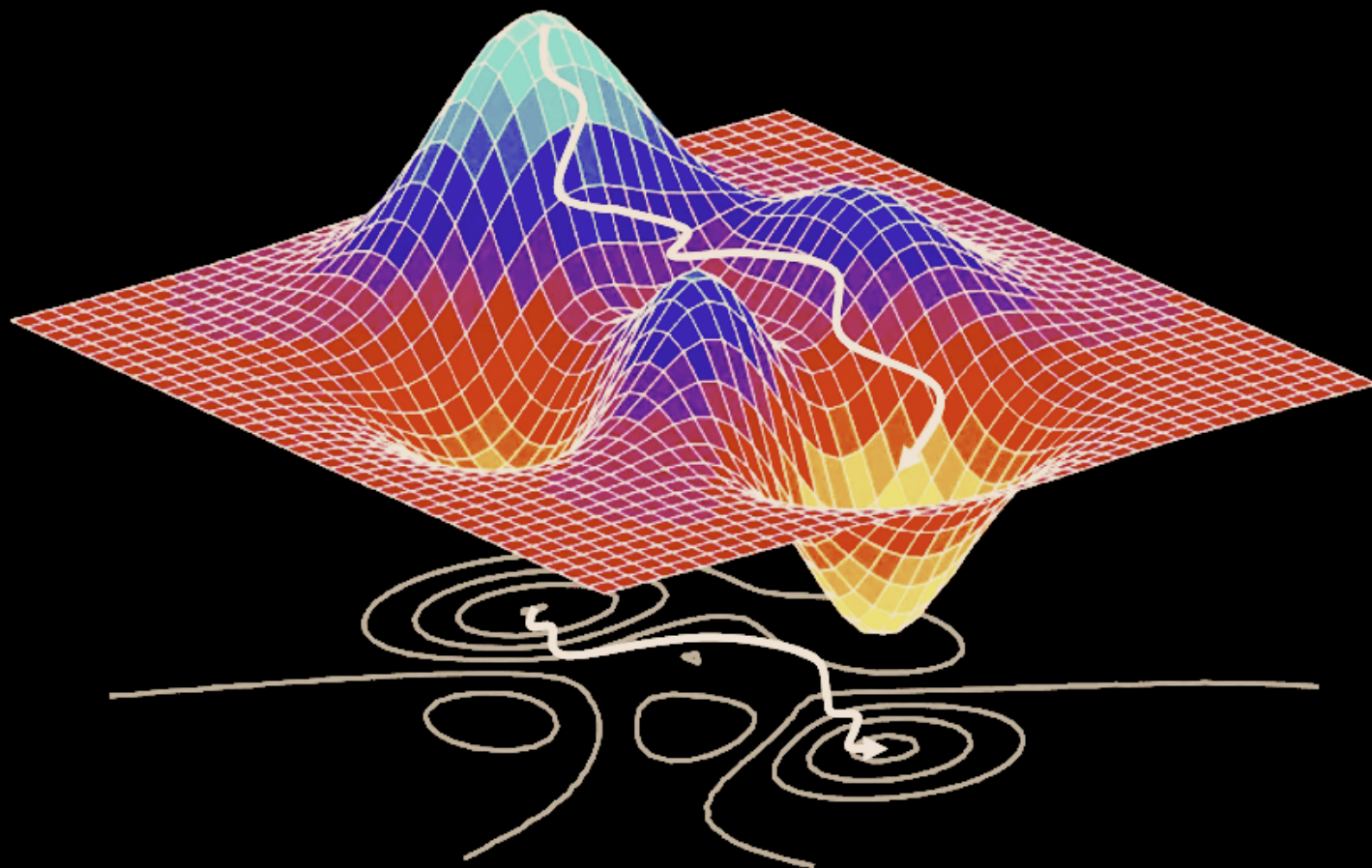
Производная и градиент



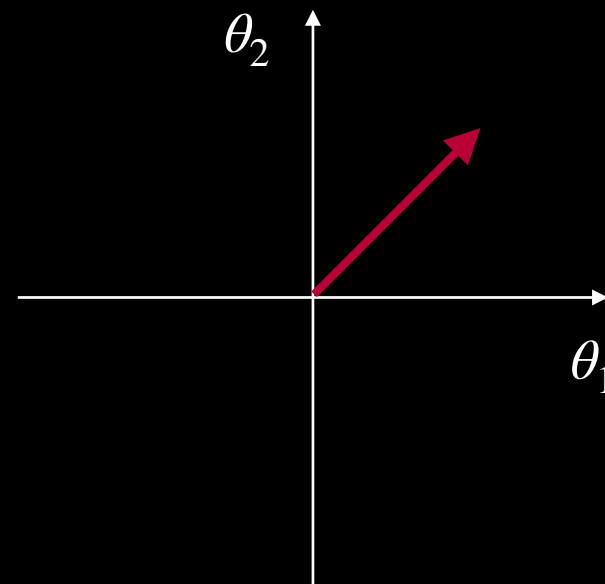
Производная и градиент



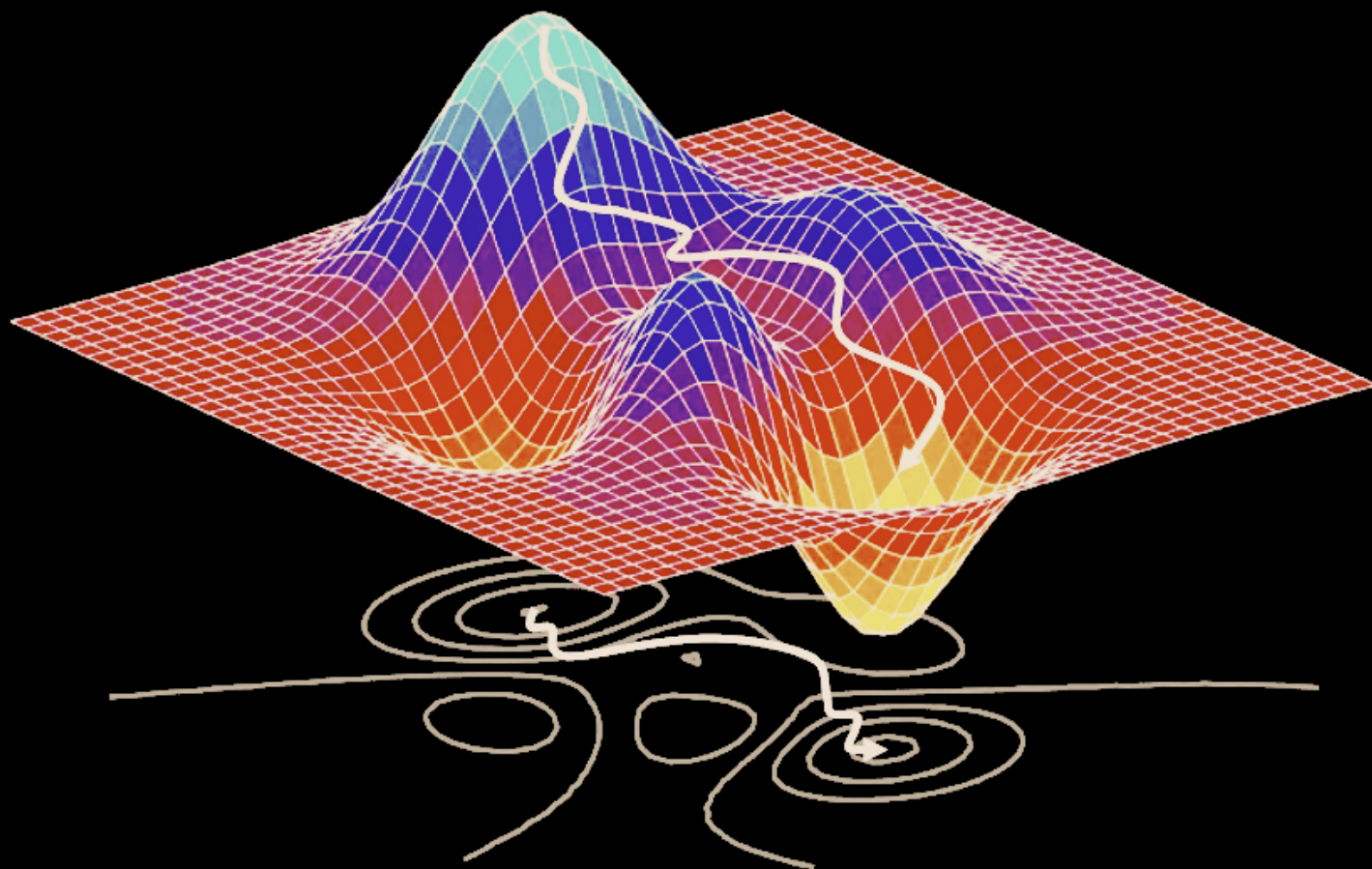
Производная и градиент



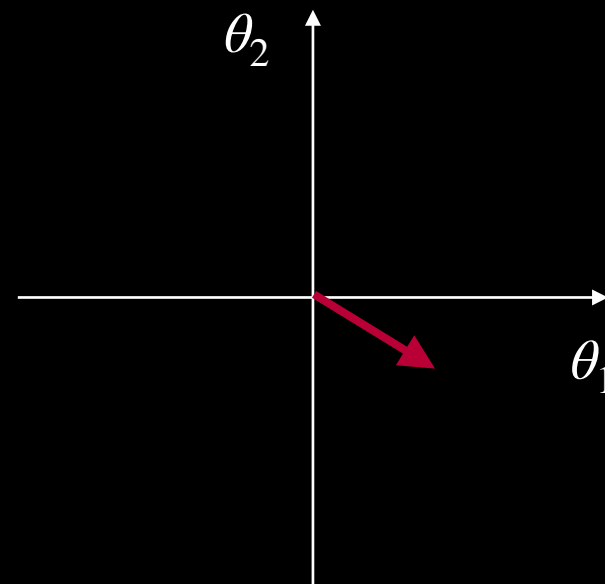
$$\nabla f(\theta_1, \theta_2) = \begin{bmatrix} \frac{\partial f}{\partial \theta_1} \\ \frac{\partial f}{\partial \theta_2} \end{bmatrix} \quad \text{— градиент}$$



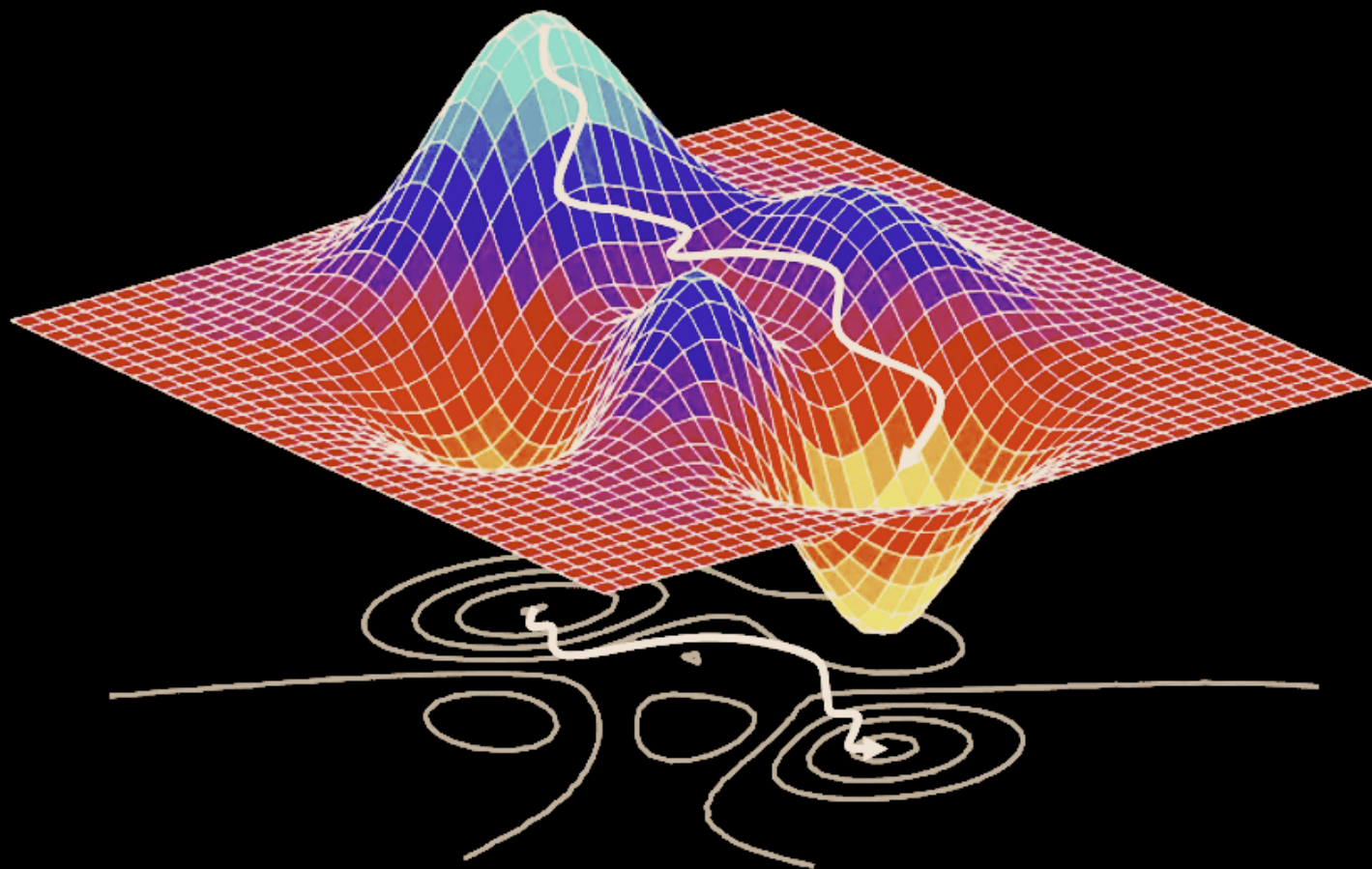
Производная и градиент



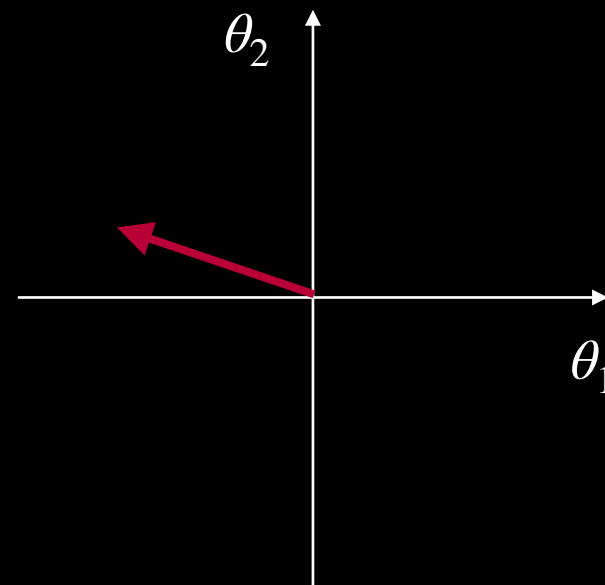
$$\nabla f(\theta_1, \theta_2) = \begin{bmatrix} \frac{\partial f}{\partial \theta_1} \\ \frac{\partial f}{\partial \theta_2} \end{bmatrix} \quad \text{— градиент}$$



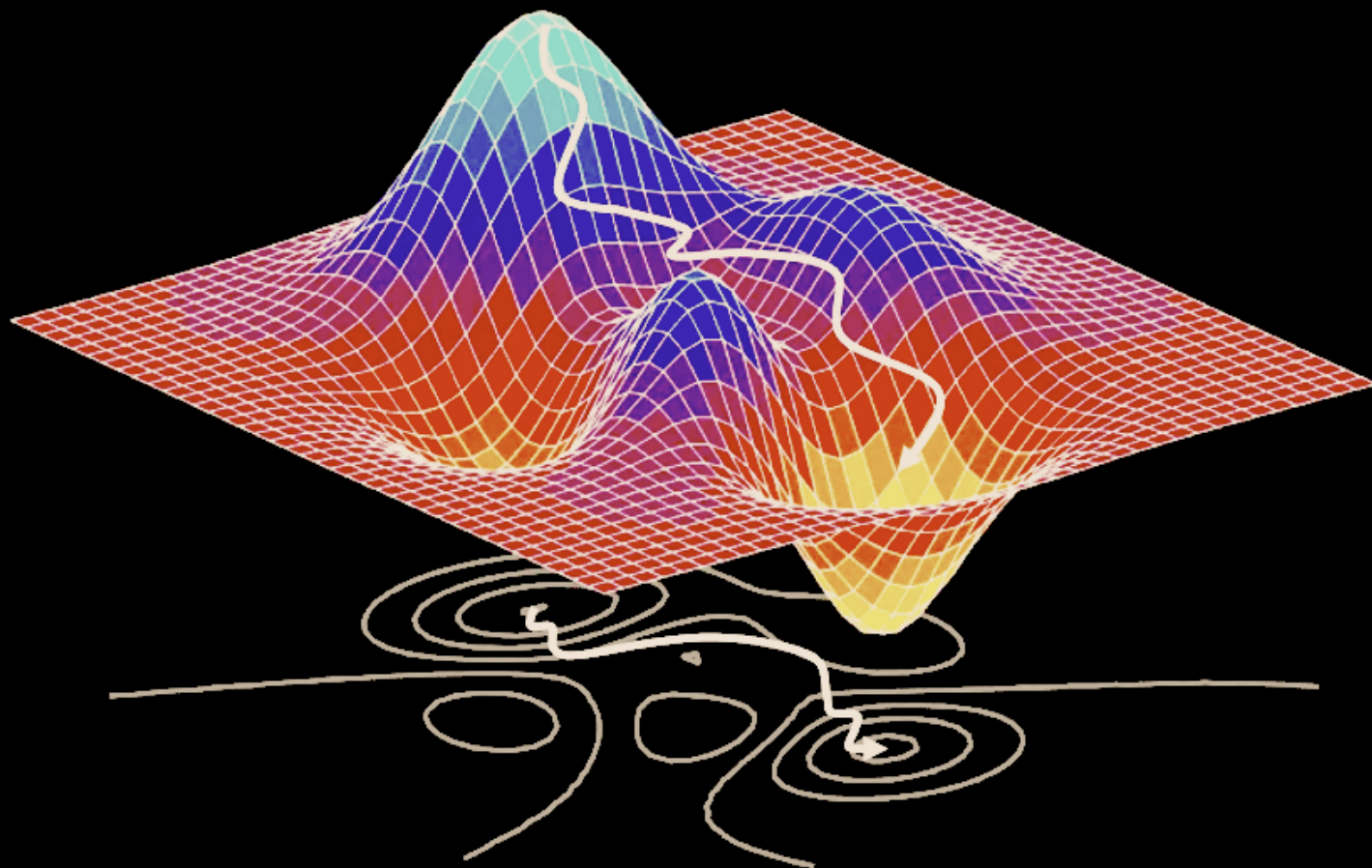
Производная и градиент



$$\nabla f(\theta_1, \theta_2) = \begin{bmatrix} \frac{\partial f}{\partial \theta_1} \\ \frac{\partial f}{\partial \theta_2} \end{bmatrix} \quad \text{— градиент}$$



Производная и градиент



$E(\mathbf{W})$ — функция, которую хотим минимизировать

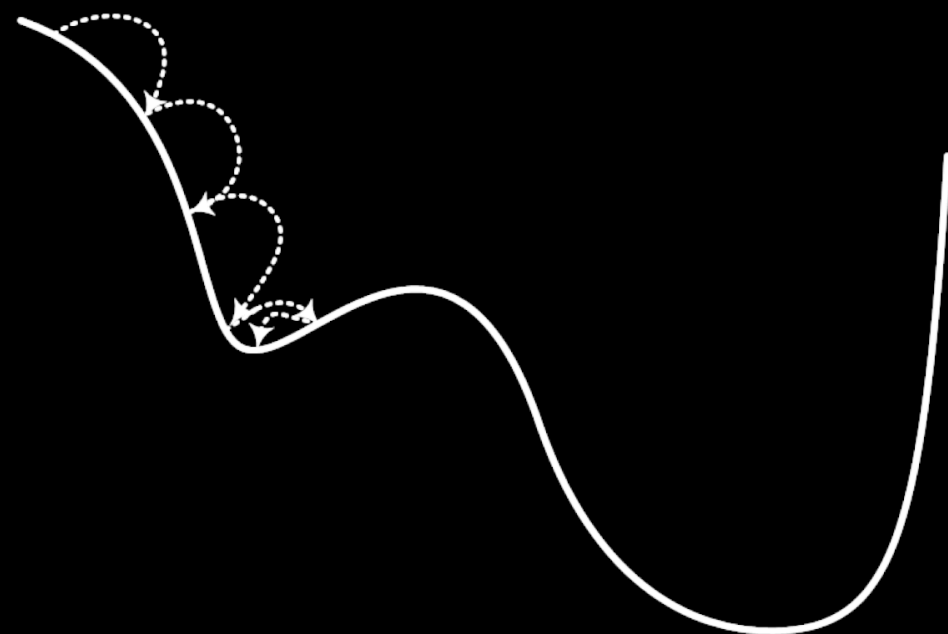
Градиентный спуск:

$$\mathbf{W}^{(t+1)} = \mathbf{W}^{(t)} - \eta \nabla E$$

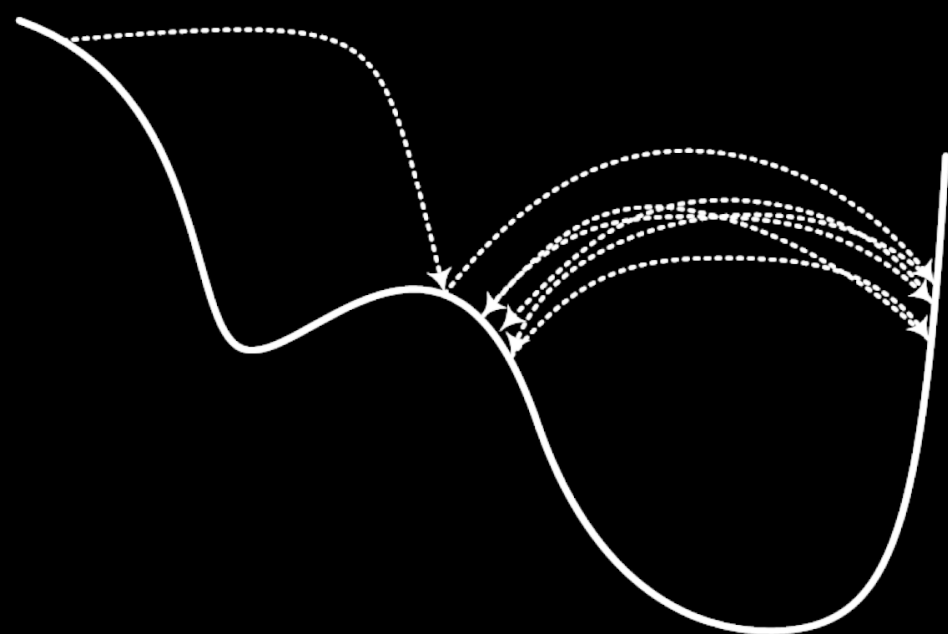
..... Градиент

└..... Размер шага

Выбор шага оптимизации

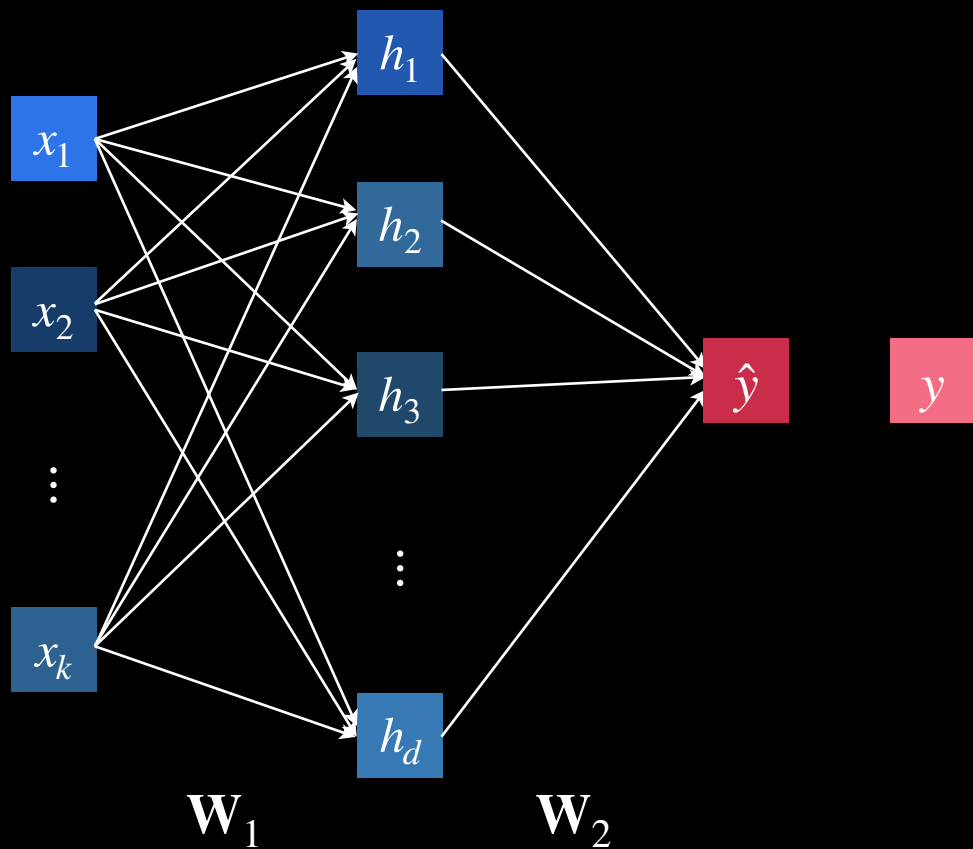


(a)



(б)

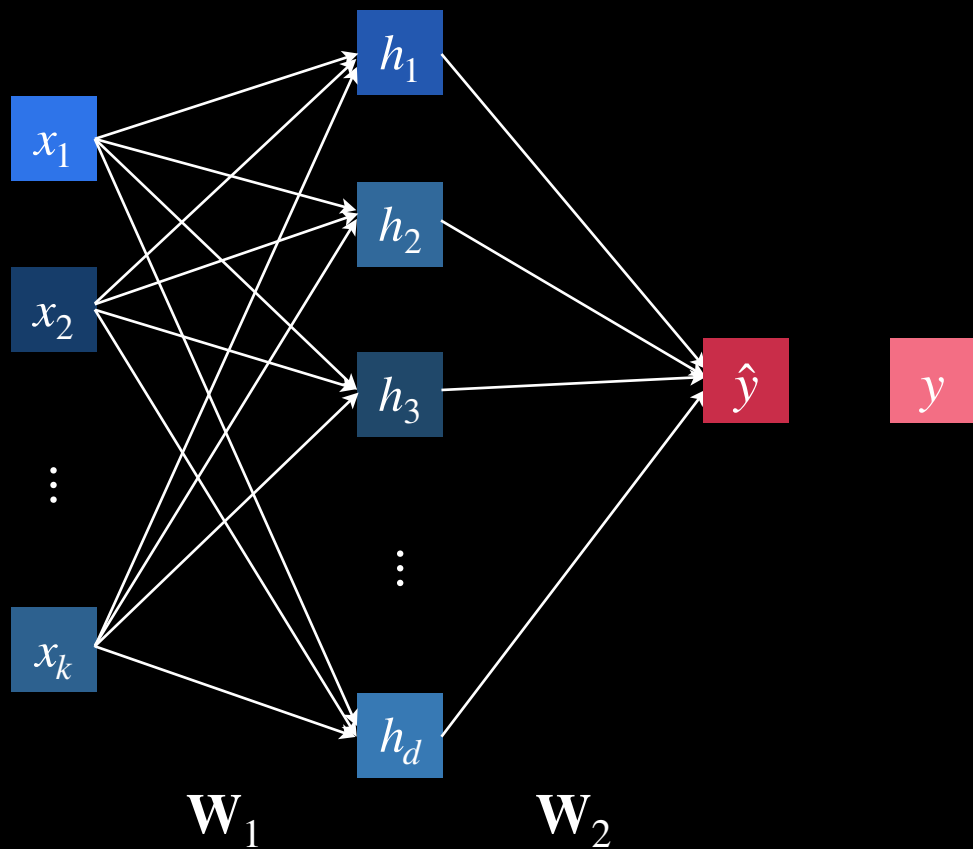
Обучение нейронных сетей



$$\mathbf{h} = f_h(\mathbf{W}_1 \mathbf{x})$$

$$\hat{y} = f_o(\mathbf{W}_2 \mathbf{h})$$

Обучение нейронных сетей



$$\mathbf{h} = f_h(\mathbf{W}_1 \mathbf{x})$$

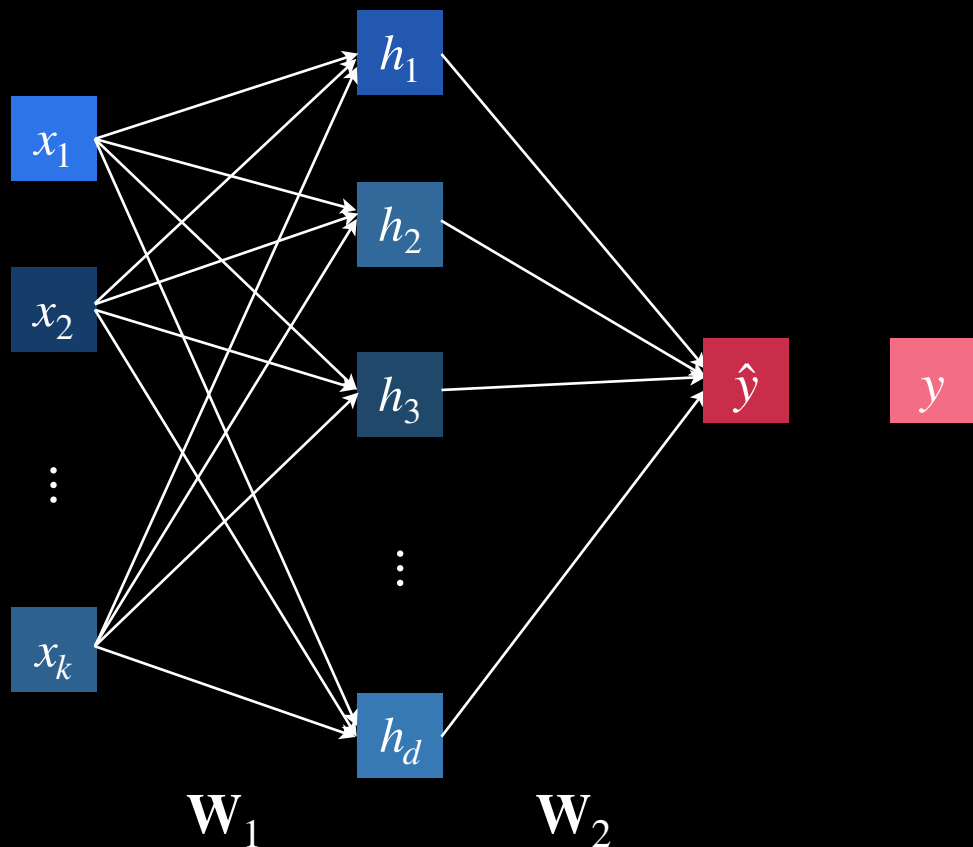
$$\hat{y} = f_o(\mathbf{W}_2 \mathbf{h})$$

$$E(\hat{y}, y)$$

.....

Функция ошибки

Обучение нейронных сетей



$$\mathbf{h} = f_h(\mathbf{W}_1 \mathbf{x})$$

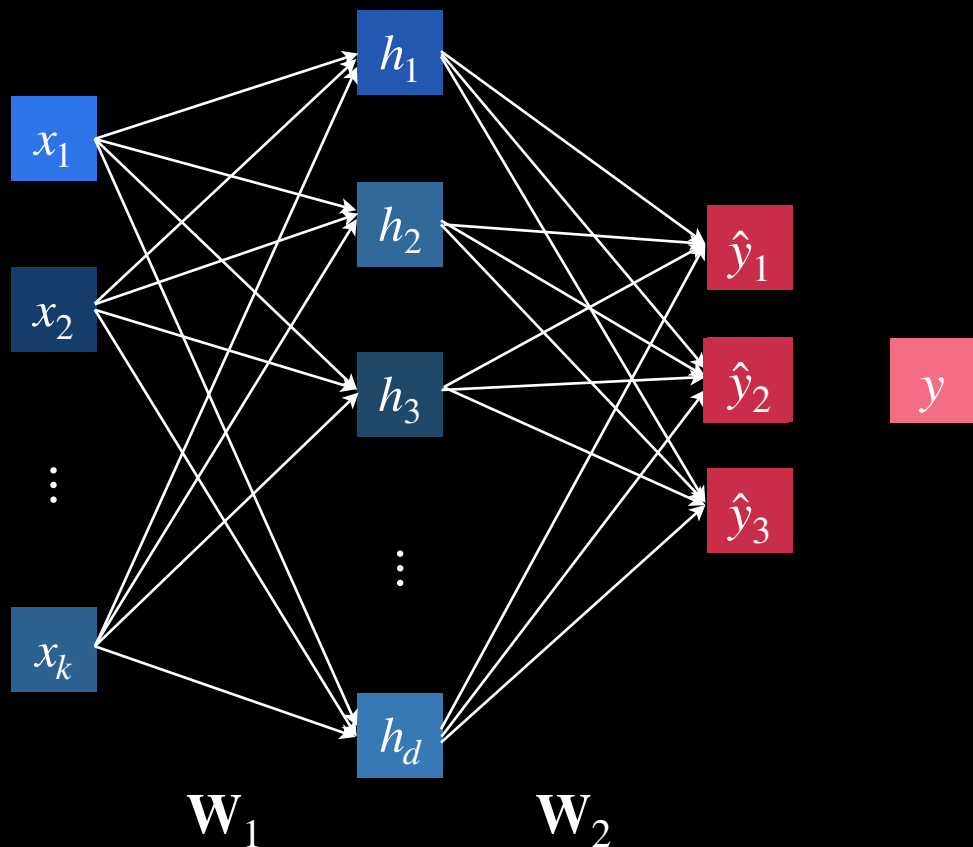
$$\hat{y} = f_o(\mathbf{W}_2 \mathbf{h})$$

$E(\hat{y}, y)$ Функция ошибки

Для задачи регрессии:

$$E(\hat{y}, y) = (\hat{y} - y)^2$$

Обучение нейронных сетей



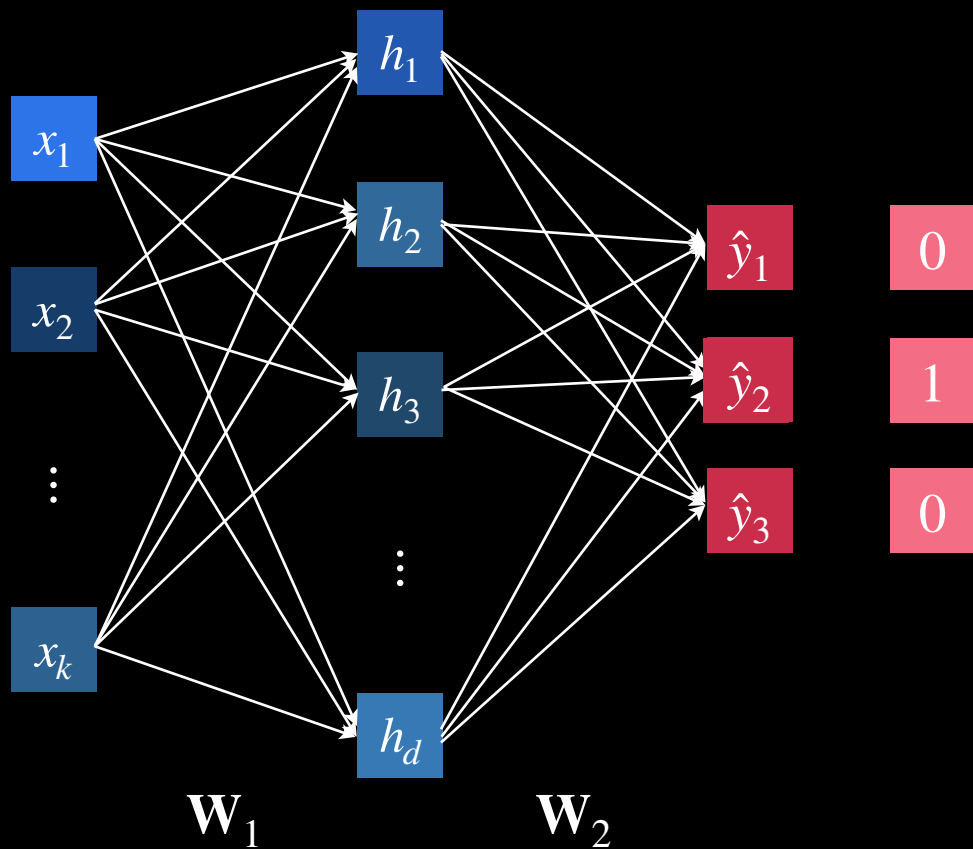
$$\mathbf{h} = f_h(\mathbf{W}_1 \mathbf{x})$$

$$\hat{\mathbf{y}} = f_o(\mathbf{W}_2 \mathbf{h})$$

$E(\hat{y}, y)$ Функция ошибки

Для задачи классификации:

Обучение нейронных сетей



$$\mathbf{h} = f_h(\mathbf{W}_1 \mathbf{x})$$

$$\hat{\mathbf{y}} = f_o(\mathbf{W}_2 \mathbf{h})$$

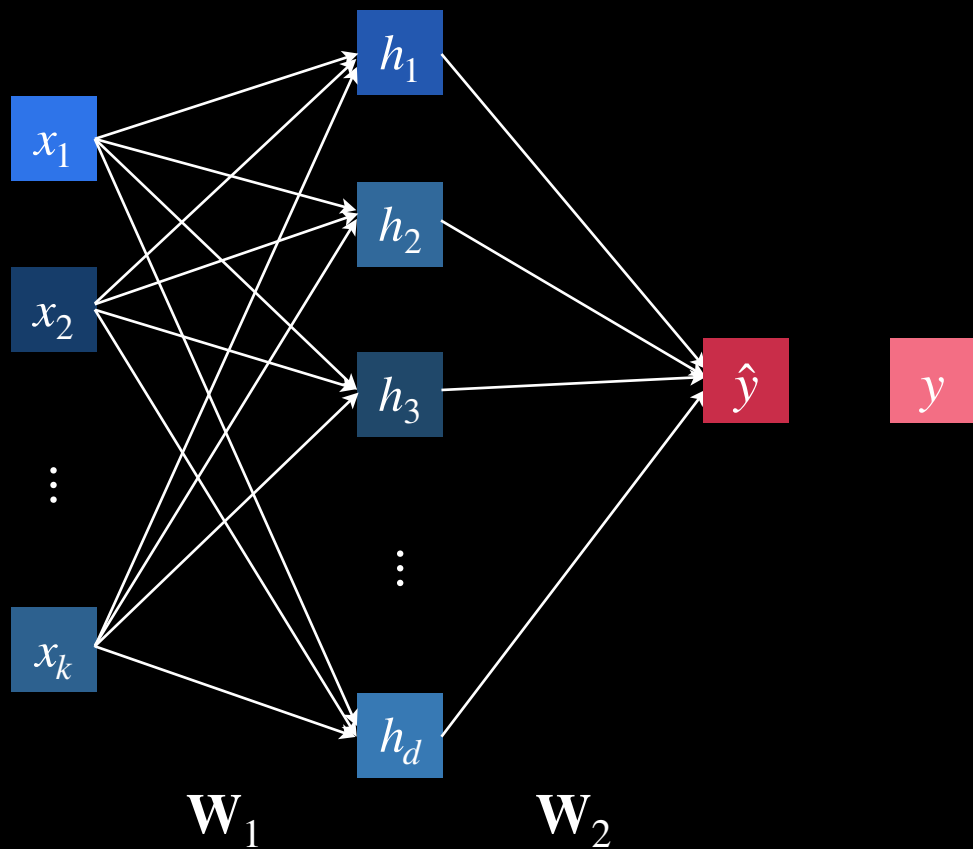
$E(\hat{\mathbf{y}}, \mathbf{y})$ Функция ошибки

Для задачи классификации:

$$E(\hat{\mathbf{y}}, \mathbf{y}) = - \sum_i y_i \ln \hat{y}_i = - \ln(\hat{y}_y)$$

Перекрёстная энтропия
(cross-entropy)

Обучение нейронных сетей



$$\mathbf{h} = f_h(\mathbf{W}_1 \mathbf{x})$$

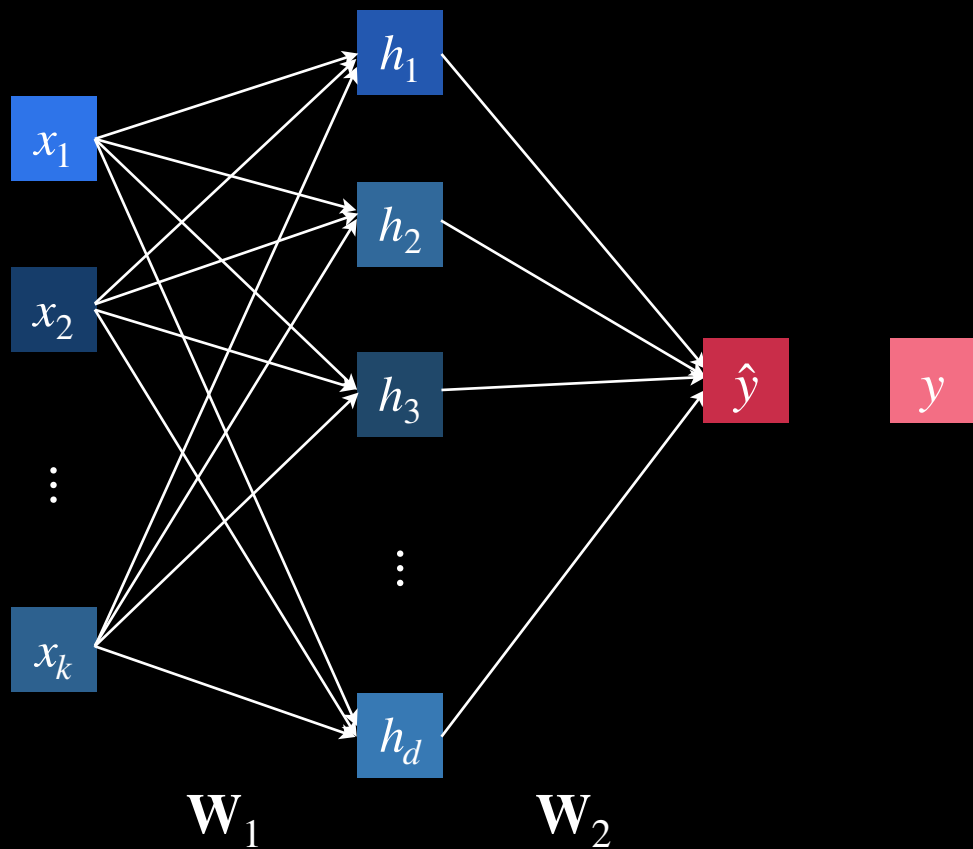
$$\hat{y} = f_o(\mathbf{W}_2 \mathbf{h})$$

$$E(\hat{y}, y)$$

.....

Функция ошибки

Обучение нейронных сетей



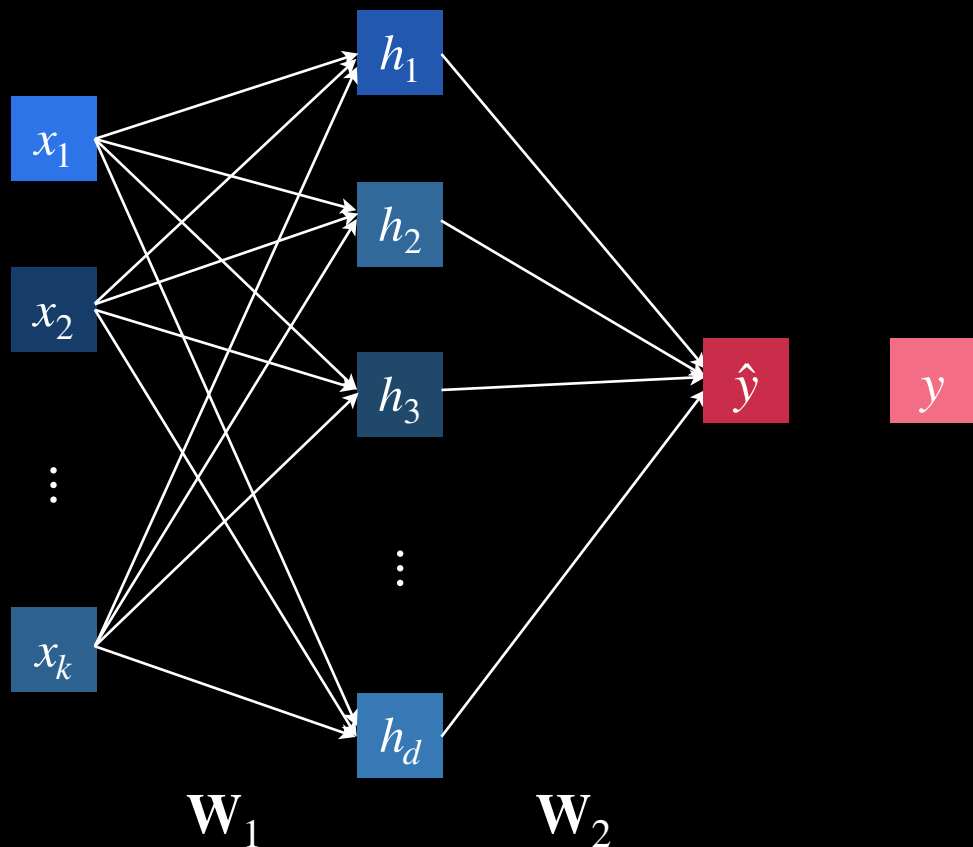
$$\mathbf{h} = f_h(\mathbf{W}_1 \mathbf{x})$$

$$\hat{y} = f_o(\mathbf{W}_2 \mathbf{h})$$

$E(\hat{y}, y)$ Функция ошибки

$$E(\hat{y}, y) \longrightarrow \min_{\mathbf{w}}$$

Обучение нейронных сетей



$$\mathbf{h} = f_h (\mathbf{W}_1 \mathbf{x})$$

$$\hat{y} = f_o (\mathbf{W}_2 \mathbf{h})$$

$E(\hat{y}, y)$ Функция ошибки

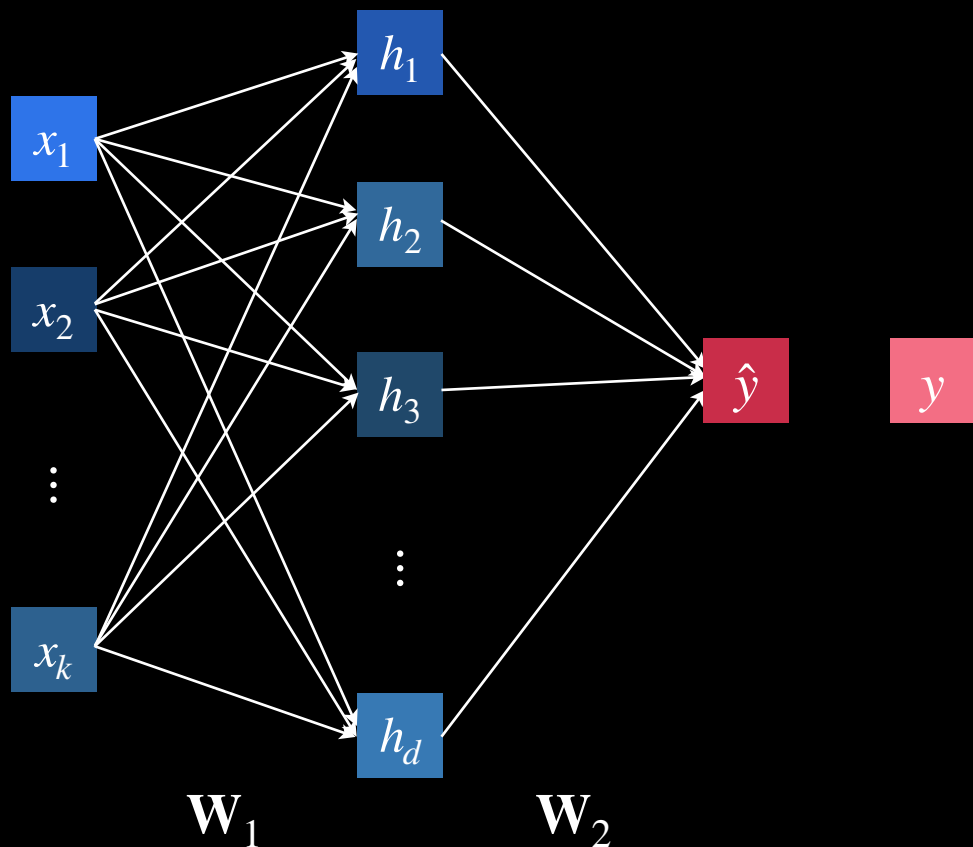
$$E(\hat{y}, y) \longrightarrow \min_{\mathbf{w}}$$

Трюк:

Функции активации — дифференцируемые

Функция ошибки — тоже

Обучение нейронных сетей



$$\mathbf{h} = f_h (\mathbf{W}_1 \mathbf{x})$$

$$\hat{y} = f_o (\mathbf{W}_2 \mathbf{h})$$

$E(\hat{y}, y)$ Функция ошибки

$$E(\hat{y}, y) \longrightarrow \min_{\mathbf{w}}$$

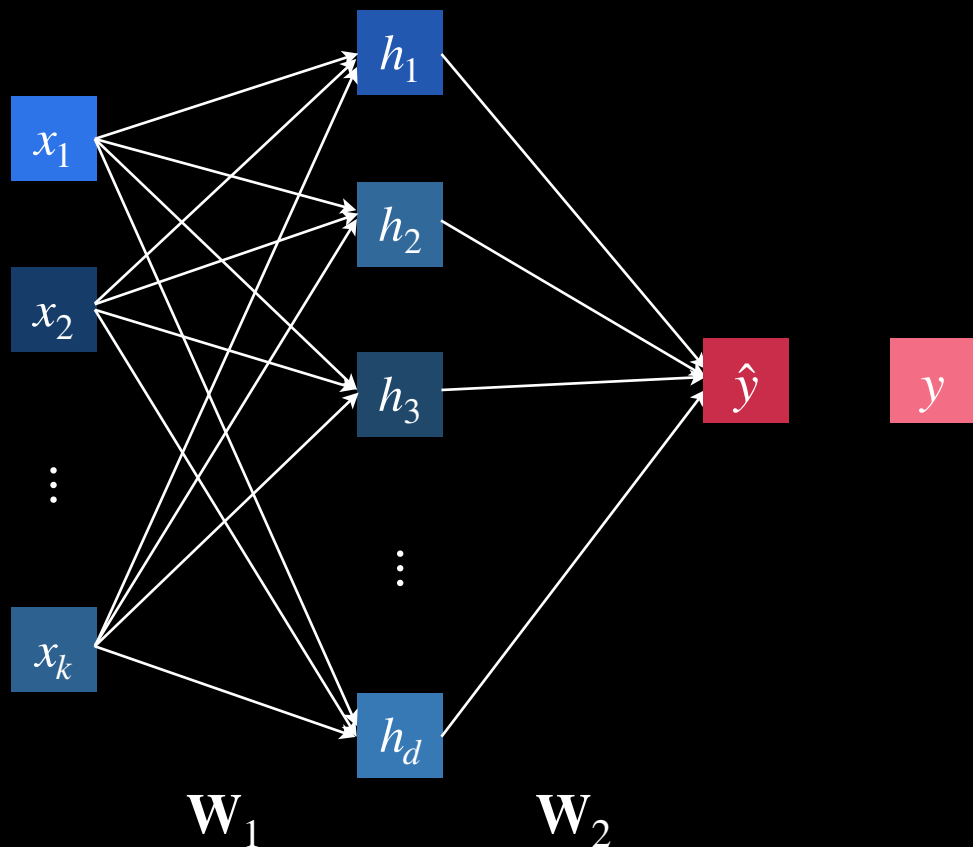
Трюк:

Функции активации — дифференцируемые

Функция ошибки — тоже

Значит можно посчитать производную ошибки по каждому из параметров!

Обучение нейронных сетей



$$\mathbf{h} = f_h(\mathbf{W}_1 \mathbf{x})$$

$$\hat{y} = f_o(\mathbf{W}_2 \mathbf{h})$$

$E(\hat{y}, y)$ Функция ошибки

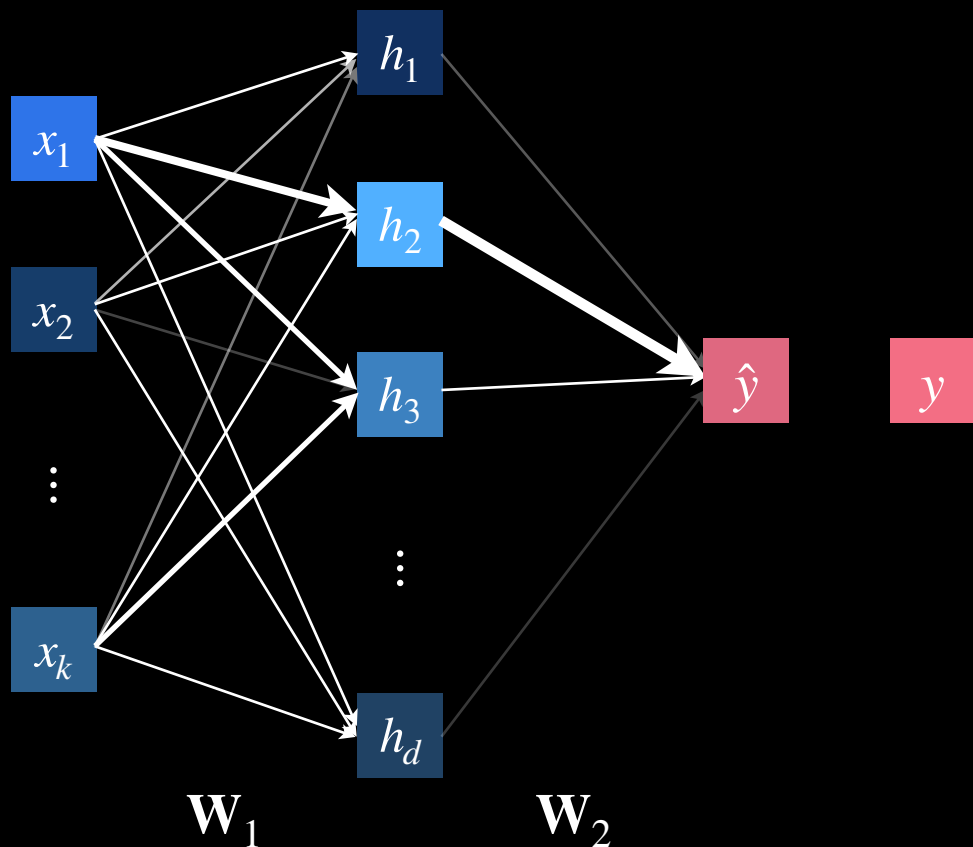
Градиентный спуск:

$$\mathbf{W}^{(t+1)} = \mathbf{W}^{(t)} - \eta \nabla E$$

..... Градиент

└..... Размер шага

Обучение нейронных сетей



$$\mathbf{h} = f_h(\mathbf{W}_1 \mathbf{x})$$

$$\hat{y} = f_o(\mathbf{W}_2 \mathbf{h})$$

$E(\hat{y}, y)$ Функция ошибки

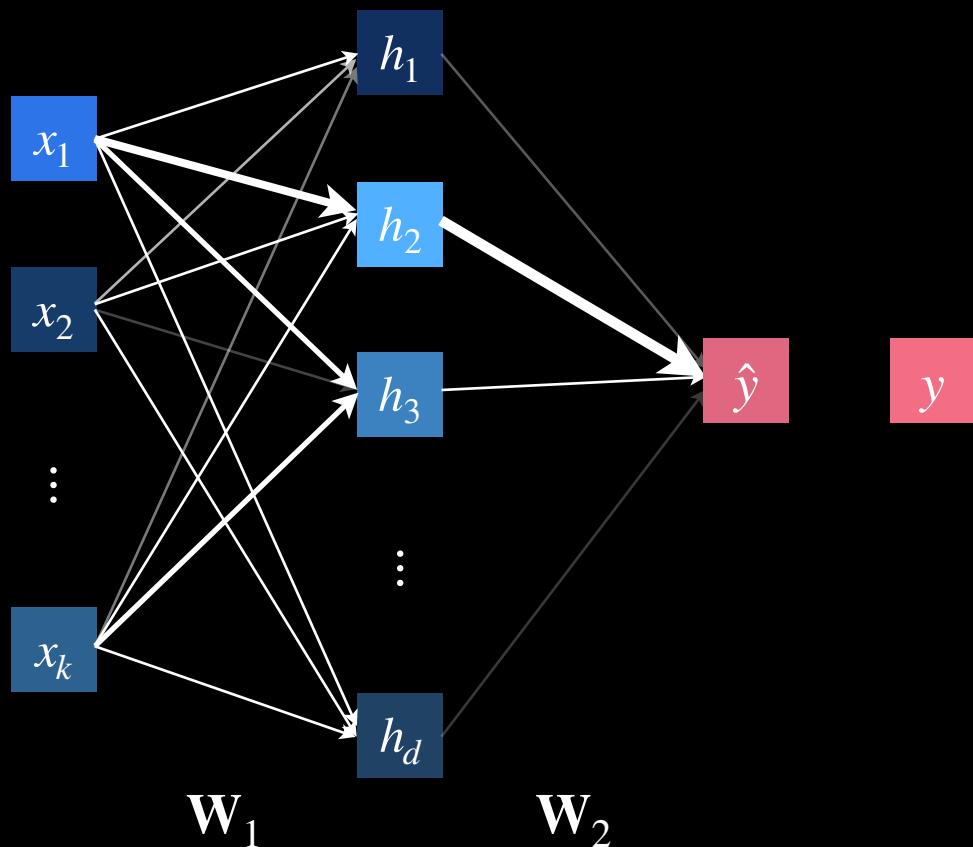
Градиентный спуск:

$$\mathbf{W}^{(t+1)} = \mathbf{W}^{(t)} - \eta \nabla E$$

..... Градиент

..... Размер шага

Обучение нейронных сетей



$$\mathbf{h} = f_h(\mathbf{W}_1 \mathbf{x})$$

$$\hat{y} = f_o(\mathbf{W}_2 \mathbf{h})$$

$E(\hat{y}, y)$ Функция ошибки

Градиентный спуск:

$$\mathbf{W}^{(t+1)} = \mathbf{W}^{(t)} - \eta \nabla E$$

..... Градиент

.....
Размер шага

Градиенты за нас будет считать Pytorch

Дифференцируемое программирование

```
import torch
from torch import nn, Tensor
from torch.optim import Optimizer

def training_step(
    features: Tensor,                # входные признаки
    target: Tensor,                  # правильные ответы
    model: nn.Module,                # модель с обучаемыми параметрами
    loss_fn: tuple[Tensor, Tensor] -> Tensor, # функция ошибки
    optimizer: Optimizer,            # оптимизатор для обновления параметров
):
    prediction = model.forward(features) # прямой проход: вычисление предсказаний
    loss = loss_fn(prediction, target)    # расчёт ошибки
    loss.backward()                       # обратный проход: вычисление градиентов
    optimizer.step()                     # обновление параметров модели
```